

IMPERIAL COLLEGE LONDON

**Department of Mechanical Engineering
Advanced Mechanical Engineering Course**

**Using Machine Learning to Perform Automated Segmentation of
Brain White Matter**

By

**David Viertel
BSc Physics, TU Vienna**

A thesis submitted in partial fulfilment of the requirements for the Master of Science Degree of
Imperial College London and the Diploma of Membership of Imperial College

September 2024

Word limit: 18,000
Word Count: 15,093

Abstract

Convection Enhanced Delivery of drug molecules in the brain could enable more effective treatment of neurological conditions. However, modelling fluid flow in the brain is difficult without accurate models of the White Matter (WM) microstructure. Such models can be created by reconstructing segmented images, which can either be segmented manually or automatically. Manual segmentation is extremely time consuming, so developing an automatic technique for WM segmentation would be advantageous.

Multiple different Deep Convolutional Neural Network (DCNN) architectures are first validated on an idealised WM dataset and a rat WM dataset. For the latter, the segmentation results are compared with a state-of-the-art DCNN from literature trained on the same dataset. After validation, the DCNNs are trained using two manually segmented sheep WM datasets from the fornix in the brain. The results of these are iteratively improved on by using various preprocessing techniques.

Due to large intensity discrepancies in the datasets, the DCNNs trained initially were not successful. Binarisation and intensity remapping were both attempted as solutions for the intensity discrepancy. However, they revealed another flaw in the datasets, namely the different levels of clutter (i.e., unlabelled structures which resemble myelin-axon pairs). Using a DCNN trained on a high-clutter dataset to segment a low-clutter dataset worked reasonably well. However, the reverse situation does not work. This is because the network is overfitting on the low-clutter dataset and learns to simply segment everything that is similar to myelin-axon pairs. A final preprocessing technique was devised to declutter datasets, in order to confirm that the poor segmentation results were due to significant clutter. This massively improved the segmentation results, indicating that the clutter was responsible.

Although decluttering datasets is not a feasible preprocessing technique for general use, as it requires the ground truth segmentation, the findings reinforce that DCNNs need the training and testing datasets to be of similar structure. The implications of this are that it may be more difficult than anticipated to train a DCNN for automatic sheep WM segmentation, as two samples from the same region of the brain were already dissimilar enough to inhibit training.

Acknowledgements

I would like to thank my supervisor, Professor Daniele Dini, for the guidance on this project throughout the year. He always faced every obstacle in the project with positivity and offered invaluable advice when I was not sure how to proceed.

I would also like to thank my co-supervisor, Dr. Tian Yuan, for the immense amount of support offered this last year. He was always happy to have a call or meeting to help me with any concerns I would have and has also given invaluable feedback on my first draft of this thesis.

Contents

Abstract	ii
Acknowledgements	iii
Contents	vi
List of Figures	xi
List of Tables	xii
1 Introduction	1
1.1 Motivation	1
1.2 Objectives	2
2 Literature Review	3
2.1 Image Segmentation in Medical Research	4
2.1.1 Imaging Techniques	4
2.1.2 Manual versus Automatic Segmentation	5
2.1.3 State-of-the-Art Segmentation Techniques	5
2.2 The Building Blocks of a DCNN	7
2.2.1 The Convolutional Layer	7
2.2.2 Downsampling	8
2.2.3 Fully Connected Layers	9

2.3	DCNN Architectures for Image Segmentation	10
2.3.1	Fully Convolutional Network	10
2.3.2	ResNet	11
2.3.3	UNet	11
2.3.4	UNet 3+	12
3	Datasets and Methodology	14
3.1	Datasets	17
3.1.1	Idealised White Matter Dataset	18
3.1.2	Rat White Matter Dataset	19
3.1.3	Sheep White Matter Dataset	20
3.2	Creating the Training Labels	22
3.2.1	Idealised Dataset	22
3.2.2	Rat White Matter	22
3.2.3	Sheep White Matter	23
3.3	Preprocessing the Datasets	25
3.3.1	Denoising	25
3.3.2	Homogenising the Background Illumination	27
3.3.3	Binarisation	29
3.3.4	Remapping the Intensities	29
3.3.5	Feature Decluttering	30
3.4	Construction of the DCNNs	32
3.4.1	Fully Convolutional Network	32
3.4.2	ResNet-18	32
3.4.3	ResNet-34	32
3.4.4	UNet	33
3.4.5	UNet 3+	33

3.5	Training the DCNNs	33
3.6	Quantifying the Quality of the Segmentation	34
3.6.1	Precision	34
3.6.2	Recall	34
3.6.3	The F1 Score (Dice Coefficient)	35
3.6.4	Three Class Segmentation	35
4	Results and Discussions	36
4.1	Segmentation of Validation Datasets	38
4.1.1	Idealised White Matter	38
4.1.2	Rat White Matter	45
4.2	Segmentation of Sheep White Matter	49
4.2.1	Denoised Dataset	50
4.2.2	Binarised Dataset	55
4.2.3	Intensity-remapped Dataset	59
4.2.4	Decluttered Dataset	63
5	Conclusions and Perspectives	66
5.1	Conclusion	66
5.2	Future Research	67

List of Figures

1.1	(a) A schematic of the blood-brain barrier and an illustration of the CED technique. (b) The brain white matter visualised and an illustration of a neuron.	2
2.1	The Architecture of a basic DCNN for Image Classification (adapted from [43]).	7
2.2	The Convolution Operation. The kernel slides in the arrow's direction from the shaded region to the red outline and takes the dot product between all the entries (adapted from [45]).	8
2.3	Illustration of the Node connections in a Convolutional Layer (reproduced from [45]).	8
2.4	Demonstration of the Max Pooling Operation (adapted from [43]).	9
2.5	Illustration of the Node connections in a Fully Connected Layer (reproduced from [45]).	9
2.6	The Architecture of a Simple FCN for Image Segmentation (adapted from [54]).	10
2.7	The Residual Block, the Fundamental Component of the ResNet (reproduced from [63]).	11
2.8	A diagram of the UNet architecture.	12
2.9	(a) A schematic of the UNet architecture. (b) A schematic of the UNet 3+ architecture.	13
3.1	A flowchart which illustrates the thesis's whole procedure to create DCNNs for automatic segmentation of sheep WM.	16

3.2	The brain white matter visualised using diffusion tensor imaging, showing the nerve fibres as colourful tracts. The illustration to the right depicts a neuron, with the nerve fibres (myelinated axons) making up the white matter, and the cell bodies (soma) making up the grey matter (reproduced from [5]).	17
3.3	A slice of the idealised WM dataset	18
3.4	A slice of the rat WM dataset, imaged by Abdollahzadeh et al. [36]. To serve as an example, a section of myelin is contained within the yellow circle and a section of an axon is contained within the red circle.	19
3.5	A slice of each sheep WM dataset: (a) Fornix01 and (b) Fornix02. The two datasets originate from the fornix of two different sheep and were imaged by Bernardini et al. [7]. To serve as an example, a section of myelin is contained within the yellow circle and a section of an axon is contained within the red circle.	20
3.6	A slice of the idealised WM segmentation mask. (Colour scheme: turquoise = idealised myelin-axon pair).	22
3.7	A slice of the rat WM segmentation mask, created for this thesis using ACSO [36]. Unlike the segmentation mask for the idealised WM dataset and the sheep WM dataset, ACSO generates a segmentation mask which separates axons and myelin. (Colour scheme: turquoise = myelin, purple = axon). . .	23
3.8	(a) A magnified view of the point cloud available from Bernardini et al. [7]. The magnification was necessary as the one pixel wide perimeter is otherwise too thin to be seen. (b) A magnified view of the continuous point cloud from the image dilation. (c) A slice of the final segmentation mask from the Fornix02 sheep WM dataset. (Colour scheme: turquoise = idealised myelin-axon pair)	24
3.9	The effect of denoising the (a) raw rat and (c) raw sheep WM datasets is depicted in (b) and (d). The BM3D denoising algorithm [85] was used for this procedure.	26
3.10	A slice of the (a) denoised rat WM and the two denoised sheep WM samples, (b) Fornix01 and (c) Fornix02.	27

3.11	(a) An example slice of inhomogeneous background illumination, which is subtracted from the denoised sheep WM. A slice of the (b) Fornix01 and (c) Fornix02 samples with homogenised background illumination.	28
3.12	A slice of the binarised (a) Fornix01 and (b) Fornix02 sheep WM samples. The binarisation was performed using the “adaptthresh” [88, 89] MATLAB function.	29
3.13	A slice of the (a) Fornix01 and (b) Fornix02 sheep WM samples with their intensities remapped using the “imadjust” [90] MATLAB function.	30
3.14	A slice of the (a) intensity-remapped Fornix01 and (b) decluttered Fornix01 sheep WM samples.	31
4.1	A slice of the (a) ground truth segmentation and the (b-f) segmentation results from the five models of the idealised WM dataset. (Colour scheme: turquoise = idealised myelin-axon pair)	38
4.2	Examination of Segmentation Error: Magnified view of missing segmentation in the middle of large idealised myelin-axon pairs. (Colour scheme: Blue = TP, Green = FN, Red = FP).	40
4.3	Examination of Segmentation Error: Magnified view of merging of the segmentation at edges of touching idealised myelin-axon pairs. (Colour scheme: Blue = TP, Green = FN, Red = FP).	41
4.4	Examination of Segmentation Error: Magnified view of over-labelling along part of a small myelin-axon pair’s outer edge. (Colour scheme: Blue = TP, Green = FN, Red = FP).	42
4.5	Examination of Segmentation Error: Magnified view of under-labelling along part of a large myelin-axon pair’s outer edge. (Colour scheme: Blue = TP, Green = FN, Red = FP).	43
4.6	Examination of Segmentation Error: Magnified view of randomly segmented patches of the background, which is only observed for the FCN and UNet. (a) and (b) show different regions in the dataset. (Colour scheme: Blue = TP, Green = FN, Red = FP).	44

4.7	(a) A slice of the ground truth segmentation and the (b-f) segmentation results from the five models of the rat WM dataset. (Colour scheme: turquoise = myelin, purple = axon).	45
4.8	Examination of Segmentation Error: Magnified view of axon, marked in yellow, which is missing from the (a) ground truth segmentation mask but is present in the segmentation mask of several DCNNs: (b), (e), (f). (Colour scheme: turquoise = myelin, purple = axon).	46
4.9	A slice of the denoised sheep WM samples: (a) Fornix01 and (b) Fornix02. Both samples, taken from two different sheep, are from the fornix in the brain.	50
4.10	A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the denoised sheep WM (Fornix01).	50
4.11	A slice of the sheep WM samples with homogenised background intensities: (a) Fornix01 and (b) Fornix02. Both samples, taken from two different sheep, are from the fornix in the brain.	52
4.12	A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the denoised sheep white matter (Fornix02).	53
4.13	A slice of the binarised sheep WM samples: (a) Fornix01 and (b) Fornix02. Both samples, taken from two different sheep, are from the fornix in the brain.	55
4.14	A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the binarised sheep WM (Fornix01).	55
4.15	A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the binarised sheep WM (Fornix02).	57
4.16	A slice of the intensity-remapped sheep WM samples: (a) Fornix01 and (b) Fornix02. Both samples, taken from two different sheep, are from the fornix in the brain.	59
4.17	A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the intensity-remapped WM (Fornix01).	59
4.18	A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the intensity-remapped sheep WM (Fornix02).	61

4.19	A slice of the (a) feature decluttered Fornix01 dataset, used to test the DCNNs, and (b) intensity-remapped Fornix02 dataset, used to train the DCNNs. Both samples, taken from two different sheep, are from the fornix in the brain. . .	63
4.20	A slice of (a) the ground truth segmentation and (b-f) the segmentation results from the five models of the decluttered sheep WM (Fornix01).	64

List of Tables

4.1	Evaluation Metrics for the Segmentation of Idealised White Matter	39
4.2	Evaluation Metrics for the Segmentation of Rat White Matter Myelin	47
4.3	Evaluation Metrics for the Segmentation of Rat White Matter Axons	47
4.4	Evaluation Metrics for the Segmentation of Rat White Matter: Weighted Average	48
4.5	Evaluation Metrics for the Segmentation of Denoised Sheep WM (Fornix01).	51
4.6	Evaluation Metrics for the Segmentation of Denoised Sheep WM (Fornix02).	53
4.7	Evaluation Metrics for the Segmentation of Binarised Sheep WM (Fornix01).	56
4.8	Evaluation Metrics for the Segmentation of Binarised Sheep WM (Fornix02).	58
4.9	Evaluation Metrics for the Segmentation of the intensity-remapped Sheep WM (Fornix01).	60
4.10	Evaluation Metrics for the Segmentation of the intensity-remapped Sheep WM (Fornix02).	61
4.11	Evaluation Metrics for the Segmentation of Decluttered Sheep WM (Fornix01).	65

Chapter 1

Introduction

1.1 Motivation

Due to ageing populations, neurological conditions have become the second most common cause of death and the premier cause of disability in the world [1]. To combat this, the Enhanced Delivery Ecosystem for Neurosurgery in 2020 (EDEN2020) project aims to develop and improve systems for minimally invasive treatment of neurological conditions [2]. One such system is Convection-Enhanced Delivery (CED) [3], which allows therapeutic drugs to be delivered directly to the target using pressure gradients instead of concentration gradient-based diffusion. The latter distribution method is often difficult to manage, as the larger drug concentrations needed for effective diffusion may cause undesirable effects [3, 4]. Another advantage of CED is the ability to directly infuse the drug into the brain's extracellular space and drive it with a pressure gradient [3], depicted in Figure 1.1a. This allows the blood-brain barrier, also shown in Figure 1.1a, to be bypassed, whose selective permeability blocks the diffusion of the large molecules used for many common therapeutics [3, 5].

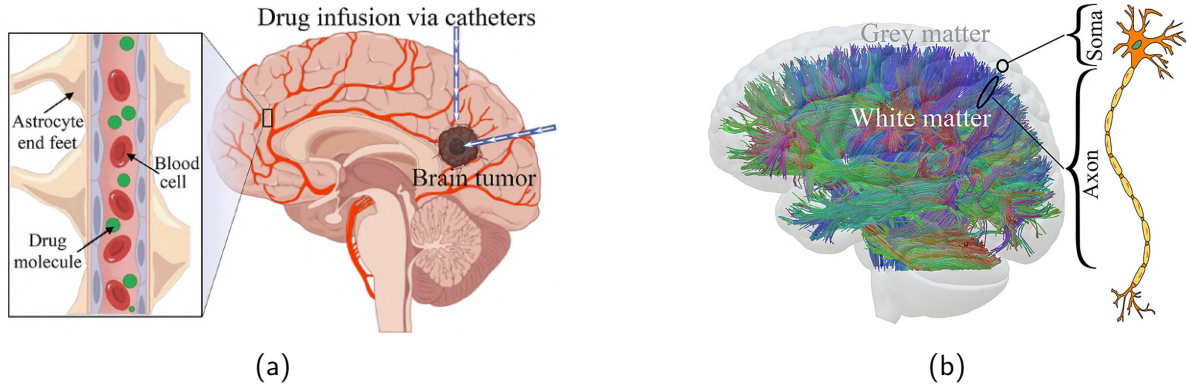


Figure 1.1: (a) A schematic of the blood-brain barrier to the left and an illustration of the CED technique to the right. (b) The brain white matter visualised using diffusion tensor imaging, showing the nerve fibres as colourful tracts. The illustration to the right depicts a neuron, with the nerve fibres (myelinated axons) making up the white matter, and the cell bodies (soma) making up the gray matter (reproduced from [5]).

A major difficulty with modelling CED in the brain is the heterogeneity and anisotropy of the White Matter (WM) microstructure [3, 6], the effects of which on fluid flow are still not well-defined [7]. As shown in Figure 1.1b, WM is made up of tracts of nerve fibres that lead to the neuron's cell bodies in the grey matter. The individual nerve fibres are myelinated axons through which electrical impulses travel to enable connectivity in the brain [8]. In order to better understand the biomechanics of WM microstructure, the morphology of myelinated axons within the WM needs to be studied in three dimensions. Such studies can be enabled by three-dimensional reconstructions, but these require accurate and high-resolution segmentations of the WM microstructure [9]. Segmentation is the act of classifying each pixel in an image, e.g., labelling a pixel as being part of a myelinated axon. One option is to manually segment images of WM, and while this will produce accurate results, it is extremely time-consuming [10]. As a result, increasing the obtainable WM segmentation accuracy of automated techniques is advantageous. With improved automatic segmentation tools, WM volumes can be accurately reconstructed at a significantly faster rate, enabling more comprehensive studies of WM biomechanics. Therefore, this thesis intends to use deep learning to create a tool for automatic segmentation of WM.

1.2 Objectives

The objective of this thesis is to apply existing state-of-the-art machine learning methods for automated segmentation, particularly Deep Convolutional Neural Networks (DCNN), to segment sheep (*Ovis aries*) white matter (WM) datasets. The datasets are from Bernardini et al.'s substudy [7] of the EDEN2020 [2] project, where they imaged and manually segmented the sheep WM. These manual segmentations will then be used to train various DCNN architectures and test their image segmentation accuracy. Additionally, multiple different image preprocessing techniques are tested to improve the DCNN segmentation results.

Chapter 2

Literature Review

To perform 3D reconstruction, one first needs to acquire images of the object to be reconstructed in three dimensions, meaning a stack of two-dimensional images. The next step is to segment each image, which is the act of labelling a region of pixels as distinct from its surroundings. For example, labelling pixels belonging to a tumour as distinct from the healthy tissue. Once a stack of images has been segmented, a 3D model can be rendered [11].

As the segmentation is the most difficult aspect of the 3D reconstruction process, that is what this literature review will focus on. In the first part of the literature review, the use of image segmentation within the medical field will be reviewed, including the existing imaging modalities and segmentation techniques. In particular, the difficulties associated with manual segmentation will be discussed and the alternative automatic segmentation methods will be presented. In the second part of the literature review, deep learning-based segmentation techniques will be explored, in particular the Deep Convolutional Neural Network (DCNN). To begin, the basic components of DCNNs are explained, followed by an introduction to four of the most popular DCNN architectures for medical image segmentation. These architectures will then serve as the basis for the DCNNs which are used in this thesis.

Section 2.1 of the Literature Review has been adapted from a literature review previously submitted as coursework.

2.1 Image Segmentation in Medical Research

Image segmentation is one of the most important yet challenging tasks in medical imaging. As segmentation allows regions of interest (such as organs, tumours or cells) to be clearly separated from the rest of the image, this tool is a great aid for both medical practice and research [12, 13]. For medical practice, image segmentation is used for various applications such as disease diagnosis, monitoring tumour growth in patients [14] and marking tumorous regions for radiotherapy [15]. In medical research, examples of use cases include using segmented images to study cell morphology, meaning their shape, size and structure, [16] or to create 3D reconstructions which can be used for simulations, such as modelling blood vessels to study blood flow with Computation Fluid Dynamics (CFD) [17].

In Section 2.1.1 of the literature review some different imaging techniques used for medical imaging are presented and the implications of using a high spatial resolution imaging technique are discussed. Section 2.1.2 examines the main advantages and disadvantages of both manual and automatic segmentation. Lastly, Section 2.1.3 introduces the current state-of-the-art automatic segmentation techniques.

2.1.1 Imaging Techniques

For most applications in the medical field, the resolution of imaging techniques such as Magnetic Resonance Imaging (MRI) and Computed Tomography (CT) is sufficient, both of which usually have a spatial resolution in the neighbourhood of 1mm [18–20]. For example, tears in the rotator cuff (a shoulder muscle) can be fully imaged with MRI resolutions, segmented and then 3D reconstructed to accurately determine the tear pattern, which is extremely useful for surgical planning [21]. On the other hand, to fully and accurately capture the ultrastructure of brain White Matter (WM), imaging techniques with resolutions of nanometre scale, such as Electron Microscopy (EM), are necessary [7, 22].

The need to use high-resolution imaging techniques for the 3D reconstruction process, like EM, brings about some unavoidable difficulties. One of these difficulties is the cubically growing size of the image datasets as the imaging resolution increases. For the segmentation of a $10 \times 10 \times 10 \text{ cm}^3$ sample, an MRI with millimetre scale resolution will produce a dataset with a magnitude of 10^6 voxels [23]. In comparison, the same $10 \times 10 \times 10 \text{ cm}^3$ sample imaged with Scanning EM (SEM), in that case using a resolution of $10 \times 10 \times 50 \text{ nm}^3$, will produce a dataset size in the magnitude of 10^{20} voxels. Even in a realistic scenario from a paper by Abdollahzadeh et al. [24], where the brain WM sample was of size $15 \times 15 \times 50 \text{ }\mu\text{m}^3$, and the imaging resolution was $15 \times 15 \times 50 \text{ nm}^3$, the dataset size was still in the magnitude of 10^8 voxels.

Another difficulty associated with the usage of some high-resolution imaging techniques, such as EM, is sample preparation. Unlike MRI, which is non-invasive and non-toxic [25], EM

requires intensive sample preparation and imaging is performed in a vacuum, meaning that it is unsuitable for living samples [24, 26]. Accounting for both ethical and logistical considerations, this poses a great challenge in obtaining usable samples of human brain WM, as the quality of a WM sample drops significantly within a day after death if it is not properly preserved [27, 28]. Therefore, in a sub-study [7] for the EDEN2020 [2] project, where brain WM was manually segmented and 3D reconstructed, sheep's brain was used for the WM datasets. This is because it sheep's brain has been shown to have advantages over small animal models in regards to studying the human brain [29]. The datasets and results from that sub-study serve as the basis for this thesis.

2.1.2 Manual versus Automatic Segmentation

Despite needing significant time and labour investment, in addition to requiring a high level of expertise, manual segmentation is still commonly used to perform 3D reconstruction of medical images [21, 30, 31]. Although the difficulty of manual segmentation has stopped this tool from becoming widely used in healthcare, this type of segmentation is still performed for medical research, including to help develop methods of automatic segmentation [7, 21]. As discussed in Section 2.1.1, for most medical applications, the lower resolution of MRI is sufficient, which will produce an image dataset size in the magnitude of 10^6 voxels in typical scenarios [23], as opposed to upwards of 10^8 voxels for EM images [24]. Manually segmenting MRI datasets reportedly takes an expert a couple of hours, which is still reasonable [21, 32]. Contrastingly, it has been reported by Berning et al. [10] that the manual segmentation of a 5×10^8 voxel large EM image dataset took about 1500 work hours. It is due to this immense time requirement that the development of methods for automatic segmentation is of great interest.

Although automatic methods do not require a substantial amount of human work hours like manual segmentation, the automation does come at a cost. The main challenges that various automated techniques face are the reduced accuracy compared to manually performed segmentation, the large amount of training data needed and the sensitivity to image imperfections, such as noise or uneven image illumination [30, 33–35]. Fortunately, there is a continuous effort to improve these tools and bring their performance in line with the quality and reliability of manual segmentation. In Section 2.1.3, some of these tools are introduced and compared with each other.

2.1.3 State-of-the-Art Segmentation Techniques

A popular class of automatic segmentation techniques are those that are based on thresholding algorithms. Thresholding divides an image into two regions, usually subject and background, by using a threshold for certain characteristics, like illumination intensity or area, to label each voxel [35]. A powerful thresholding algorithm is the Region-Growing algorithm, which is built upon the assumption that voxels belonging to the same label are clustered together. In

this algorithm, after an initial point is chosen, neighbours of this point (and the neighbours of the neighbour) are classified as belonging to the same region if they fall within a set similarity threshold [35]. The disadvantages of using thresholding algorithms for automatic segmentation are the high sensitivity to image imperfections, especially noise, and the need for occasional human intervention. An example of the latter is shown in another paper by Abdollahzadeh et al. [36], where they introduce their algorithmic segmentation pipeline “AutomatiC 3D Segmentation Of axoNs” (ACSON), which uses an advanced Region-Growing algorithm to automatically segment brain WM ultrastructure. In this paper, they reported that ACSON failed to correctly segment mitochondria, meaning that this segmentation had to be done manually, reportedly taking approximately 6 hours [36]. As a result, research groups like Abdollahzadeh et al.’s [36] have shifted their focus from algorithmic techniques to ones that utilise the ever-growing field of machine learning in hopes that no human intervention will be necessary.

As computers became more powerful and the field of machine learning increasingly advanced, the application of this tool for computer vision tasks was inevitable. A paper from 1998 by LeCun et al. [37], which introduced the LeNet-5, laid the groundwork for the Deep Convolutional Neural Network (DCNN) [38]. This type of network has since become the premier choice for image classification and segmentation [39–41]. The main advantage of DCNNs over algorithmic techniques is their feature extraction. While algorithms often need to be tuned to work on new tasks, deep learning-based tools are able to learn new relationships automatically by extracting features from the images. [39, 42]. Before the state of the art DCNN architectures are introduced in Section 2.3, the basic building blocks of DCNNs are introduced in Section 2.2.

2.2 The Building Blocks of a DCNN

As explained in Section 2.1.3, using a Deep Convolutional Neural Network (DCNN) is a popular choice for various computer vision tasks, including image segmentation. In this Section, the three most fundamental building blocks of a DCNN will be introduced. These blocks can be seen in Figure 2.1, where we have:

- The convolutional layers (blue)
- Max pooling for downsampling (red)
- Fully connected layers (green)

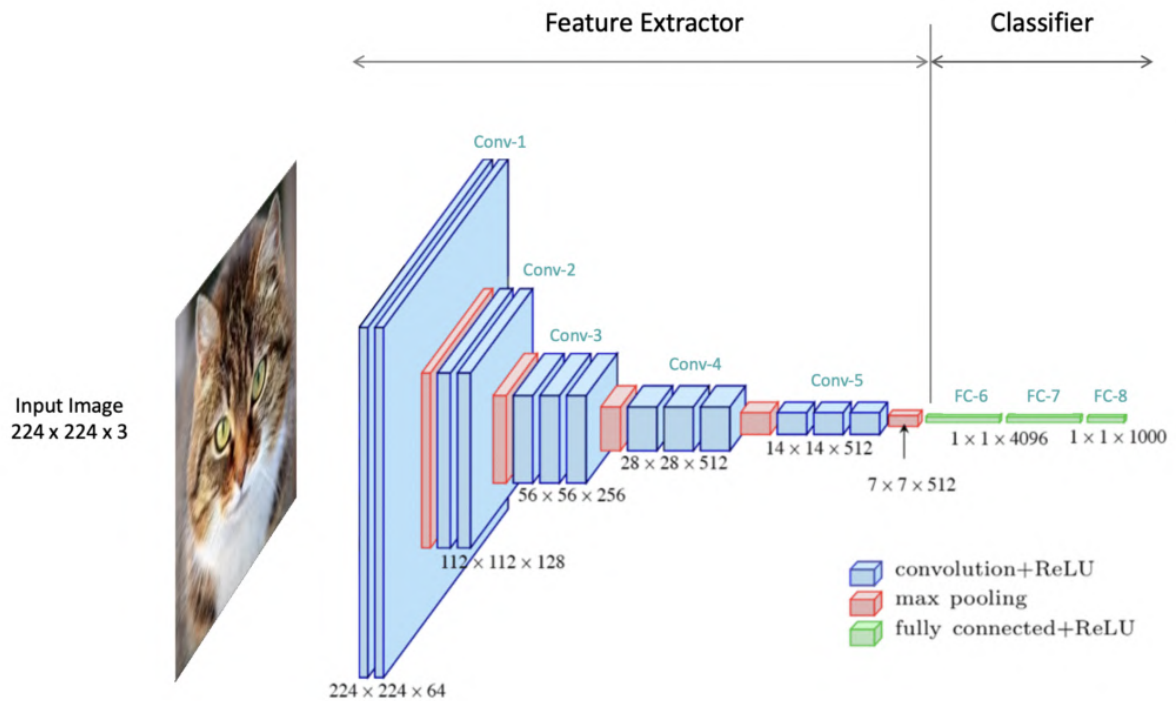


Figure 2.1: The Architecture of a basic DCNN for Image Classification (adapted from [43]).

2.2.1 The Convolutional Layer

The model starts with convolutional layers, as seen in Figure 2.1, which are responsible for extracting the image's features. A convolutional layer first applies a convolution operation to the input image and then an activation [43], such as the Rectified Linear Unit (ReLU) activation (information on the ReLU is available here [44]). As depicted in Figure 2.2, the convolution operation can be thought of as a sliding dot product because the kernel slides along the image's input matrix and takes the dot product between the corresponding entries.

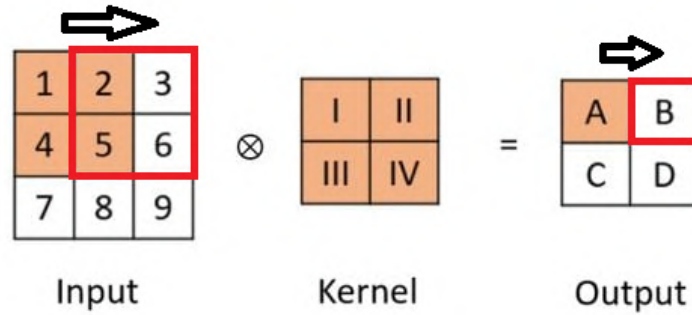


Figure 2.2: The Convolution Operation. The kernel slides in the arrow's direction from the shaded region to the red outline and takes the dot product between all the entries (adapted from [45]).

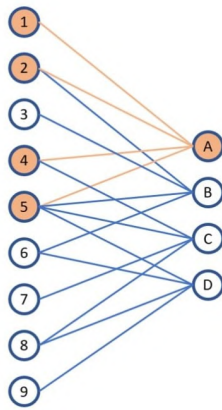


Figure 2.3: Illustration of the Node connections in a Convolutional Layer (reproduced from [45]).

The illustration in Figure 2.3 shows an example of how the neural network nodes in a convolutional layer are connected. The key detail is that an output node is only connected to some of the input nodes. More specifically, as seen in Figure 2.2, a certain output node (e.g., output A) is connected only to those input nodes that contributed to the dot product with the kernel to create the output (e.g., input 1, 2, 4 and 5) [45].

Differently weighted kernels in the convolutional operation can be used to create different outputs, each extracting unique features from the input image [46]. The three numbers under the layers in

Figure 2.1 represents the “input image height” x “input image width” x “number of channels”, the latter being the number of unique feature maps created by the convolutional operation.

2.2.2 Downsampling

An important part of DCNNs is the pooling layers, which are responsible for downsampling the feature maps. As seen in Figure 2.1, a pooling layer is usually added after every couple of convolutional layers. There are several different methods to achieve pooling, one of the most commonly used ones being max pooling, which is demonstrated in Figure 2.4. In pooling operations, a 2x2 filter covers the input image without overlap, as seen by the coloured shaded regions in Figure 2.4. Then, for max pooling, the largest value within each filter is the output [43]. The size of the pooling operation's output is half the width and height of the input image, which is why this is a form of downsampling.

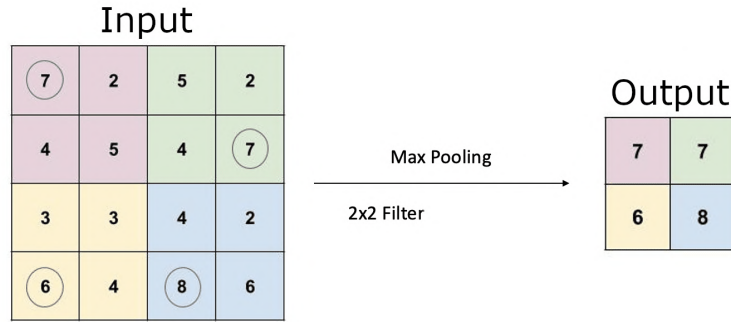


Figure 2.4: Demonstration of the Max Pooling Operation (adapted from [43]).

There are two main benefits to max pooling in DCNNs. Firstly, as only the largest value from each 2x2 filter is passed onto the output, this operation extracts the most significant features from each input, allowing the model to focus on those only [47, 48]. This also helps reduce overfitting, as the model is not being trained on redundant information [49]. Secondly, the feature map's reduced spatial dimensions also result in a decreased computational complexity, which allows more powerful models to be trained on the same hardware [47–49].

2.2.3 Fully Connected Layers

Regular DCNNs, which are built for image classification (i.e., an image as the input and a class, such as a cat or dog, as the output), have fully connected layers after the convolutional and pooling layers [50], as depicted in Figure 2.1. Unlike in convolutional layers, where input nodes are only connected to some output nodes (as seen in Figure 2.3), all inputs are connected to all outputs in fully connected layers.

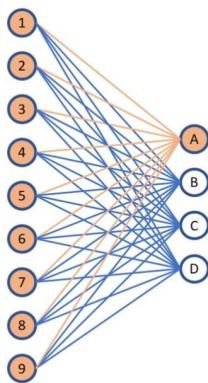


Figure 2.5: Illustration of the Node connections in a Fully Connected Layer (reproduced from [45]).

An example is illustrated in Figure 2.5, where there are nine inputs all flowing into each of the four outputs. This is useful for classification tasks, as the inputs coming from the final convolutional layer, which is responsible for extracting features from the image, all contribute to each of the four outputs of the fully connected layer [43, 51]. Therefore, in the final fully connected layer of the network, where each output corresponds to one of the classification classes, the image will be classified as the output with the strongest response.

Unfortunately, DCNNs with fully connected layers are unsuitable for image segmentation tasks. The fully connected layers map the two-dimensional matrix from the convolutional layers to a one-dimensional output vector, de-

stroying the spatial information [52, 53]. As a result, the individual pixels in the image cannot be classified with accuracy, which is necessary for accurate image segmentation. To address this issue, the basic DCNN architecture needed to be modified, as is discussed in Section 2.3.

2.3 DCNN Architectures for Image Segmentation

For this literature review, the field has been surveyed and the most popular and significant DCNN architectures for image segmentation have been determined. The following architectures, all developed within the last decade, are particularly popular for medical image segmentation.

2.3.1 Fully Convolutional Network

The Fully Convolutional Network (FCN) was created in 2014 by Long et al. [54] to enable the usage of convolutional networks for accurate image segmentation. As the fully connected layers, which make up a basic DCNN classifier (see Figure 2.1), only return one output per class, the novelty of the FCN was the replacement of these with a pixel-by-pixel classifier. The foundation of the classifiers that are able to label the input's individual pixels are upsampling layers. This is because the pooling layers (see Section 2.4) have reduced the size of the model's feature map, meaning that a corresponding amount of upsampling is required to restore it [54, 55]. A simple FCN is depicted in Figure 2.6, where the convolutional and pooling layers are followed by an upsampling layer to restore the dimensions and perform pixel-wise predictions of the input.

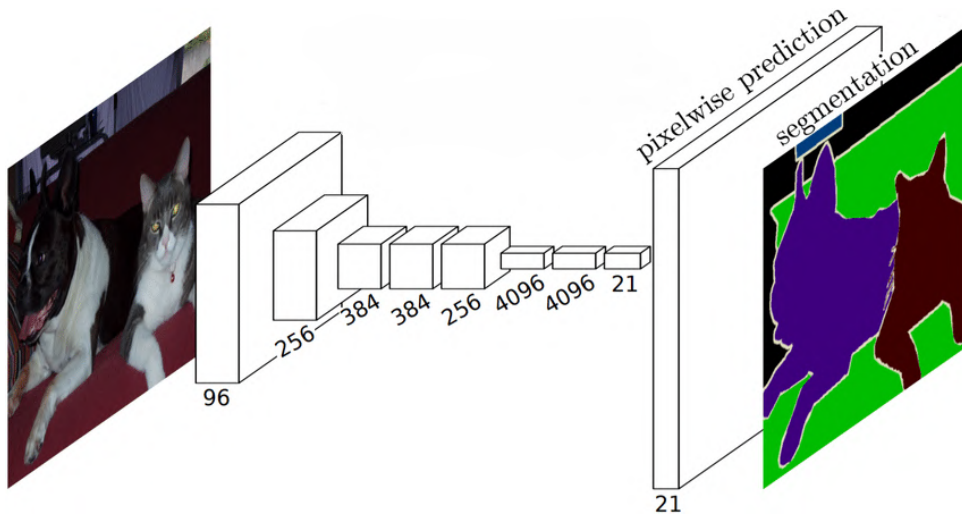


Figure 2.6: The Architecture of a Simple FCN for Image Segmentation (adapted from [54]).

Although it is possible to simply use basic upsampling operations (e.g., bilinear upsampling [56]) to restore the dimensions of the feature map, this is not recommended as the output segmentation mask will be extremely coarse [54]. To get better results, the upsampling operations are typically combined with additional convolutional layers, which allow for additional

feature extraction during the upsampling process. Another common approach is to replace the upsampling operations with transpose convolutional layers (i.e., inverse convolutional layers), which are able to both extract features and upsample the model's feature map [54, 57].

2.3.2 ResNet

The Residual Network, ResNet for short, is an FCN with an improved backbone that was created in 2015 by He et al. [58]. A network's backbone is the layers which are responsible for the feature extraction [59], so the backbone of the FCN described in Section 2.3.1 is the ensemble of convolutional and pooling layers. In the creation of the ResNet, He et al. were seeking to mitigate the “vanishing gradient problem”, which is a phenomenon that commonly impacts the training process of deep neural networks [58, 60, 61]. This is because, for deep networks, the gradients required to update the model weights often vanish as they propagate through the layers [62].

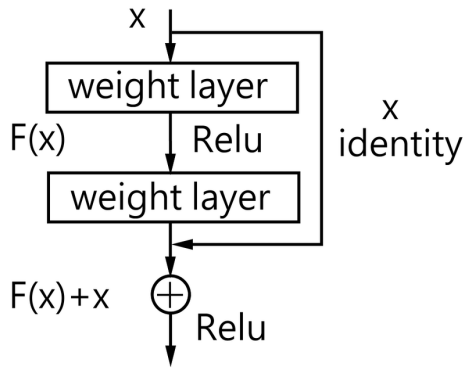


Figure 2.7: The Residual Block, the Fundamental Component of the ResNet (reproduced from [63]).

The fundamental component and backbone of ResNets and He et al.'s [58] proposed solution for the “vanishing gradient problem” is the residual block, as depicted in Figure 2.7. These use a skip connection, which adds the input x to the output $F(x)$ of two weight layers (i.e., convolutional layers) down the line. This mitigates the “vanishing gradient problem” in the following way: if the gradient of one of the layers vanishes, then the skip connection allows input to flow past those layers. Due to this, the training process of the network as a whole will not be affected [42, 58].

2.3.3 UNet

The UNet was specifically developed for biomedical image segmentation by Ronneberger et al. [64] in 2015. Since then, the UNet has become the premier choice of DCNN architecture to segment medical images [65]. As seen in Figure 2.8, the architecture is composed of an encoder and a decoder. The encoder has the same structure as the feature extractor for basic DCNNs as seen in Figure 2.1, which consists of convolutional and pooling layers. The decoder, consisting of convolutional and transpose convolutional layers, performs the upsampling necessary for segmentation, as described in Section 2.3.1.

Similarly to the ResNet described in the previous Section, the revolutionary part of the UNet is the skip connections [65]. However, they serve a completely different purpose here. As seen in Figure 2.8, the blue skip connections connect the output of the encoder (before downsampling) to the input of the corresponding decoder (i.e., the decoder with the same image dimensions as the encoder output). This combines the “where” in the image, which

is lost during downsampling, with the “what” in the image, which is gained during feature extraction in the encoder [64–66]. Although the upsampling layers can also do this themselves, the addition of the skip connections allows for significantly more accurate localisation, resulting in more accurate segmentations [65].

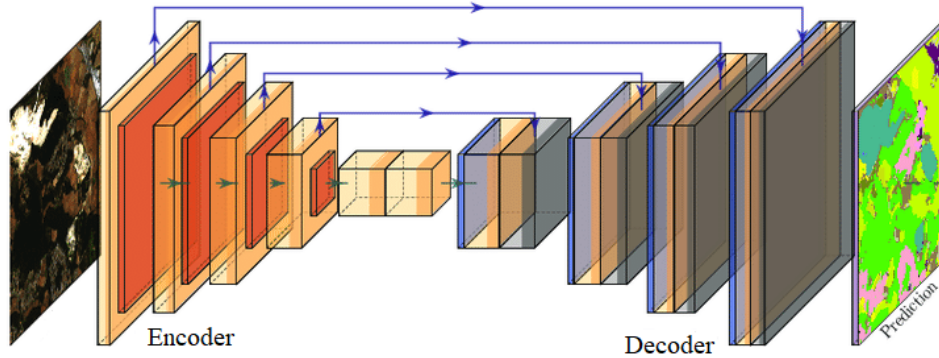


Figure 2.8: A diagram of the UNet architecture. The orange blocks are the convolutional and pooling layers of the encoder, while the blue blocks are the convolutional and transpose convolutional layers of the decoder. The orange slices within the decoder are the outputs from the encoder fed to the inputs of the decoder via the skip connections (adapted from [67]).

2.3.4 UNet 3+

The UNet 3+ is an improved version of the UNet architecture developed by Huang et al. [68] in 2020. This architecture uses the same encoder and decoder layers as the UNet but introduces two new features: full-scale skip connections and full-scale deep supervision [68]. As the UNet 3+ architecture is too complicated to reasonably depict in a diagram, Figure 2.9b depicts a schematic of the UNet 3+, with a schematic of the UNet for reference in Figure 2.9a. The full-scale skip connections can clearly be seen in the UNet 3+ schematic, which allows each decoder layer to combine and learn from the outputs of all previous decoders and encoders simultaneously [65, 68]. The full-scale deep supervision is also depicted in Figure 2.9b as the “sup” above the decoder layers. In full-scale deep supervision, each individual decoder has a side output that is compared and minimised with the ground truth, much like the final output, which is compared in normal DCNNs [68]. Both of these additions improved upon the regular UNet’s ability to segment low-level detail with high spatial accuracy [65].

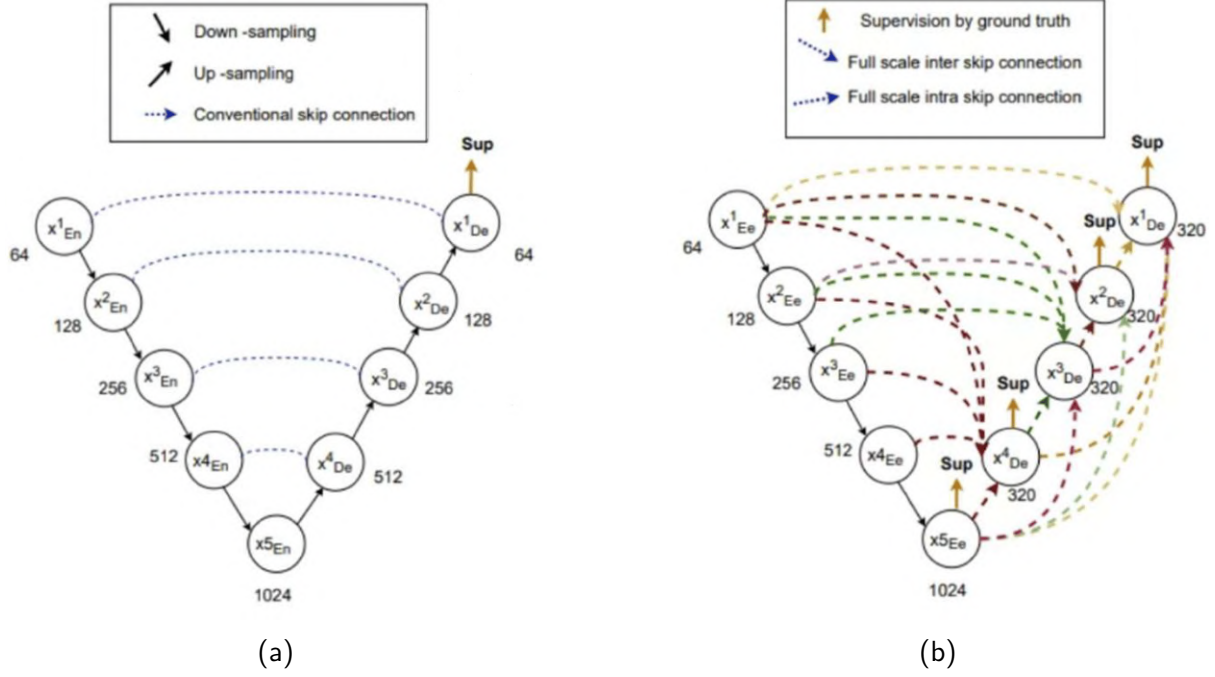


Figure 2.9: (a) A schematic of the UNet architecture introduced in Section 2.3.3. Here, the subscripts are the number of feature maps for each encoder/decoder layer and the “sup” at the last decoder layer is the standard machine-learning comparison of the output with the ground truth. (b) A schematic of the UNet 3+ architecture. Here, the “full-scale skip connections” link each decoder with all previous decoders and all encoders with a smaller or equal number of feature maps (the subscript under the layers). The “sup” above the decoders indicates “full-scale deep supervision”, which means each decoder has a side output which is compared to the ground truth (adapted from [69]).

Chapter 3

Datasets and Methodology

The aim of this thesis is to successfully train a Deep Convolutional Neural Network (DCNN), which is able to automatically segment sheep White Matter (WM). This Chapter discusses all the datasets and the entirety of the methodology used for this thesis. In order, the Sections in this Chapter discuss:

- The datasets used to test/validate the DCNN architectures and train the final DCNN models (Section 3.1).
- The procedure to create the training labels for each dataset (Section 3.2).
- The various preprocessing techniques applied to the datasets to aid the network training (Section 3.3).
- The architecture of each of the DCNNs used in this thesis (Section 3.4).
- The procedure to train the DCNN models (Section 3.5).
- The evaluation metrics that are employed to quantify the results (Section 3.6).

The procedure used in this thesis to create the DCNNs for automatic segmentation of sheep WM, starting from the raw datasets all the way to obtaining the final DCNN outputs, is illustrated in the flowchart in Figure 3.1. The middle of the flowchart depicts the trained DCNNs, where inputs flow via the thin black arrows and the output segmentation mask flows out via the orange arrow. The processes above the middle are those which primarily deal with the sheep WM image, whereas the processes below the middle primarily deal with the segmentation mask used for DCNN training.

The green arrows in Figure 3.1 indicate a preprocessing technique, which all have their procedure described in the associated Sections of this Chapter. All the preprocessing along an arrow's path builds upon each other, meaning that subsequent preprocessing steps are not

performed on the raw data. To begin, the raw sheep White Matter (WM) is denoised, which is followed by a procedure to homogenise the background illumination of the datasets. Then, the homogenised WM is either binarised or has its intensities remapped, both of which are aimed to amplify and normalise the intensities in the images. In the final preprocessing step named “Feature Decluttering”, which builds on the intensity-remapped WM, the manual segmentation mask is used to remove large, unlabelled features in the training dataset in order to aid the DCNN training.

The blue arrow in Figure 3.1 indicates the creation of the segmentation mask using the manual segmentations of the sheep WM datasets from Bernardini et al.’s work [7]. As seen in the flowchart, the available manual segmentation data is in the form of a point cloud. Full segmentation masks were created from these using various MATLAB functions, which are described in Section 3.2.3.

As indicated by the thin black arrows in Figure 3.1, a new DCNN is trained after all preprocessing steps except the homogenisation of the background illumination, which is just a procedure to prepare the dataset for binarisation and intensity remapping. The results for each trained DCNN are discussed in the Section displayed on the black arrows. As the DCNNs were trained and evaluated sequentially, each subsequent preprocessing procedure was chosen with the intent to improve upon the results of the previous procedure.

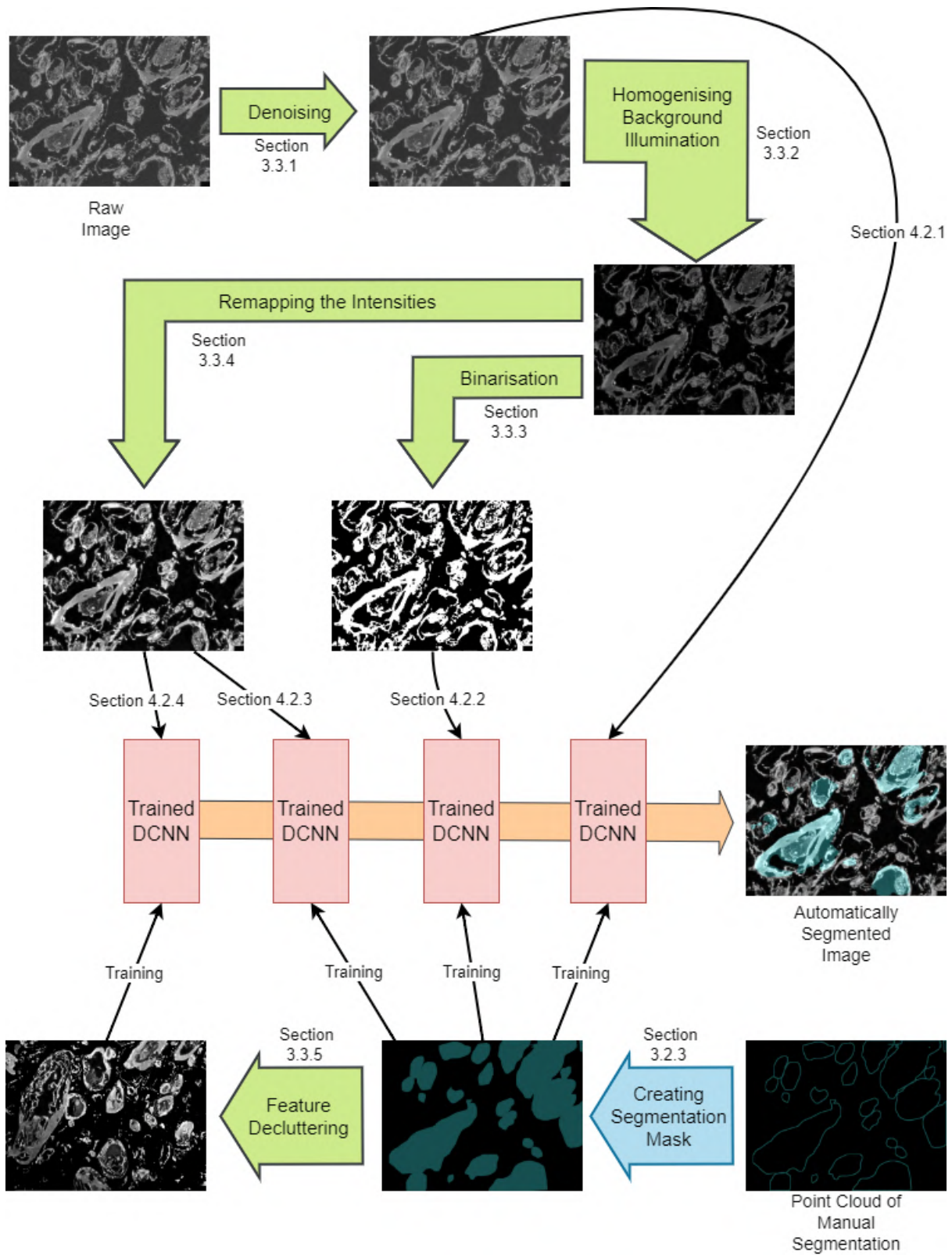


Figure 3.1: A flowchart which illustrates the thesis's whole procedure to create DCNNs for automatic segmentation of sheep WM.

3.1 Datasets

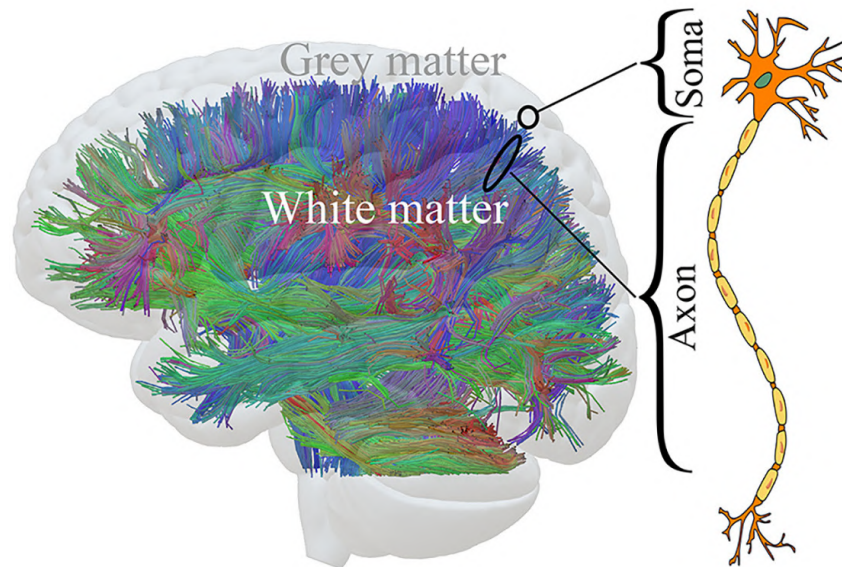


Figure 3.2: The brain white matter visualised using diffusion tensor imaging, showing the nerve fibres as colourful tracts. The illustration to the right depicts a neuron, with the nerve fibres (myelinated axons) making up the white matter, and the cell bodies (soma) making up the grey matter (reproduced from [5]).

Half of the human brain is comprised of White Matter (WM), which itself consists of bundles of millions of myelinated axons [70], as seen in Figure 3.2. An illustration of a myelinated axon is depicted on the Figure’s right side, which is an axon, the structure responsible for conducting electrical impulses between neurons [70], covered in a myelin sheath, the axon’s insulating layer [71]. The region between the individual myelinated axons in the WM is called the extracellular matrix [7], which consists of various proteins and carbohydrates [72]. For the remainder of the thesis, the extracellular matrix will simply be referred to as the “background” of the images.

Three different WM datasets are used in this thesis: an idealised WM dataset, a rat WM dataset and a sheep WM dataset. The purpose of the idealised WM dataset is to validate the models’ basic segmentation ability, while the purpose of the rat WM dataset is to validate the models’ segmentation performance compared to a state-of-the-art DCNN model. Finally, the DCNN models will be trained to segment sheep WM datasets for the EDEN2020 [2] project.

3.1.1 Idealised White Matter Dataset

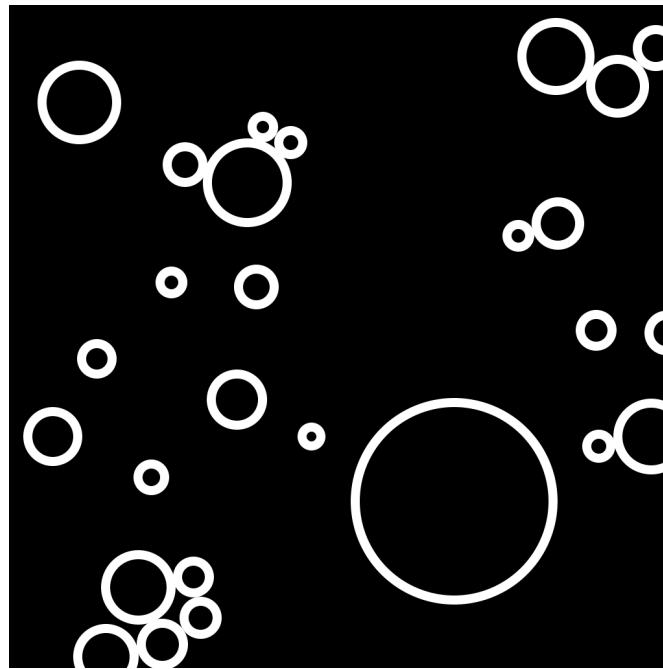


Figure 3.3: A slice of the idealised WM dataset

The purpose of the idealised WM dataset is to test that the five different networks (FCN, ResNet-18, ResNet-34, U-Net and U-Net 3+) all work well before I attempt to use them for the real WM datasets, which are introduced in Section 3.1.2 and 3.1.3. In order to make sure the tests are as relevant as possible, this dataset has been created to mimic the features present in WM datasets, namely circular structures with a hollow interior. Therefore, as seen in Figure 3.3, the dataset consists of circles with various spacing and sizes. The white and solid circles represent the idealised myelin, while each circle's black interior represents an idealised axon.

The idealised WM dataset was provided by the thesis' co-supervisor Dr Tian Yuan. It was generated using code to model axonal cross-sections from Dr Yuan's own research [73].

In total, there are 400 images in the idealised WM dataset, each with a resolution of 1024×1024 pixels. As the dataset is large enough, a conventional 80/20 train-test split will be done. This means that 80% of the dataset will be used for training the DCNNs (i.e., 320 images) and the remaining 20% (i.e., 80 images) will be used for testing the DCNNs' segmentation ability.

3.1.2 Rat White Matter Dataset

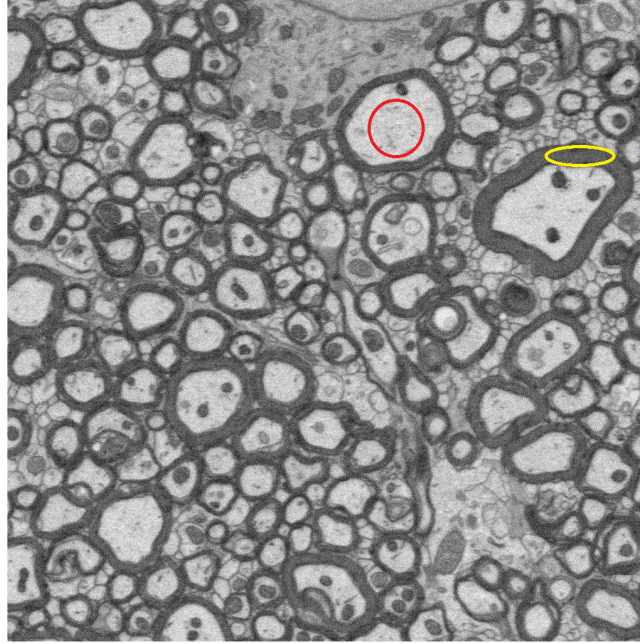


Figure 3.4: A slice of the rat WM dataset, imaged by Abdollahzadeh et al. [36]. To serve as an example, a section of myelin is contained within the yellow circle and a section of an axon is contained within the red circle.

The purpose of the rat WM dataset is to validate the performance of the DCNNs in this thesis compared to a state-of-the-art DCNN. The state-of-the-art DCNN is Abdollahzadeh et al.'s DeepACSON FCN [24], which was trained using the rat WM dataset.

The rat WM, depicted in Figure 3.4, was imaged by Abdollahzadeh et al. [36] using a Serial Block-face Scanning Electron Microscope (SBEM). The volume of the imaged WM sample was $15.36 \times 15.36 \times 14.25 \mu\text{m}^3$, which resulted in a total of 285 individual images, each with a spatial resolution of $15 \times 15 \times 50 \text{ nm}^3$.

An example of myelin and an example of an axon is marked in Figure 3.4. Compared to the sheep WM dataset, which is introduced in the next Section, and the idealised WM dataset, the intensity values in the rat WM dataset are reversed. This means that the myelin is low intensity in the rat WM dataset, and both the axons and background are high intensity.

As rat WM dataset has 285 images, each with a resolution of 1024×1024 pixels, a conventional 80/20 train-test split may not have enough training data. Therefore, to be safe, a 85/15 train-test split was used instead, meaning that there are 242 training images and 43 testing images.

3.1.3 Sheep White Matter Dataset

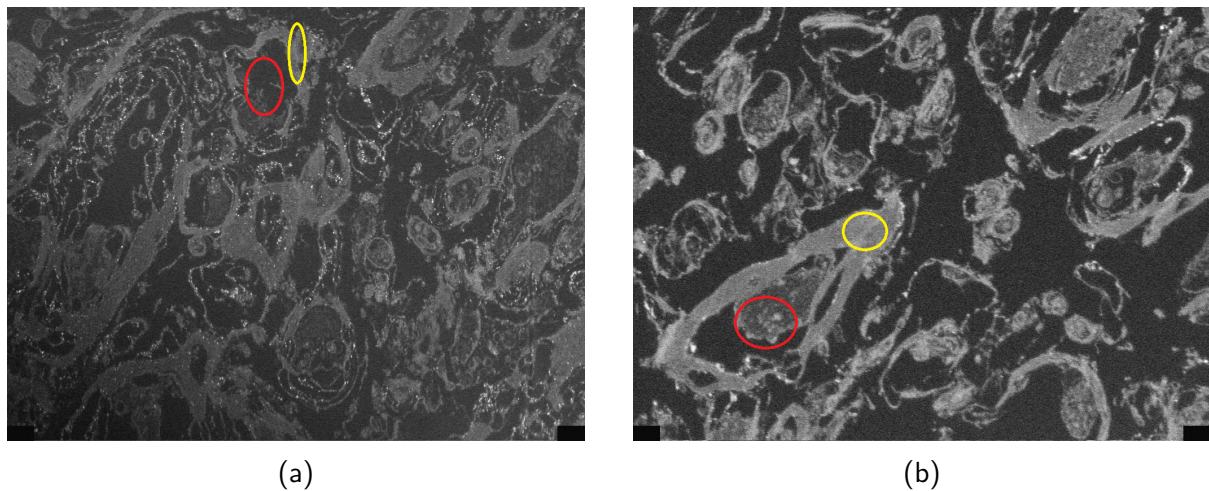


Figure 3.5: A slice of each sheep WM dataset: (a) Fornix01 and (b) Fornix02. The two datasets originate from the fornix of two different sheep and were imaged by Bernardini et al. [7]. To serve as an example, a section of myelin is contained within the yellow circle and a section of an axon is contained within the red circle.

The sheep WM dataset will be used to train a DCNN that can automatically segment sheep WM for the EDEN2020 project [2].

The sheep WM samples, depicted in Figure 3.4, were imaged by Bernardini et al. [7] using a Focused Ion Beam Scanning Electron Microscope (FIB-SEM). The volume of the imaged WM samples was $15.36 \times 20.48 \times 10.2 \mu\text{m}^3$, which resulted in a total of 68 individual images, each with a resolution of $20 \times 20 \times 150 \text{ nm}^3$. The Fornix01 dataset (Figure 3.5a) and the Fornix02 dataset (Figure 3.5b) come from two different sheep, but both are WM samples from the fornix region of the brain.

An example of myelin and an example of an axon is marked in Figure 3.5. Compared to the rat WM dataset in Figure 3.4, the structures of the myelin and the axon are not as well defined in the sheep WM datasets. This adds an additional layer of complexity to the task of training DCNNs using these datasets.

Choosing a train-test split for Fornix01 and Fornix02 is tricky, as they only have 68 images in the dataset. Using a train-test split of 90/10 or lower is not feasible, as the testing set will not be big enough for any statistical significance. However, increasing the size of the test set will have a significant impact on the DCNN training quality. Therefore, the Python package “Albumentations” [74] is used to artificially increase the size of the training dataset using image augmentations. The use of image augmentations, such as rotations, cropping, warping and mirroring, to add additional images to the training dataset is fairly common, especially for the medical field where large datasets are rare [75, 76]. To train these DCNNs, an initial 80/20 train-test split is used to obtain a testing set of 14 images. Albumentations is then

used to double the training dataset from 54 images to 108. This is still a relatively small dataset, but it is significantly better than before.

3.2 Creating the Training Labels

None of the WM datasets had segmentation masks available that could be used “as is” for training DCNNs. In this Section, the methodology used to create the segmentation masks for each of the three datasets will be described. Any code that is used in this Section is available on GitHub.

3.2.1 Idealised Dataset

To create the segmentation mask for the idealised WM dataset from Section 3.1.1, the MATLAB function “`imfindcircles`” [77] was used to extract the radius and centre coordinates from all the circles in the dataset. Then, for each circle in the dataset, the area of a filled circle with the respective parameters is inserted into an empty image array to create the segmentation mask. The MATLAB code for this is available on this project’s GitHub [78] in the “`create_labels`” folder as “`idealised_dataset_mask.m`”.

A slice of the generated segmentation mask is depicted in Figure 3.6. Here, the idealised myelin and idealised axons are labelled together, forming a myelin-axon pair.

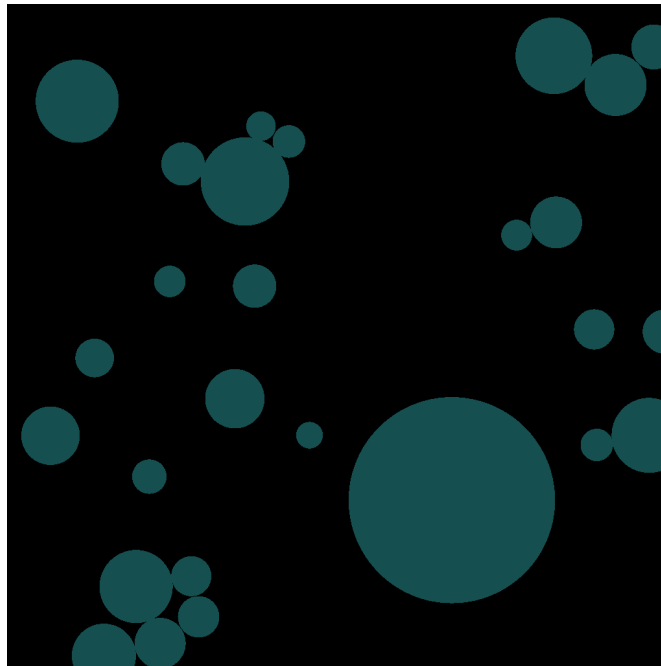


Figure 3.6: A slice of the idealised WM segmentation mask. (Colour scheme: turquoise = idealised myelin-axon pair).

3.2.2 Rat White Matter

To create the segmentation masks for their DeepACSON DCNN, Abdollahzadeh et al. [24] used their previously developed algorithmic-based automatic segmentation pipeline “Automatic 3D Segmentation Of axoNs” (ACSON) [36]. To develop ACSON, they created a technique named Bounded Volume Growing (BVG), which combines edge detection and seeded

region growing algorithms [79] in three dimensions. As seen in Figure 3.7, ACSON labels the myelin (turquoise) as separate from the axons (purple). Unlike for both the idealised WM and the sheep WM datasets, the segmentation mask for the rat WM separates axons and myelin. As the training labels used to train DeepACSON are not publicly available, the rat WM training labels were recreated for this thesis using ACSON [36]. The MATLAB code for ACSON is available on Dr. Abdollahzadeh's GitHub [80].

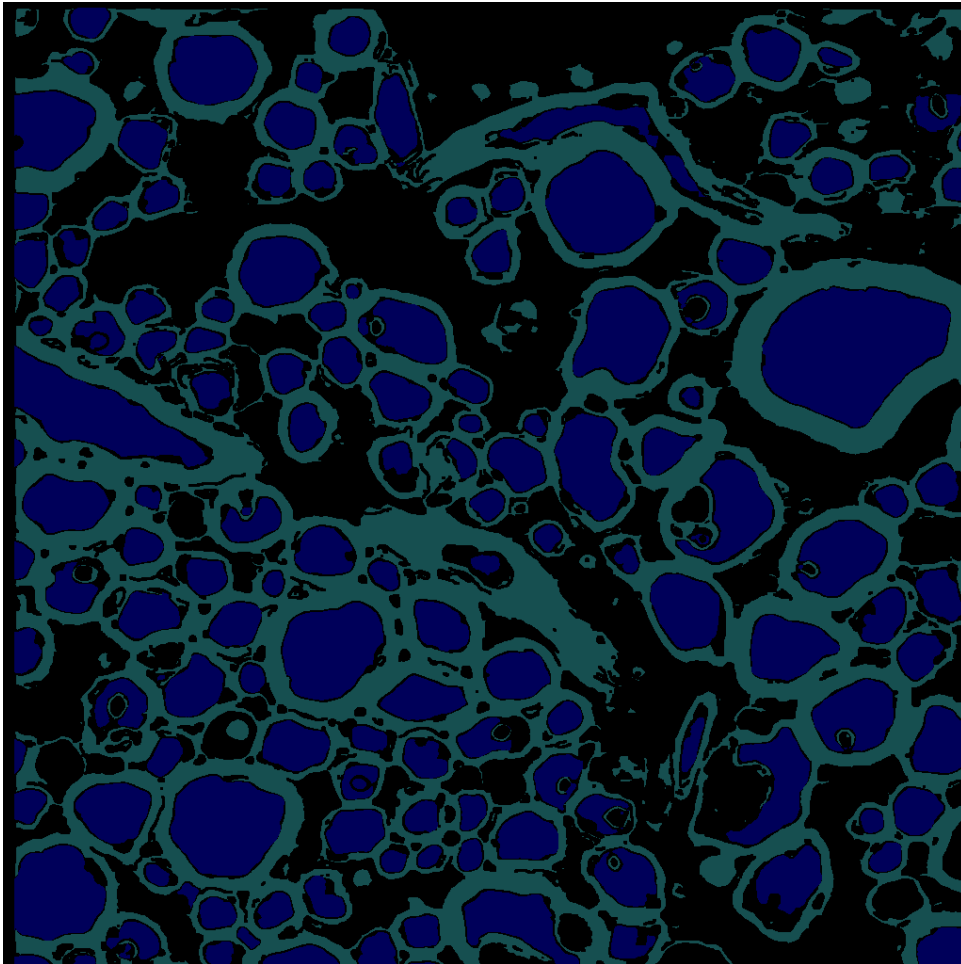


Figure 3.7: A slice of the rat WM segmentation mask, created for this thesis using ACSON [36]. Unlike the segmentation mask for the idealised WM dataset and the sheep WM dataset, ACSON generates a segmentation mask which separates axons and myelin. (Colour scheme: turquoise = myelin, purple = axon).

3.2.3 Sheep White Matter

To manually segment the sheep WM datasets, Bernardini et al. [7] used the Materialise Mimics software [81] to trace around the perimeter of the myelin present in each image slice. The traced perimeters are then filled to create the full segmentation areas. As a result, the myelin and axons are labelled together, forming a myelin-axon pair.

The available data on these segmentations is in the form of individual three-dimensional

reconstructions for each myelin-axon pair, saved in “.stl” files. As these only contain a point cloud of the segmentation perimeter, the full segmentation areas need to be restored before training DCNNs. Figure 3.8a shows a magnified view of the point cloud perimeter, whose area can only be filled if the perimeter is continuous. To make it continuous, the MATLAB function “imdilate” [82] performs morphological dilation on the image. Since the point cloud is dense enough, using a one-pixel radius structuring element for the dilation was enough to connect all the pixels and create a continuous perimeter, as depicted in 3.8b. Finally, the “imfill” [83] MATLAB function was used to fill the area enclosed by all continuous perimeters in the image, creating the final segmentation mask seen in Figure 3.8c.

The MATLAB code for this procedure is available on this project’s GitHub [78] in the “create_labels” folder as “Fornix_dataset_mask.m”.

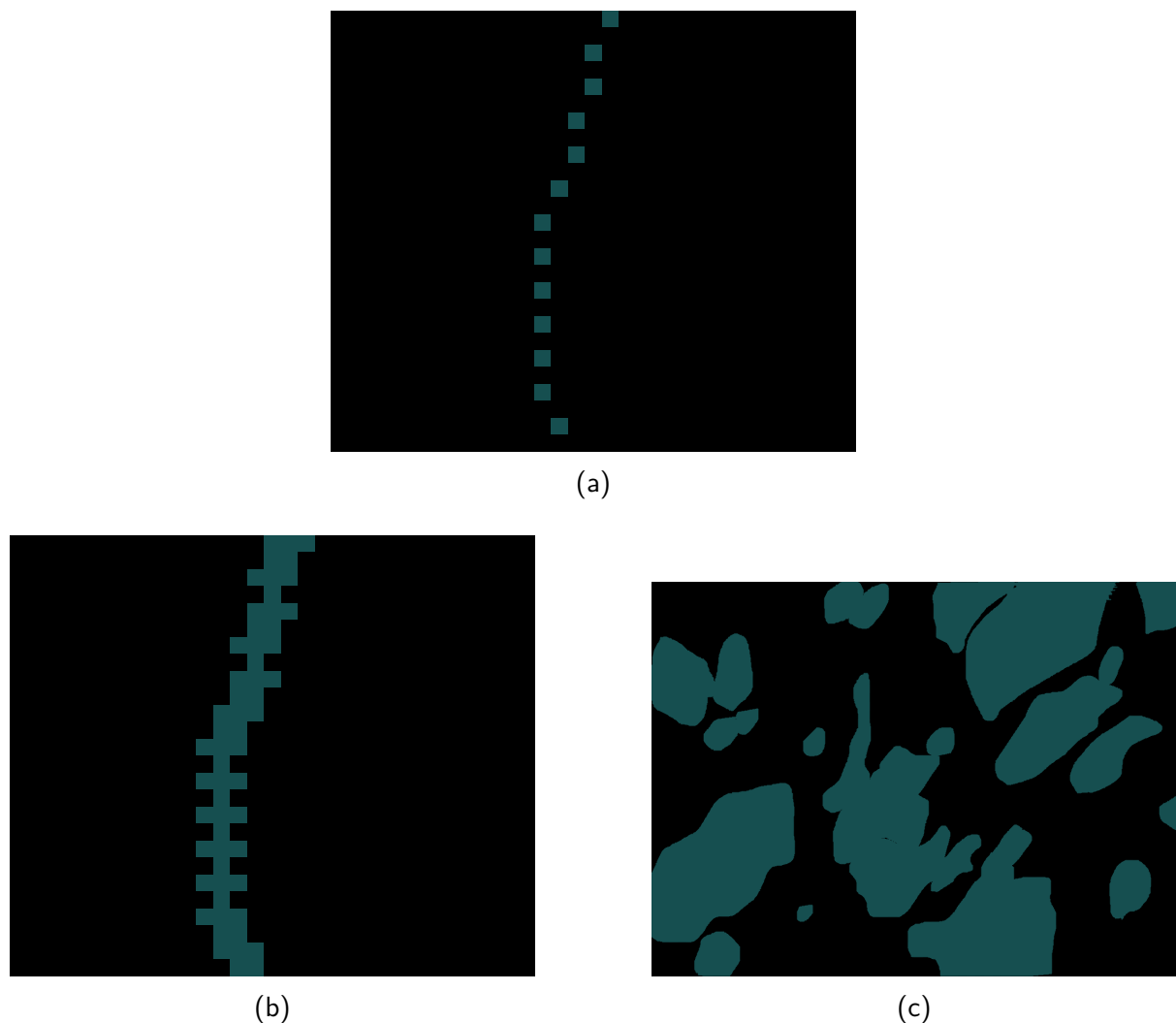


Figure 3.8: (a) A magnified view of the point cloud available from Bernardini et al. [7]. The magnification was necessary as the one pixel wide perimeter is otherwise too thin to be seen. (b) A magnified view of the continuous point cloud from the image dilation. (c) A slice of the final segmentation mask from the Fornix02 sheep WM dataset. (Colour scheme: turquoise = idealised myelin-axon pair)

3.3 Preprocessing the Datasets

All of the preprocessing methods discussed in this Section have their own respective MATLAB scripts, which are available on this project's GitHub [78] in the “preprocessing_scripts” folder.

Each preprocessing method described in this Section, except denoising and homogenisation of background illumination, was chosen with the intent to improve upon the segmentation results from datasets preprocessed with the procedures described before them. Therefore, in addition to the description of the method, these preprocessing procedures' Sections have a justification that briefly explains why they were performed. The full explanations are found in the Results and Discussion Chapter of the thesis.

3.3.1 Denoising

Both the rat White Matter (WM) dataset introduced in Section 3.1.2 and the sheep WM datasets introduced in Section 3.1.3 are denoised before being used for training and testing the DCNN models.

The Block-Matching and 3D filtering (BM3D) denoising algorithm [84] was used to denoise all the datasets using the bm3d Python package available on PyPI [85]. This was implemented in the Python script “denoising.py” found on GitHub [78].

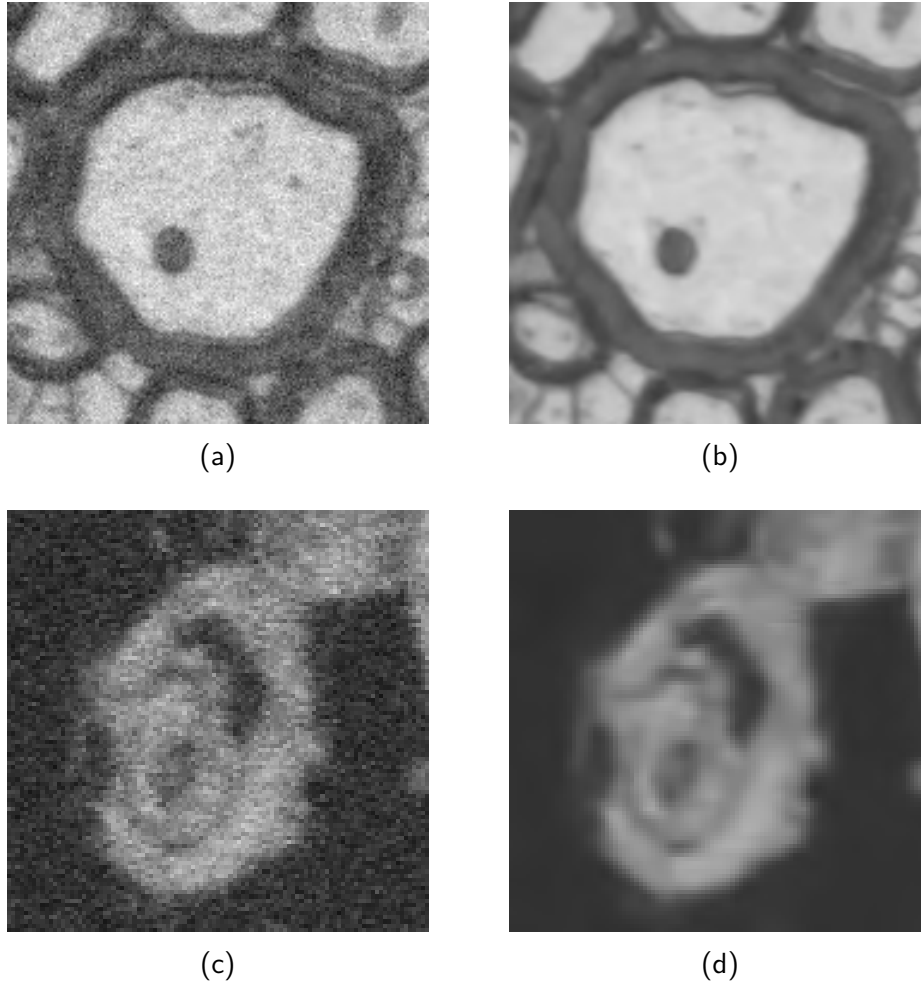
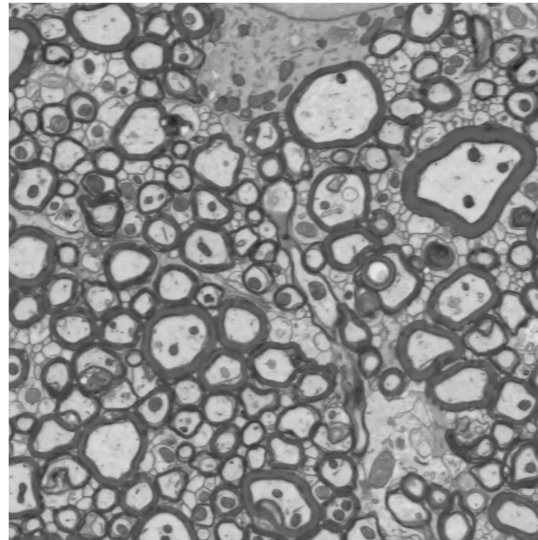
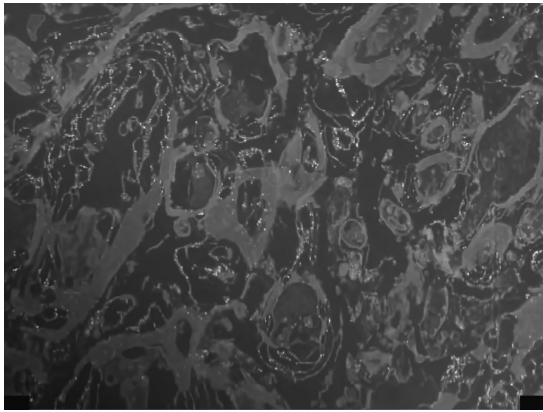


Figure 3.9: The effect of denoising the (a) raw rat and (c) raw sheep WM datasets is depicted in (b) and (d). The BM3D denoising algorithm [85] was used for this procedure.

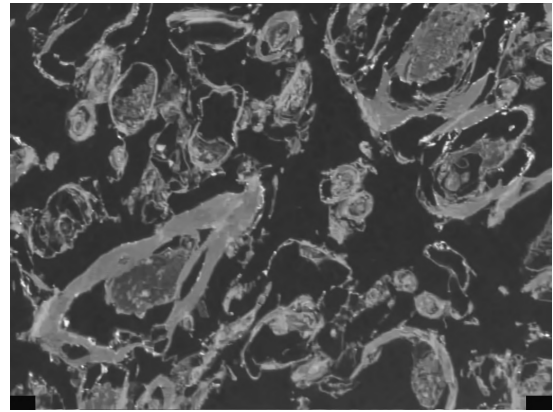
Figure 3.9 shows the effect of denoising on the rat brain WM and sheep brain WM datasets. Simply put, the denoising algorithm smooths out the sharp pixel-to-pixel intensity gradients in the images. The final result of this preprocessing operation on a slice of the three different datasets is shown in Figure 3.10 below.



(a)



(b)



(c)

Figure 3.10: A slice of the (a) denoised rat WM and the two denoised sheep WM samples, (b) Fornix01 and (c) Fornix02.

Denoising is a standard procedure done to make sure that there are fewer irregular intensity gradients in the images, which interfere with DCNN training and segmentation. In Abdollahzadeh et al.'s paper on DeepACSON [24], the source for the rat brain WM dataset, it is reported that the segmentation performance improved when the rat brain datasets were denoised. Other studies have also shown that DCNN performance is improved when both the training and testing datasets are denoised [86].

3.3.2 Homogenising the Background Illumination

In this preprocessing technique, which was applied to the datasets after it was denoised as described in Section 3.3.1, regions of the images that are inhomogeneously bright are removed. This and the other remaining preprocessing techniques described in Section 3.3 are only applied to the sheep WM datasets.

To homogenise the background illumination, morphological opening has first been used to re-

move features in the foreground (i.e., structures of the WM). This is done using the MATLAB function “`imopen`” [87], which performs morphological erosion followed by morphological dilation. This only leaves the background illumination, which is initially inhomogeneous, as seen in Figure 3.11a. Subtracting the resulting background illumination from the denoised dataset will homogenise the image’s illumination, which is depicted in Figure 3.11b for Fornix01 and Figure 3.11c for Fornix02. This preprocessing procedure is adapted from Bernardini et al.’s paper [7], the source of the sheep brain dataset. A MATLAB script called “`homogeniseBackground.m`” implements this procedure, which is available on GitHub [78].

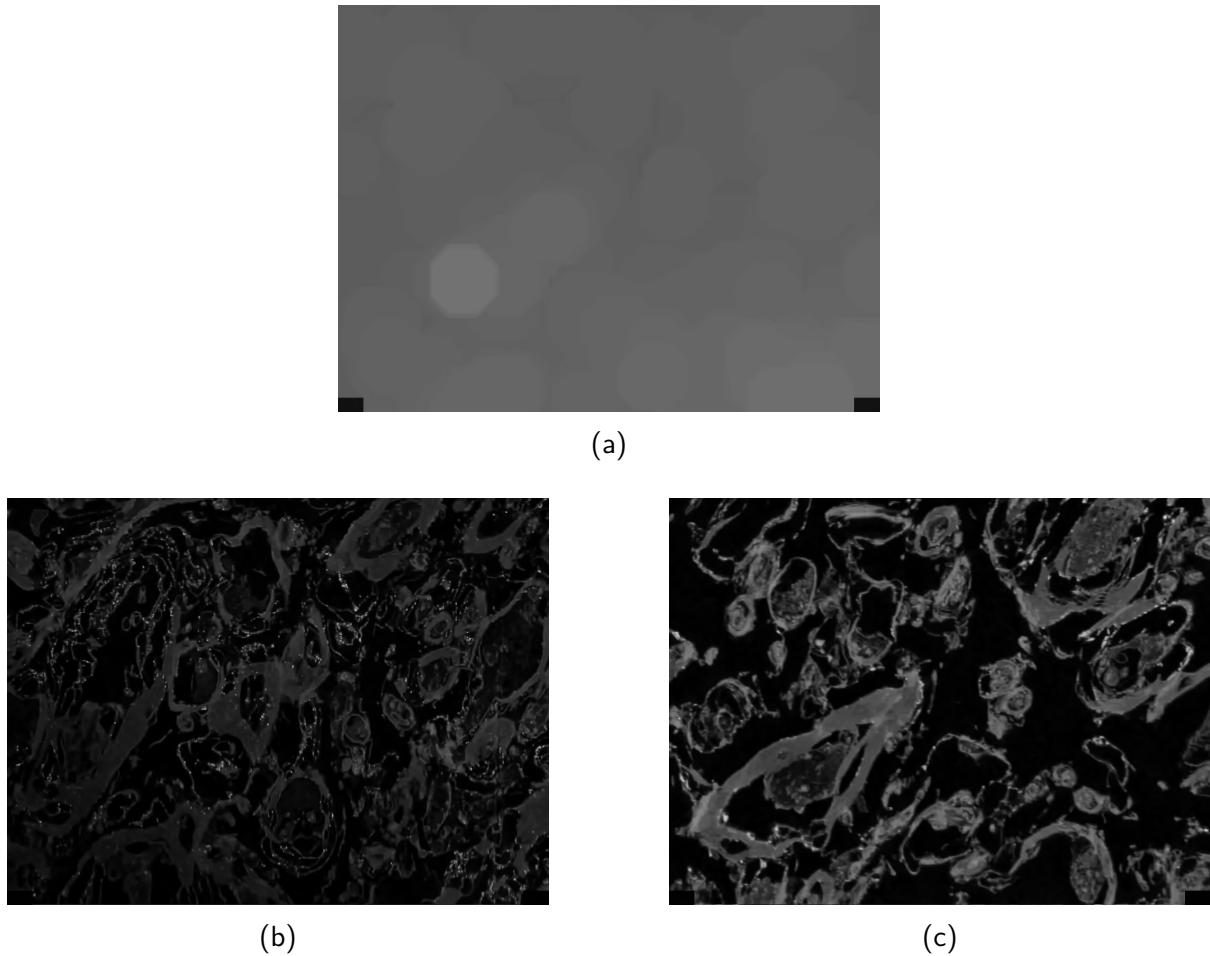


Figure 3.11: (a) An example slice of inhomogeneous background illumination, which is subtracted from the denoised sheep WM. A slice of the (b) Fornix01 and (c) Fornix02 samples with homogenised background illumination.

The datasets produced by this preprocessing step are not used to train any DCNNs. Instead, the purpose of this preprocessing step was to ensure that the illumination within each image of the dataset is more uniform. This was necessary before any further preprocessing techniques were performed, as will be described in Section 4.2.1.

3.3.3 Binarisation

In binarisation, all the high intensity (white) structures in the image past a certain intensity threshold are set to the maximum intensity (255), while all low intensity structures are set to the minimum intensity (0). This preprocessing step is performed on datasets that have both been denoised as described in Section 3.3.1 and have had their background illumination homogenised as described in Section 3.3.2.

Instead of applying a single intensity threshold for the entire image, the MATLAB function “`adaptthresh`” [88, 89] is used. This function computes a separate intensity threshold value for each pixel by comparing the average illumination in the neighbourhood of the pixel to the rest of the image. This allows the threshold to be adapted to the overall illumination of the specific dataset. In addition to this, the binarisation will also be invariant to any uneven background illumination that may remain after the homogenisation from Section 3.3.2. The results of this preprocessing procedure are depicted in Figure 3.12a for Fornix01 and Figure 3.12b for Fornix02. The “`binarisation.m`” MATLAB script available on GitHub [78] implements this procedure.

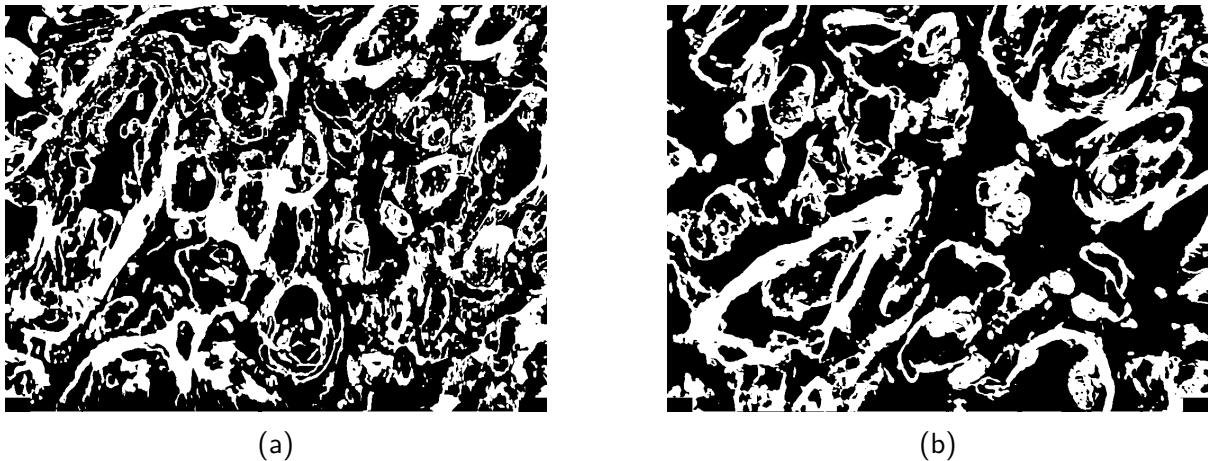


Figure 3.12: A slice of the binarised (a) Fornix01 and (b) Fornix02 sheep WM samples. The binarisation was performed using the “`adaptthresh`” [88, 89] MATLAB function.

As will be described further in Section 4.2.1, the reason why this preprocessing technique was used is to remove the discrepancy in intensity between the Fornix01 and Fornix02 datasets. This discrepancy is seen in Figures 3.10b and 3.10c in Section 3.3.1, where the brightness (i.e., intensity) of the myelin in the Fornix02 dataset is noticeably higher than for the Fornix01 datasets. After binarisation, both datasets’ myelin has the same intensity.

3.3.4 Remapping the Intensities

To remap the intensity of a dataset, the range of intensities available in the dataset are stretched to a new intensity range, in this case the whole spectrum (i.e., 0 to 255). This preprocessing step is performed on datasets that have both been denoised as described in

Section 3.3.1 and have had their background illumination homogenised as described in Section 3.3.2.

The intensity remapping is done using the MATLAB function “`imadjust`” [90]. This function sets the image’s lowest 1% of intensities to 0 and the highest 1% to 255, while the intensities between are distributed linearly across the full range. The results of this preprocessing procedure are depicted in Figure 3.12a for Fornix01 and Figure 3.12b for Fornix02. A MATLAB script named “`intensityRemapping.m`” implements this procedure, which is available on this project’s GitHub [78].

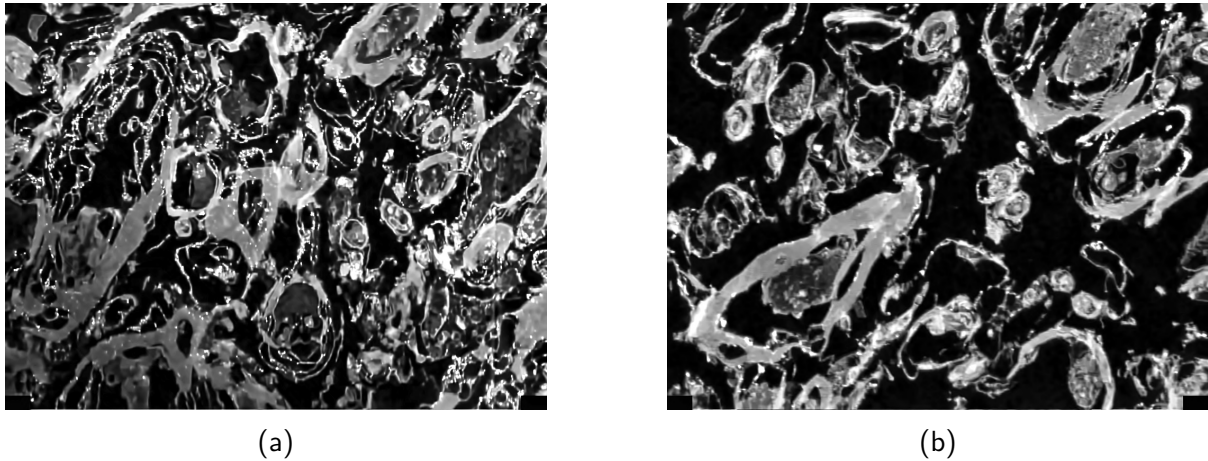


Figure 3.13: A slice of the (a) Fornix01 and (b) Fornix02 sheep WM samples with their intensities remapped using the “`imadjust`” [90] MATLAB function.

Similarly to binarisation, intensity remapping is also used to remove the discrepancy in intensity between the Fornix01 and Fornix02 datasets. However, as will be described further in Section 4.2.2, this preprocessing technique was chosen because, unlike binarisation, intensity remapping preserves the intensity gradients within the image.

3.3.5 Feature Decluttering

The final preprocessing technique is to remove features which resemble myelin-axon pairs yet have not been labelled in the manual segmentation from the sheep brain WM dataset. For this thesis, such features will be referred to as clutter. Before a dataset is decluttered, it undergoes denoising as described in Section 3.3.1, homogenisation of the background illumination as described in Section 3.3.2, and lastly, intensity remapping as described in Section 3.3.4.

To remove the unlabelled features, the dataset is compared with the corresponding manual segmentation mask. Any features outside the labelled region and above a certain area threshold are set to black (zero intensity). This is achieved by separately evaluating the area threshold on the binarised version of the dataset (see Section 3.3.3) and then temporarily adding the areas below the threshold to the segmentation mask. In this context, the area is the number of pixels which are connected to each other in the binary image. Finally, any

pixels outside the temporary segmentation mask are set to black. To retain some smaller, unlabelled features in the dataset, the area threshold is set to 5000 pixels. The results of this preprocessing procedure are depicted in Figure 3.14b for Fornix01, next to the intensity-remapped Fornix01 dataset before it was decluttering for comparison in Figure 3.14a. The MATLAB script “featureRemoval.m”, available on this project’s GitHub [78], performs this preprocessing step.

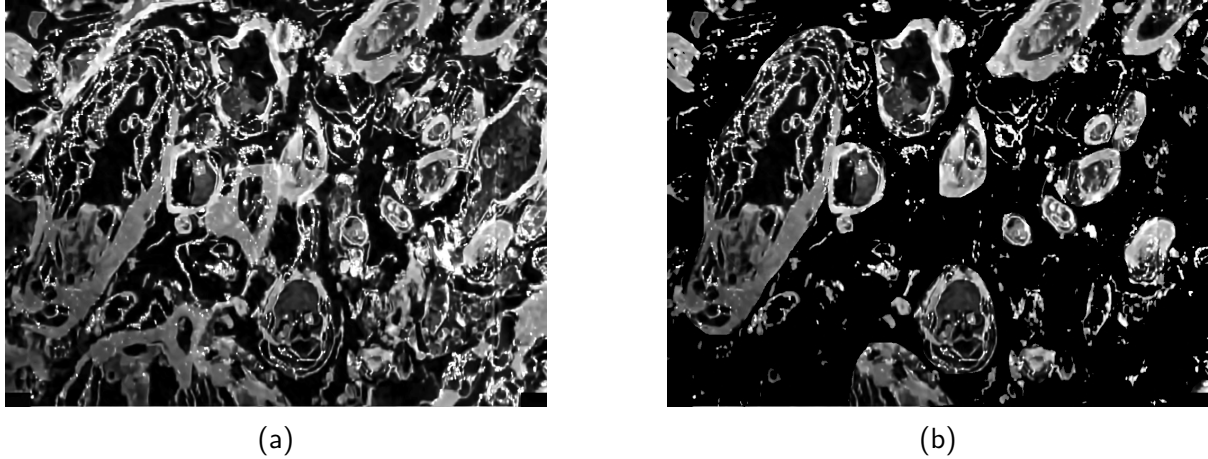


Figure 3.14: A slice of the (a) intensity-remapped Fornix01 and (b) decluttered Fornix01 sheep WM samples.

Unlike the previously discussed preprocessing techniques, decluttering datasets cannot be performed on new datasets that need to be segmented, as it requires knowledge of the segmentation ground truth (i.e., the correct segmentation) in advance. In this case, we have access to the manual segmentation, allowing us to declutter the Fornix01 dataset. As will be discussed further in Section 4.2.3, decluttering is performed to test the segmentation performance of the high-clutter Fornix01 dataset when it has a similar amount of clutter as the Fornix02 dataset.

3.4 Construction of the DCNNs

In the following, this thesis's implementations of the DCNN architectures described in Section 2.3 of the Literature Review will be discussed. A total of 5 networks have been created, with different levels of complexity in terms of architecture (i.e., the backbone of the network) and in terms of parameter count (i.e., the number of trainable weights in the network). Testing a wide range of networks is necessary, as it is not a given that the most advanced network architecture will perform best for the sheep WM dataset. This is especially true in this case because, as mentioned in Section 3.1.3, the number of samples available for training is relatively low. A possible consequence of a low sample count combined with high network complexity is overfitting. This is when the network memorises the training data and is unable to perform well on unseen data [91].

In this Section, my contribution to the creation of the DCNN code will be described.

The PyTorch code for each of the DCNNs can be found on this project's GitHub [78] in the "training_scripts" folder.

3.4.1 Fully Convolutional Network

The Fully Convolutional Network (FCN) for this thesis was constructed from the ground up using the basic convolutional layers and max pooling layers described in Section 2.2.1 and 2.4, respectively. Several transpose convolutional layers are used to perform the upsampling necessary for image segmentation, which are able to perform feature extraction and upsampling simultaneously, as described in Section 2.3.1. Constructing the FCN in this manner mimics the encoder-decoder design of the UNet architecture described in Section 2.3.3 without the addition of skip connections.

This architecture has a total of 12.5 million parameters.

3.4.2 ResNet-18

The ResNet-18 is an 18-layer FCN with a ResNet backbone. This is one of the ResNets defined in the original paper [58]. As the original implementation was for image classification, the DCNN classifier had to be replaced with a self-designed decoder. Similarly to the FCN described in Section 3.4.1, convolutional layers are used to perform upsampling.

This architecture has a total of 12.7 million parameters.

3.4.3 ResNet-34

The ResNet-34 is a 34 layer FCN with a ResNet backbone. This is also one of the ResNets defined in the original paper [58]. As the original implementation was for image classification, the DCNN classifier had to be replaced with a self-designed. Similarly to the FCN described in Section 3.4.1, convolutional layers are used to perform upsampling.

This architecture has a total of 26.2 million parameters.

3.4.4 UNet

The UNet architecture was taken “as is” from the original UNet paper [64].

This architecture has a total of 31 million parameters. This is the network with the highest parameter count in this thesis.

3.4.5 UNet 3+

The UNet 3+ architecture was taken “as is” from the original UNet 3+ paper [68].

This architecture has a total of 26.9 million. Even though the architecture is more complex than that for the UNet, Huang et al. [68] replaced many transpose convolutional layers with simple upsampling layers in order to reduce the parameter count and likelihood to overfit.

3.5 Training the DCNNs

The training of the DCNNs is conducted using Python scripts that contain:

- Code for the DCNN architecture, written using the PyTorch library.
- Pytorch DataLoaders for “.mat” files, which contain the datasets and labels.
- The training loop with all relevant training parameters, which are outlined in this Section.
- A validation loop using the F1 score (introduced in Section 3.6.3).
- Code to perform the inference (i.e., segmentation) on the test dataset.

The Python scripts for each of the 5 DCNN architectures from Section 3.4 are available on this project’s GitHub [78] in the “training_scripts” folder.

To train the DCNNs, the cross entropy loss criterion is used with the Adam optimiser. For the Adam optimiser, a learning rate of 10^{-4} , weight decay of 10^{-5} and betas of (0.9, 0.999). Additionally, early stopping has been implemented to the training loop to prevent overfitting. The early stopping will finish the training process early if the cross entropy loss of the validation set does not decrease for ten consecutive epochs (i.e., ten passes through the whole training dataset).

As DCNNs with millions of parameters can take several days to train on a CPU, GPUs from the Imperial High Performance Computing (HPC) cluster were used to train all networks, which reduces the training time to several hours at most. The following hardware specifications were used for all training scripts: one L40S 48GB GDDR6 GPU, 192GB of RAM and one generic CPU for basic data processing within the scrip.

3.6 Quantifying the Quality of the Segmentation

To quantify the quality of the DCNN model's segmentation masks, the confusion matrix components are used. While confusion matrix itself is commonly used to evaluate the performance of classification models [92], it is the matrix's individual components that are useful for image segmentation [93]. These components are:

- True Positive (TP): the number of pixels the model has correctly labelled as part of the subject.
- False Positive (FP): the number of pixels the model has incorrectly labelled as part of the subject.
- False Negative (FN): the number of pixels the model has incorrectly labelled as part of the background.
- True Negative (TN): the number of pixels the model has correctly labelled as part of the background.

These four components can be used to calculate useful metrics for evaluating segmentation quality, such as precision, recall and the F1 score [93], which are all introduced in this Section.

3.6.1 Precision

Precision gives a measure of how frequently the DCNN model is correct when it labels a pixel as the subject [94]. This metric is useful to distinguish between a model that is actually segmenting the subject well and a model that is segmenting too much just to capture all the subject pixels. The TP value will be the same for both cases, but the precision in Equation 3.1 will be low due to the large amount of FP.

$$Precision = \frac{TP}{TP + FP} \quad (3.1)$$

As is evident from Equation 3.1, if the segmentation is perfect, meaning that there are no false positives, then the precision will be equal to one.

3.6.2 Recall

Recall is a way to measure if the DCNN model is able to find all the pixels belonging to the subject [94]. This metric is useful to verify and strengthen the value of the model's precision metric. For example, if a model is reluctant to label pixels belonging to the subject and only does so when it is certain, then the precision will be very high. However, the recall in Equation 3.2 will be low as many subject pixels will have been incorrectly labelled as background (FN).

$$Recall = \frac{TP}{TP + FN} \quad (3.2)$$

As is evident from Equation 3.2, if the segmentation is perfect, meaning that there are no false negatives, then the recall will be equal to one.

3.6.3 The F1 Score (Dice Coefficient)

Lastly, the F1 score (also called the Dice Coefficient) is a measure of the overlap between the ground truth and the model's segmentation [93]. This metric is the harmonic mean of the precision and the recall [95], which can be simplified into the confusion matrix components, as seen in Equation 3.3.

$$F1\ Score = \frac{2 \times Precision \times Recall}{Precision + Recall} = \frac{2 \times TP}{2 \times TP + FP + FN} \quad (3.3)$$

As is evident from Equation 3.3, if the segmentation is perfect, meaning that there are no false positives, then the F1 score will be equal to one.

3.6.4 Three Class Segmentation

To use the Equations 3.1 to 3.3 for evaluating three class segmentation results, which needs to be done for the rat WM dataset in Section 4.1.2, the confusion matrix components need to be redefined slightly. Instead of dealing with subject versus background and obtaining one value for each component, for three classes the distinction is "class X versus the other classes". Therefore, each class has its own set of confusion matrix components. Using these, a separate precision, recall and F1 score need to be calculated for each class. Each of the four metrics can then be used individually to gauge the model's ability to segment a certain class or be averaged over all the classes to evaluate the total performance.

Chapter 4

Results and Discussions

This Results and Discussion Chapter is divided into two main Sections: the validation of the Deep Convolutional Neural Networks (DCNN), using the idealised White Matter (WM) dataset and the rat WM dataset, and the creation of the DCNN models for segmentation of the sheep WM datasets from a sub-study [7] of the EDEN2020 [2] project.

The validation Section 4.1 is divided into two Subsections. In Subsection 4.1.1, the results from the segmentation of the idealised WM are presented and discussed. The most important result is to validate whether the DCNNs are able to properly segment idealised myelin-axon pairs with high accuracy. Nevertheless, the various minor imperfections of the segmentations are still examined in detail to gain a better understanding of each network architecture's strengths and weaknesses.

In Subsection 4.1.2, the rat WM dataset is used to validate this thesis's networks against DCNNs from the literature. In particular, the comparisons will be made to DeepACSON [24], an FCN created by Abdollahzadeh et al. to segment this rat WM dataset.

Lastly, in Section 4.2 the results from the DCNN segmentations of the sheep WM datasets will be presented and discussed. This Section is divided into the various preprocessing techniques that have been used to improve the segmentation results. For each preprocessing technique, a DCNN trained on the Fornix02 dataset was tested using the Fornix01 dataset and vice versa. This way it can be properly tested if the network is useful for automatic segmentation, as future applications of the DCNNs would not be on the same individual sheep.

For each of the Sections, the main qualitative segmentation results are displayed as a collection of six figures, one for the ground truth segmentation (Subfigure a) and one for each of the five models' results (Subfigures b to f). In these figures, the segmentation mask, which is turquoise for myelin-axon pairs, is overlaid with a low opacity on a full, unmagnified slice of the WM image. For the rat WM datasets, where myelin and axons are segmented individually, myelin is turquoise, and axons are purple. For any additional figures of the results, where

a particular aspect of the segmentation is being examined in a magnified view, the colour scheme may change. In some of these figures, the pixel colour is dependent on that pixel's respective confusion matrix component (introduced in Section 3.6), as follows: blue for a True Positive (TP) segmentation, red for a False Positive (FP) segmentation and green for a False Negative (FN) segmentation.

In each Section, the quantitative results are listed in a Table, which consists of three different segmentation evaluation metrics for each of the five DCNN models. The three different metrics are precision, recall and the F1 score, which are all introduced in Section 3.6. As described in Section 3.6.3, the F1 score is a combination of precision and recall. Therefore, it will be used as the primary metric for all discussions, except when the precision or recall values themselves are noteworthy.

4.1 Segmentation of Validation Datasets

4.1.1 Idealised White Matter

The purpose of the idealised WM dataset is to test if the five different networks (FCN, ResNet-18, ResNet-34, UNet and UNet 3+) are able to handle the structures present in WM datasets. The difficulty associated with segmenting these structures is that the network needs to distinguish between the black areas within the circles (i.e., idealised axons) and the black area outside (i.e., the image background). If the DCNN models are not able to make this distinction, then they are unlikely to be able to segment real WM datasets. The evaluation of the models' performance on this dataset will mainly be qualitative, as the dataset's basic circular structures should allow the models to nearly replicate the ground truth labels perfectly.

After the DCNNs have been validated, various minor segmentation errors will be examined in detail to gain a better understanding of each network architecture's strengths and weaknesses.

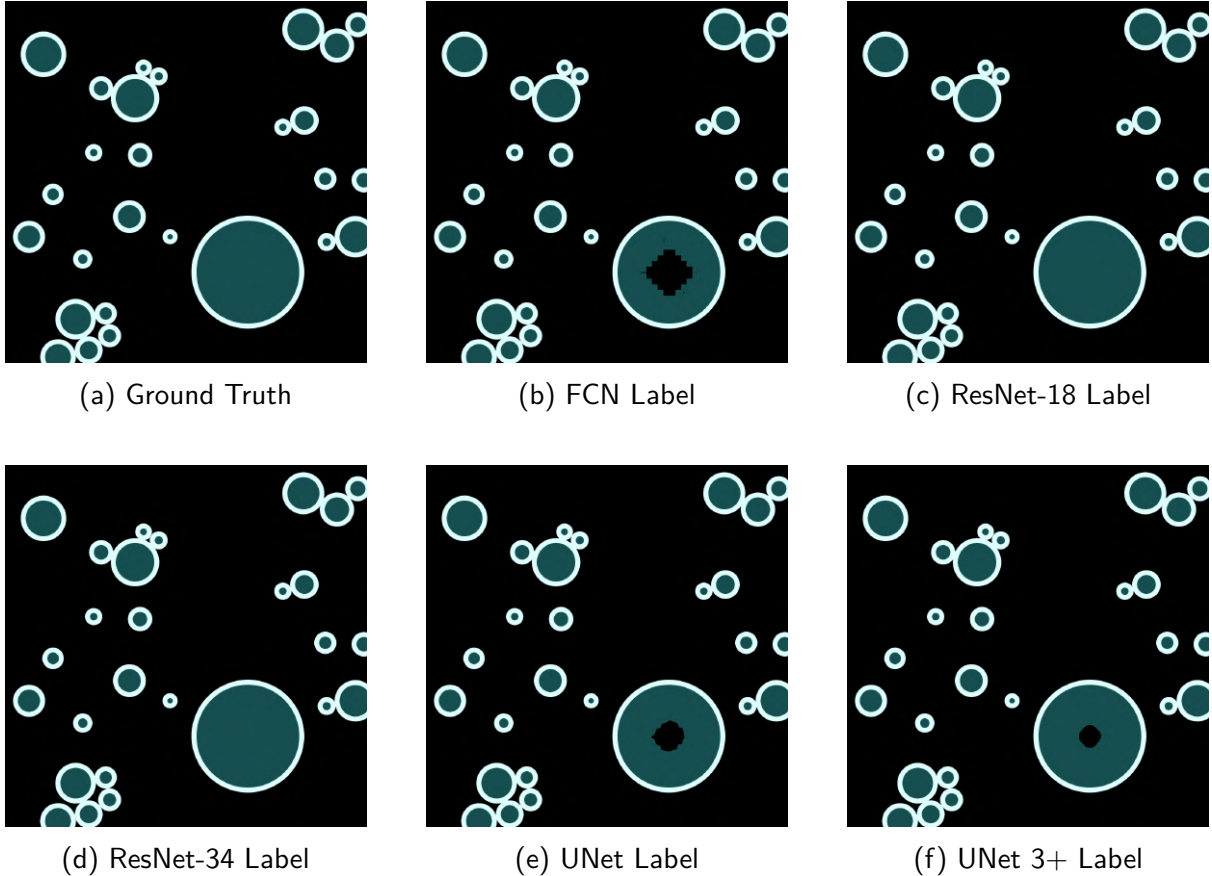


Figure 4.1: A slice of the (a) ground truth segmentation and the (b-f) segmentation results from the five models of the idealised WM dataset. (Colour scheme: turquoise = idealised myelin-axon pair)

Figure 4.1 depicts a slice of the idealised WM dataset overlaid by, in order, the labels from the ground truth, FCN, ResNet-18, ResNet-34, UNet and UNet 3+. Even though there are

some immediately obvious segmentation errors in Figures 4.1b, 4.1e and 4.1f, which will be examined in more detail together with additional errors present on other slices, the overall segmentations are great for all models. As previously mentioned, the purpose of segmenting the idealised WM dataset is to see if the models are able to distinguish between the black region inside the idealised myelin and the black background, which all were able to for most idealised myelin-axon pairs.

Table 4.1: Evaluation Metrics for the Segmentation of Idealised White Matter

Models	Precision	Recall	F1 score
FCN	0.994	0.987	0.990
ResNet-18	0.992	0.991	0.991
ResNet-34	0.992	0.991	0.991
UNet	0.991	0.993	0.992
UNet 3+	0.992	0.994	0.993

The evaluation metrics for each of the five models' segmentations are listed in Table 4.1. Since a perfect score for precision, recall and the F1 score is 1 (this is explained in Section 3.6), it is clear that all five models performed excellently for the segmentation of the idealised WM. Apart from the FCN having the highest precision, with the other models' precision all approximately equal, the rest of the metrics all increase with increasing architectural complexity.

The large holes present in the segmentation masks for the two UNets in Figures 4.1e and 4.1f make it seem counter-intuitive that they have higher F1 scores than the two ResNets, whose segmentation masks are fully filled (Figures 4.1c and 4.1d). However, this is compensated by the large quantity of small-scale errors that the ResNets exhibit throughout the dataset, examined in Sections 4.1.1.2 and 4.1.1.3.

4.1.1.1 Closer Examination of Segmentation Errors: Missing Centre

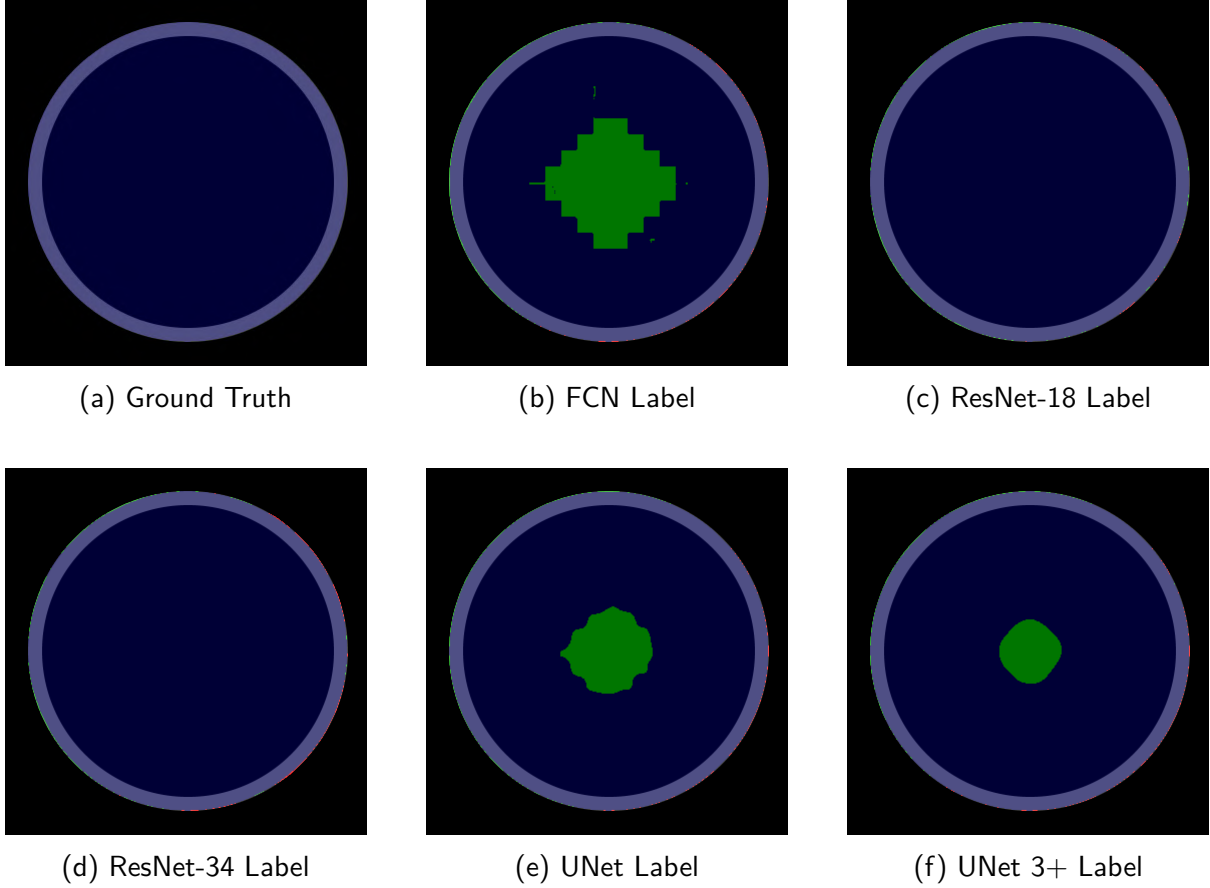


Figure 4.2: Examination of Segmentation Error: Magnified view of missing segmentation in the middle of large idealised myelin-axon pairs. (Colour scheme: Blue = TP, Green = FN, Red = FP).

Figure 4.2 is a magnified version of Figure 4.1, which examines the segmentation error where the centre of the largest idealised myelin-axon pair has not been labelled. The FCN, UNet and UNet 3+ models all display this error to various degrees, while the two ResNets (18 and 34) have fully segmented the centre of this myelin-axon pair. The FCN and UNet have further instances of this error on other slices of the dataset, whereas this is the only incident for the UNet 3+, and the two ResNets never produced it. As the error only occurs for the largest myelin-axon pairs in the dataset and the error is always concentrated in the centre, it is evident that it is related to a limitation of the models' Field of View (FOV). When the affected models are labelling the centre pixels, they are not able to detect that they are within the myelin-axon pair and, therefore, label the pixels as the background class. Based on the severity of the errors, it would indicate that the FCN has the lowest FOV, followed by the UNet and then the UNet 3+, while the two ResNets both have a sufficiently large FOV for all the structures in the dataset.

Another notable feature is the shape of the errors. The FCN's error in Figure 4.2b is almost

completely block-like, with 90-degree angles around the perimeter. Contrastingly, the UNet’s error in Figure 4.2e is very rounded yet pointed, whereas the UNet 3+’s error in Figure 4.2f is rounded and smooth. While these results are not entirely indicative of the models’ segmentation ability or resolution (since every model was able to trace the outer edge of the idealised myelin with high accuracy), they could reveal some general characteristics of the models’ architectures. For example, the block-like nature of the FCN’s segmentation is likely a result of the architecture’s simple encoder-decoder structure, which was outlined in Section 3.4.1. On the other hand, the addition of skip connections (described in Sections 2.3.3 and 2.3.4) to the encoder-decoder structure in the UNet and UNet 3+ architecture could be responsible for the more rounded segmentation. Unfortunately, as the ResNets do not display this kind of error, no conclusions about their architecture’s characteristics can be drawn.

4.1.1.2 Closer Examination of Segmentation Errors: Edge Merge

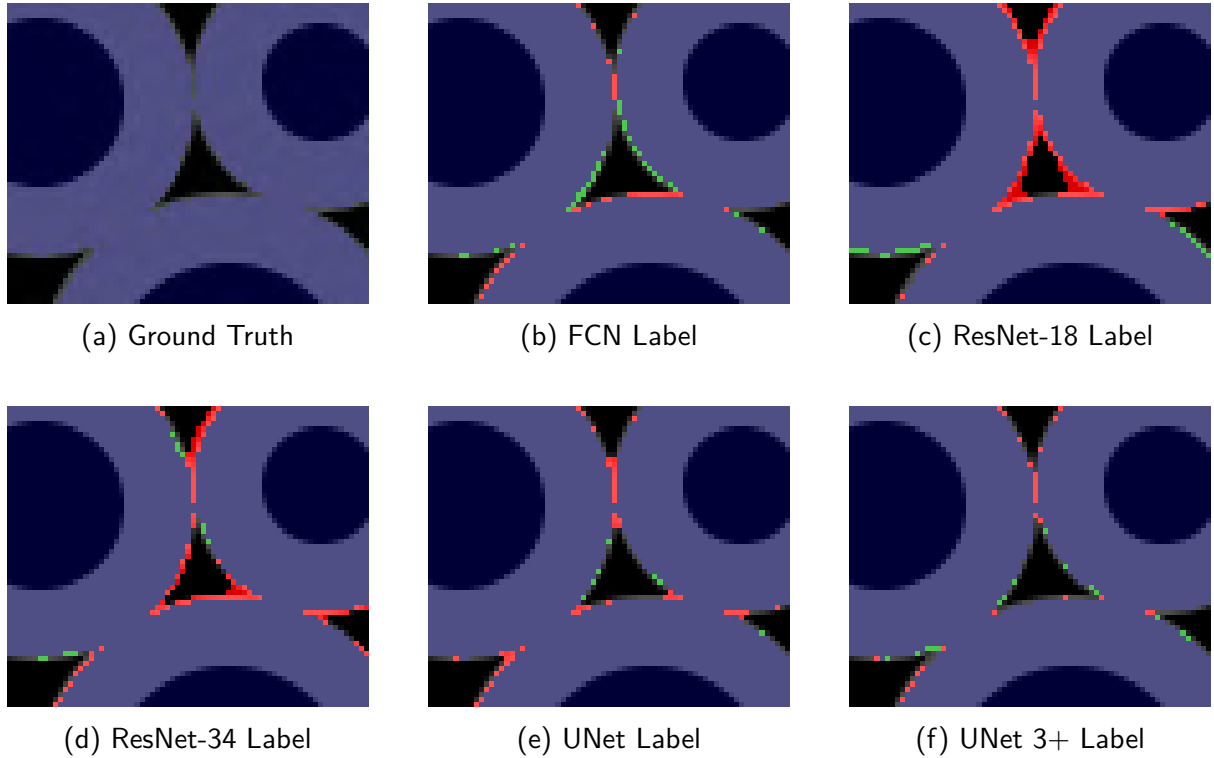


Figure 4.3: Examination of Segmentation Error: Magnified view of merging of the segmentation at edges of touching idealised myelin-axon pairs. (Colour scheme: Blue = TP, Green = FN, Red = FP).

Figure 4.3 is a very magnified view of an error that occurs at some of the contact points between idealised myelin-axon pairs. In this error, the pixels around the contact point are being over-labelled so severely that the edges of the two idealised myelin-axon pairs look merged. This error, displayed as the red FP pixels that extend into the triangular black shape between the three myelin-axon pairs in Figures 4.3c and 4.3d, is only present for the two

ResNets. Both the ResNet-18 and ResNet-34 make this error several more times throughout the dataset (another example of this error is seen in Figure 4.4). The frequent occurrence of this error is one of the reasons why the ResNets have a slightly lower F1 score than the UNets despite their full segmentation of the large myelin-axon pairs (Section 4.1.1.1).

4.1.1.3 Closer Examination of Segmentation Errors: Edge Missing

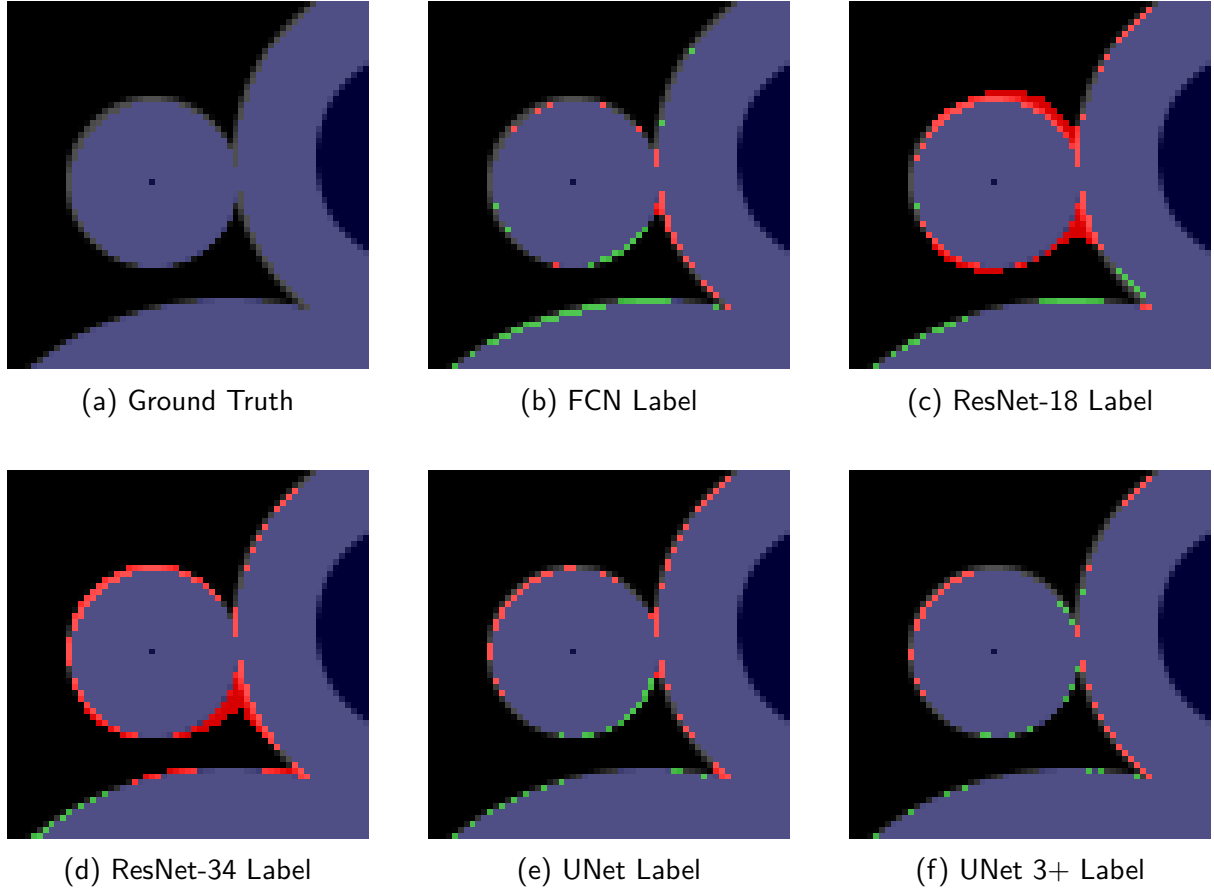


Figure 4.4: Examination of Segmentation Error: Magnified view of over-labelling along part of a small myelin-axon pair's outer edge. (Colour scheme: Blue = TP, Green = FN, Red = FP).

A further reason why the ResNet-18 and ResNet-34 have a lower F1 score than the two UNets in Table 4.1 is because of a phenomenon evidenced in Figure 4.4. Figures 4.4c and 4.4d show the error of the edges merging at the contact point of two myelin-axon pairs discussed previously in Section 4.3, but also another error, which is a general over- and under-labelling around the outer edges. In Figure 4.4, the two ResNets have strongly over-labelled the outer edge, as seen by the thick band of red FP pixels around part of the perimeter. Throughout the dataset, there are multiple examples of this type of over-labelling by the ResNets, and it usually occurs for smaller myelin-axon pairs. For larger myelin-axon pairs, the ResNets tend to under-label the outer edge more strongly than the UNets, as seen in Figure 4.5. For both the ResNets in Figures 4.5c and 4.5d and also the FCN in Figure 4.5b, there is a nearly continuous

band of green FN pixels around the perimeter of the myelin-axon pair. Meanwhile, the UNets in Figures 4.5e and 4.5f are only intermittently under-labelling the outer edge.

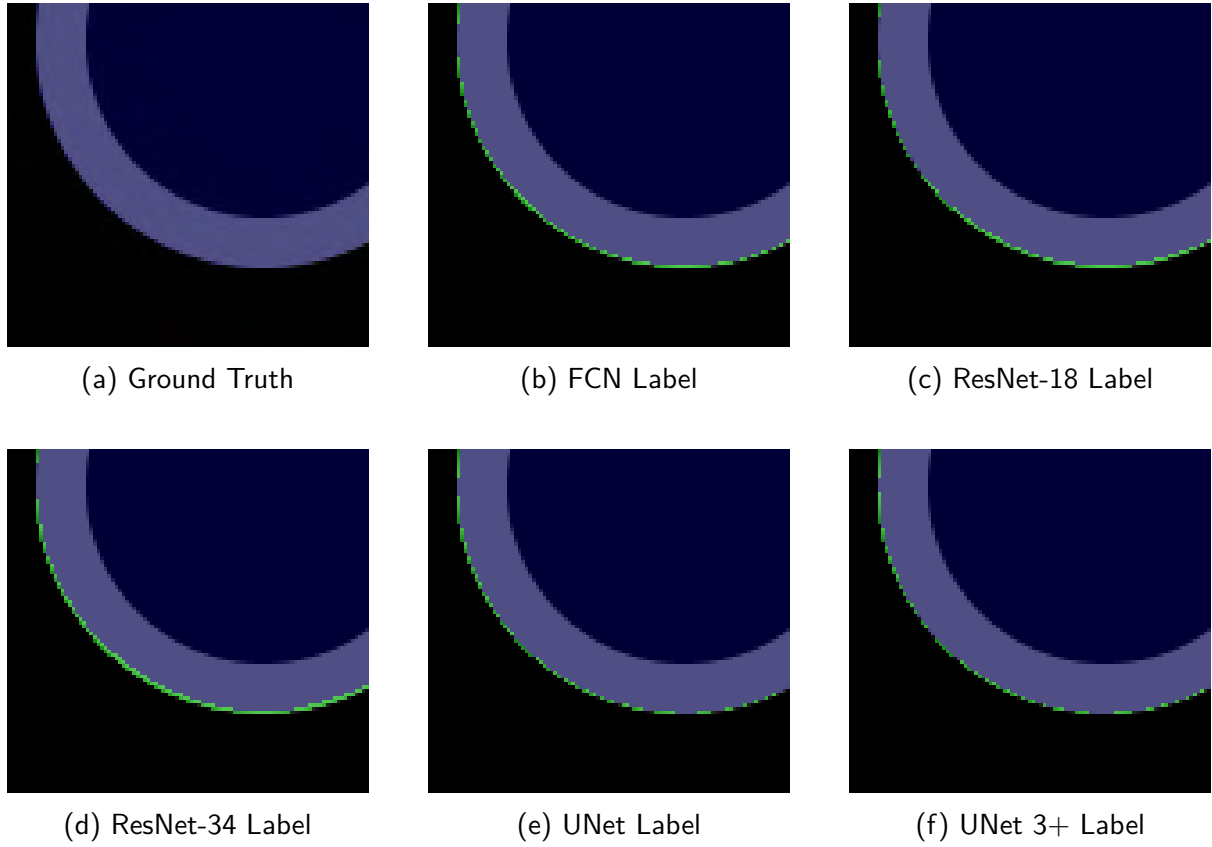


Figure 4.5: Examination of Segmentation Error: Magnified view of under-labelling along part of a large myelin-axon pair's outer edge. (Colour scheme: Blue = TP, Green = FN, Red = FP).

4.1.1.4 Closer Examination of Segmentation Errors: Random Labelling

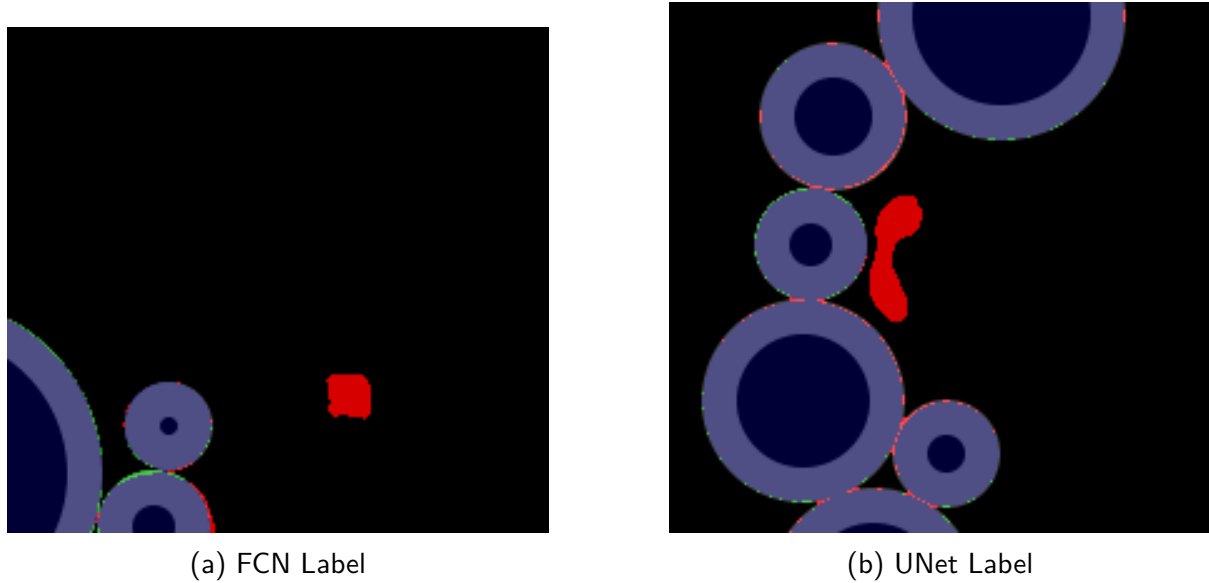


Figure 4.6: Examination of Segmentation Error: Magnified view of randomly segmented patches of the background, which is only observed for the FCN and UNet. (a) and (b) show different regions in the dataset. (Colour scheme: Blue = TP, Green = FN, Red = FP).

Figure 4.6 shows a type of segmentation error that only occurs for the FCN and the UNet. In this error, a random patch of background will be labelled by the network. This is the only example of the error for the FCN, however, this error occurs multiple times in the dataset for UNet. It is not obvious what causes this for the FCN, but this is very likely a case of overfitting for the UNet. As mentioned in Section 3.4.4, the UNet has by far the most parameters out of the five networks, meaning that it will be the most prone to overfitting [91]. When a model becomes overfit, it has learned the training data too well and memorised some aspects of it [96], producing a segmentation is is not remotely present in the testing dataset.

4.1.2 Rat White Matter

This Section of the validation presents and discusses the results from the DCNN models' segmentation of the rat WM. As mentioned in Section 3.1.2, this dataset comes from Abdollahzadeh et al.'s paper on DeepACSON [24], their own DCNN model to segment their rat WM datasets. The aim of this Section is to further validate the five DCNN models from this thesis using a real WM dataset, which is more complex due to the irregular structure of myelin and axons. Unfortunately, unlike the ground truth labels for the idealised WM dataset, those for the rat WM are not perfect segmentations. This is because they were created using ACSON [36], an algorithmic tool for automatic segmentation of WM, so the "ground truth" is itself not fully accurate (this was described further in Section 3.2.2). Therefore, qualitative evaluations are minimised, as they are less meaningful, and the focus is shifted to a quantitative comparison with the DCNN model from Abdollahzadeh et al.'s paper [24].

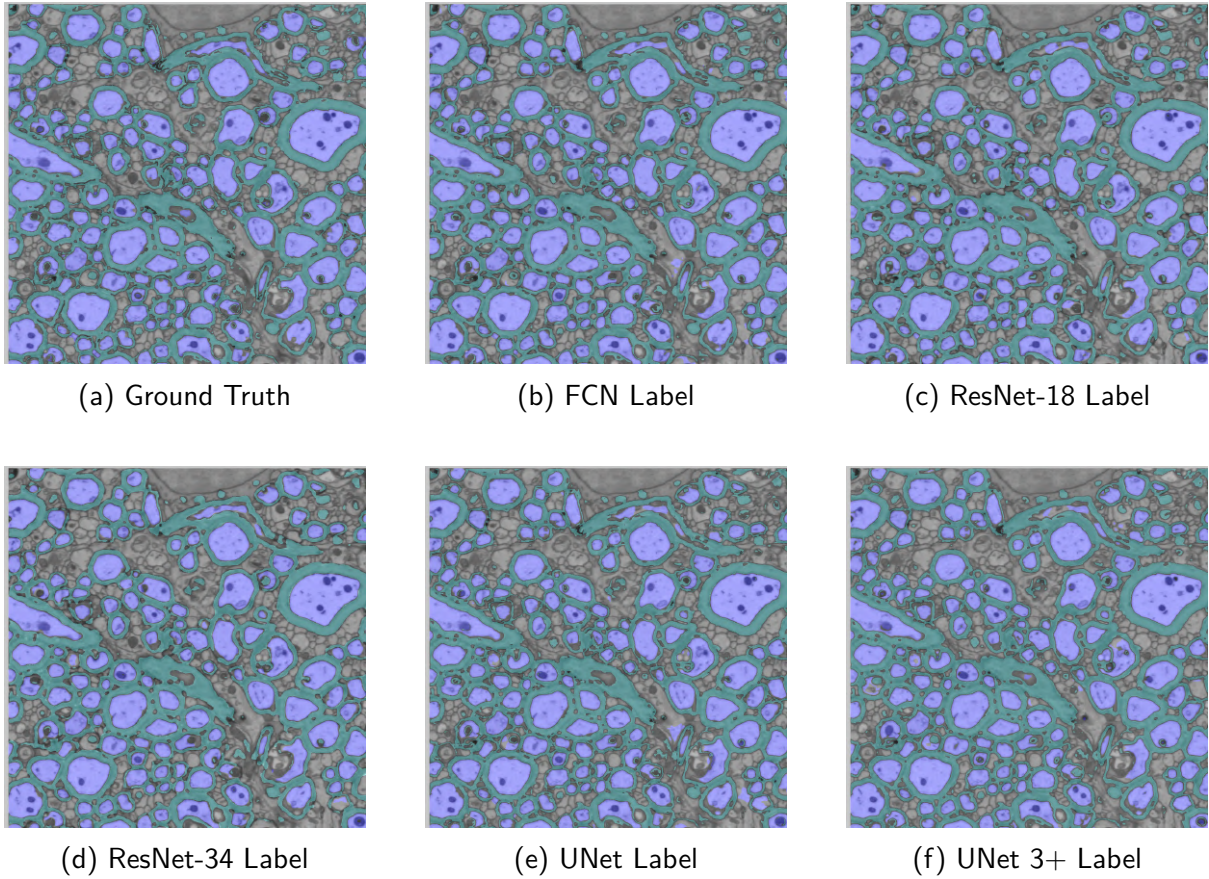


Figure 4.7: (a) A slice of the ground truth segmentation and the (b-f) segmentation results from the five models of the rat WM dataset. (Colour scheme: turquoise = myelin, purple = axon).

Overall, the automatic segmentations of the rat WM in Figures 4.7b to 4.7f are very successful for all of the DCNNs, as there are barely any discernible large-scale segmentation errors. At a slightly smaller scale, there are various discrepancies in the segmentations, such as

a small axon not present for the “ground truth” being segmented by some of the networks. However, as mentioned in Section 3.2.2, the “ground truth” is itself created using an automatic segmentation tool, so there is no guarantee that it is fully accurate. Figure 4.8 zooms in on the region of the small axon example to examine this discrepancy further.

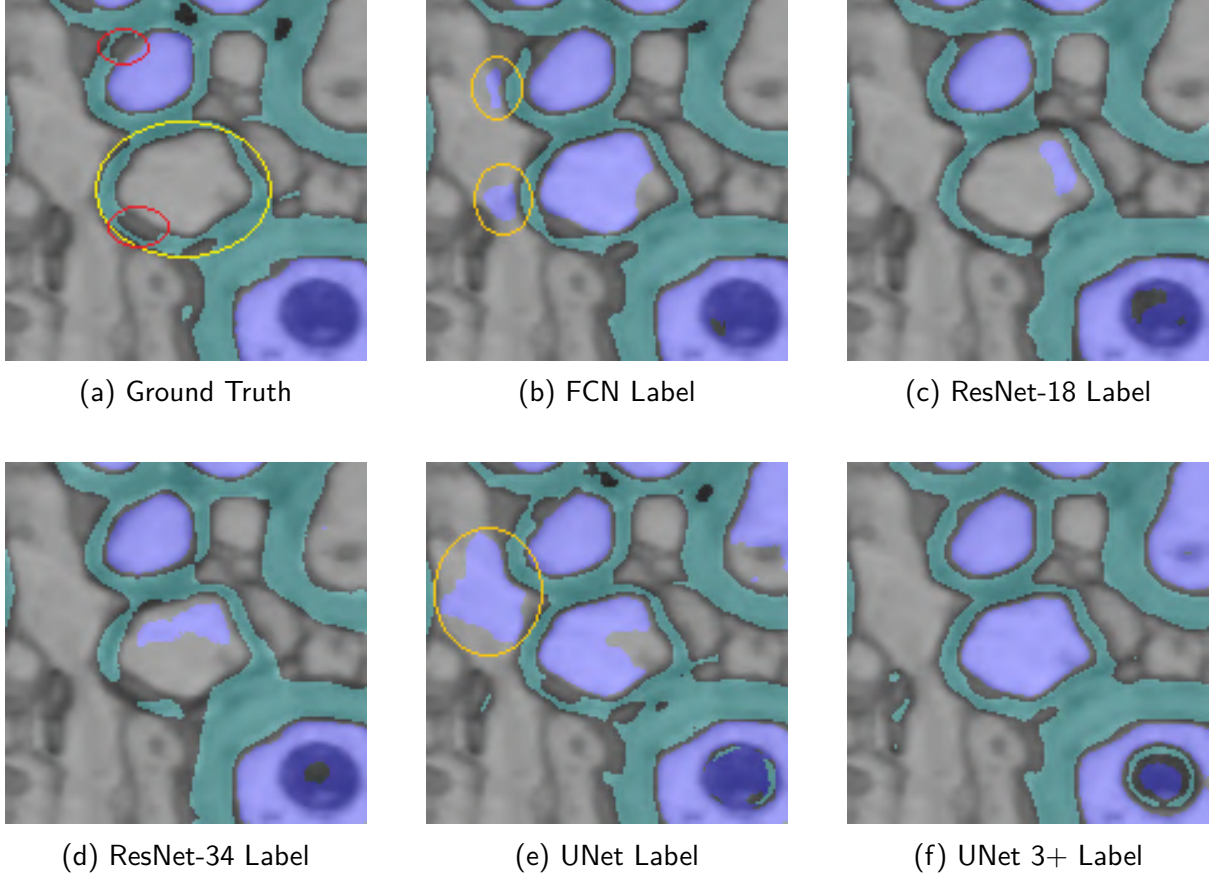


Figure 4.8: Examination of Segmentation Error: Magnified view of axon, marked in yellow, which is missing from the (a) ground truth segmentation mask but is present in the segmentation mask of several DCNNs: (b), (e), (f). (Colour scheme: turquoise = myelin, purple = axon).

As seen in the middle of Figure 4.8a, the “ground truth” created from the automatic technique ACSOON [36] did not label the area enclosed by myelin as an axon (marked in yellow). It is not clear as to why this is the case, as any imperfections of that axon’s myelin are present in the myelin of the labelled axons elsewhere in the “ground truth” image. For example, the missing segment of myelin to the left of the unlabelled axon (marked in red) cannot be the reason, as the myelin of the axon above, which is labelled, is also missing a large piece (marked in red). Regardless, this is an obvious segmentation error in the “ground truth” label, as ACSOON has labelled this same axonal structure as an axon in the subsequent slices. This means that the FCN (Figure 4.8b), UNet (Figure 4.8e) and UNet 3+ (Figure 4.8f) correctly identified this as an axon, whereas the two ResNets in Figures 4.8c and 4.8d failed to identify both the full axon and full myelin. Regardless, there is no real significance of making qualitative

comparisons, as all six segmentation masks in Figure 4.8, including the “ground truth”, are created automatically and are, therefore, error-prone.

However, an interesting thing to note is that both the FCN and the UNet, which were observed to label a random patch of the background in the idealised WM dataset (Section 4.1.1.4), have done the same in the rat WM dataset. This is marked in orange in Figures 4.8b and 4.8e, with the UNet being significantly more severe, just like for the idealised WM datasets. As discussed in Section 4.1.1.4, this phenomenon is likely a result of slight overfitting, as the UNet is the network with by far the most parameters.

Table 4.2: Evaluation Metrics for the Segmentation of Rat White Matter Myelin

Models	Precision	Recall	F1 score
FCN	0.935	0.932	0.934
ResNet-18	0.887	0.901	0.894
ResNet-34	0.866	0.881	0.874
U-Net	0.905	0.963	0.933
U-Net 3+	0.949	0.965	0.957

The evaluation metrics for each of the five models’ myelin segmentations are listed in Table 4.2. The F1 score for DeepACSON’s FCN was 0.886 [24], whereas it was 0.9337 for this thesis’s FCN. All the remaining networks in Table 4.2 also have a higher F1 score than the DeepACSON FCN. This is not that surprising, as Abdollahzadeh et al. did not only train their network on WM datasets from four different rats and two different regions of the brain but it was also tested on separate datasets. Meanwhile, for this thesis, a train-test split (described in Section 3.1.2) of a single dataset was used, meaning it is likely that the networks have been slightly overfitted to the features present in this WM dataset specifically. Nevertheless, the F1 scores are in the same neighbourhood as DeepACSON’s, so the networks can be considered successfully validated.

Table 4.3: Evaluation Metrics for the Segmentation of Rat White Matter Axons

Models	F1 Score
FCN	0.930
ResNet-18	0.907
ResNet-34	0.917
UNet	0.925
UNet 3+	0.934
DeepACSON	0.886

The evaluation metrics for each of the five models' axon segmentations are listed in Table 4.2. The F1 score for this thesis's FCN was 0.9297, while the DeepACSON FCN had an F1 score of 0.861 [24]. The remaining networks from this thesis also have significantly higher F1 scores than DeepACSON. The DeepACSON FCN is once again lower for the same reasons mentioned in the discussion for Table 4.2. As this thesis's F1 scores again are in the same neighbourhood as DeepACSON's, the networks can be considered successfully validated.

Overall, for both the myelin and the axon segmentation, the ResNets performed noticeably worse than the other three DCNNs. Unlike the idealised WM dataset in Section 4.1.1, where the evaluation metrics mostly improved with increasing network architecture complexity, the relatively simple FCN outperformed both the ResNets and the UNet for the rat WM dataset. A similar result was reported by Abdollahzadeh et al., where the UNet they tested performed worse than their FCN, which they also believe is due to slight overfitting [24].

Table 4.4: Evaluation Metrics for the Segmentation of Rat White Matter: Weighted Average

Models	Precision	Recall	F1 Score
FCN	0.931	0.932	0.932
ResNet-18	0.907	0.893	0.900
ResNet-34	0.893	0.896	0.894
U-Net	0.921	0.939	0.929
U-Net 3+	0.943	0.949	0.946

The average of the myelin and axon segmentation evaluation metrics, weighted by the number of pixels in each class, is listed in Table 4.4. These are calculated to be used as a comparison to the results of the sheep WM segmentation in Section 4.2. As described in Section 3.2.3, the manual labelling for the sheep WM dataset combined both the myelin and axon into a single class, so using the weighted average of the rat WM dataset is more appropriate for comparisons than the individual evaluation metrics.

4.2 Segmentation of Sheep White Matter

The purpose of the validation in the previous Section was to check that all networks work as expected. Although the five models had varying performances relative to each other, all of them had satisfactory evaluation metrics overall. As a result, all models will be considered equally and the focus will shift away from discussing the individual network architectures and performances. Instead, the focus will mainly be on the overall segmentation results of the DCNNs and potential preprocessing techniques that could be used to improve the segmentations.

This Section is divided into the various preprocessing techniques that have been attempted. As each subsequent preprocessing technique was chosen to improve upon the results of the previous techniques, the discussion for a preprocessing technique does not take the next technique's results into account.

For each preprocessing technique, the Fornix02 dataset was segmented by a DCNN trained using the Fornix01 dataset and the Fornix01 dataset was segmented by a DCNN trained using the Fornix02 dataset. This way it can be properly tested if the network is useful for automatic segmentation, as future applications of the DCNNs would not be on the same individual sheep.

For most preprocessing techniques, two sets of DCNNs are trained: one with the Fornix02 dataset and one with the Fornix01 dataset. In the Section for each preprocessing technique, the results for Fornix01 are presented and discussed first, then the results for Fornix02. Discussions that have taken place when analysing the results of the Fornix01 segmentation will not be repeated in full for the Fornix02 dataset. In those cases, the Fornix02 results are presented to check if the segmentations have a similar accuracy and also for the sake of completeness.

To ensure that the results are statistically significant and not due to an anomaly, all DCNNs are retrained at least once.

4.2.1 Denoised Dataset

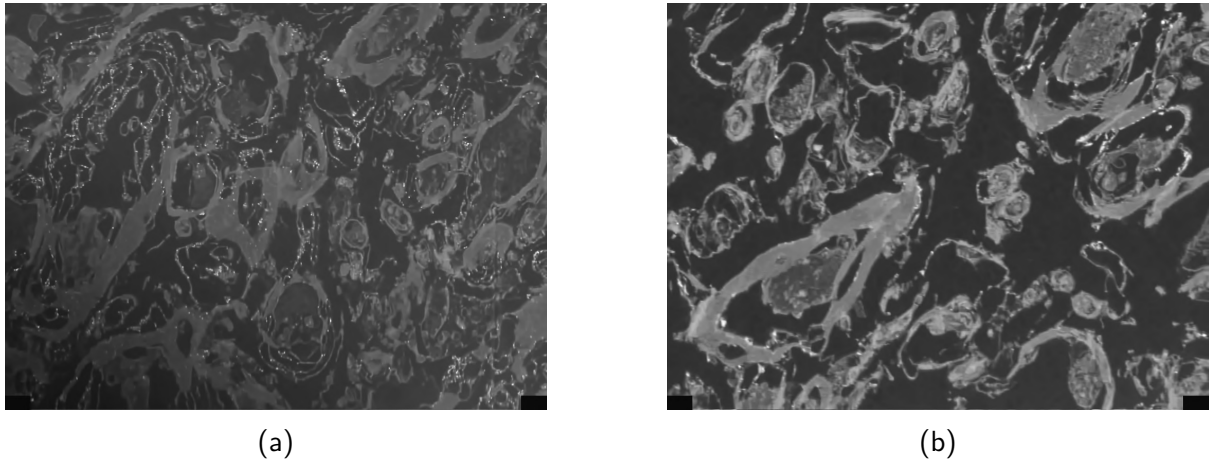


Figure 4.9: A slice of the denoised sheep WM samples: (a) Fornix01 and (b) Fornix02. Both samples, taken from two different sheep, are from the fornix in the brain.

The Fornix01 and Fornix02 sheep WM datasets used to train the networks in this section were denoised using the procedure described in Section 3.3.1, the results of which are shown again in Figure 4.9.

4.2.1.1 Segmentation of the Fornix01 Dataset

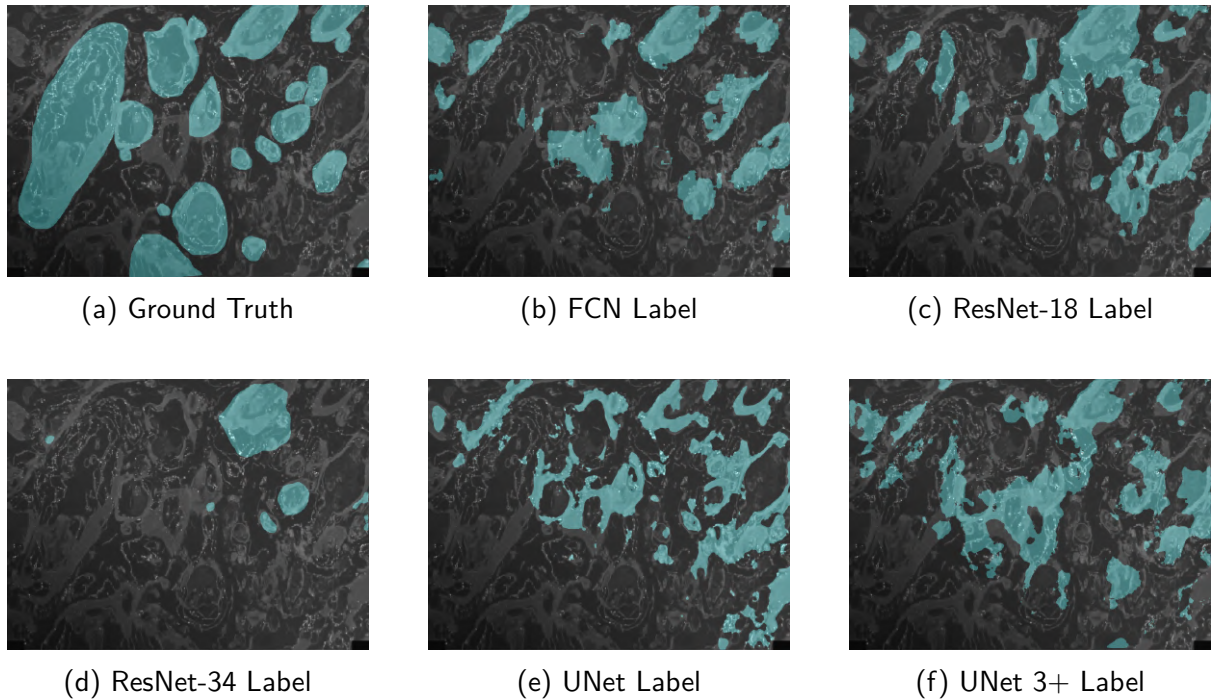


Figure 4.10: A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the denoised sheep WM (Fornix01).

Figure 4.10 depicts a slice of the denoised Fornix01 sheep WM dataset overlaid by, in order, the labels from the ground truth, FCN, ResNet-18, ResNet-34, UNet and UNet 3+. It is immediately obvious that the results are extremely poor, as none of the networks produced a segmentation that resembles the ground truth.

Table 4.5: Evaluation Metrics for the Segmentation of Denoised Sheep WM (Fornix01).

Models	Precision	Recall	F1 score
FCN	0.583	0.301	0.398
ResNet-18	0.546	0.379	0.447
ResNet-34	0.688	0.158	0.257
U-Net	0.535	0.258	0.348
U-Net 3+	0.541	0.359	0.432

The evaluation metrics in Table 4.5 confirm the results from Figure 4.10. The average recall for the denoised Fornix01 segmentation is extremely low at just 0.291. As per the definition of recall from Section 3.6, this means that less than 30% of the ground truth labels were also labelled by the five networks. Additionally, the F1 score is also low, with an average of 0.377, which is a poor result.

The cause for these poor results is the discrepancy in the image's intensities of the two denoised datasets in Figure 4.9. Not only is the intensity of the myelin in the Fornix02 dataset brighter, but the background is also darker than Fornix01. DCNN models that are trained on the Fornix02 datasets learn that the structures to be segmented are at those higher pixel intensities. When those models are used to segment the Fornix01 dataset, they do not recognise as many of the myelin-axon pairs as they have a lower intensity. This is especially evident in the bottom left corner of Fornix01 in Figure 4.9a. This region is particularly dark, and as a result, none of the networks in Figure 4.10 segmented anything in that region.

An intensity discrepancy between the training and testing datasets is known to cause issues for DCNNs. Otalora et al. [97] studied the effect that intensity variations in images of tissue can have on DCNN performance. In their case, inevitable variability in scanner configuration and slide preparation between operators caused significant intensity variations [97], which will result in an overfitted DCNN if all training data comes from the same operator. Jacobsen et al. [98] tested solutions to this problem and found that normalising the intensities in the datasets improved segmentation results significantly.

To improve the segmentation results of the Fornix01 dataset, a basic intensity normalisation technique will be attempted: binarisation. However, before normalising the intensities, the level of background illumination needs to be homogenised in the whole dataset. This is necessary because, as seen in the bottom left of Figure 4.9a, the sheep WM datasets do

not have consistent illumination within themselves, let alone consistent illumination across different datasets. Normalising the intensities before this step will not fully solve the lack of segmentation in the bottom left corner, as seen in Figure 4.10. When the Fornix01 and Fornix02 datasets were imaged by Bernardini et al. [7], they also noted the inconsistent background illumination and devised a technique to fix this, which is described in Section 3.3.2. Fornix01 and Fornix02 with homogenised background illumination are shown again in Figure 4.11.

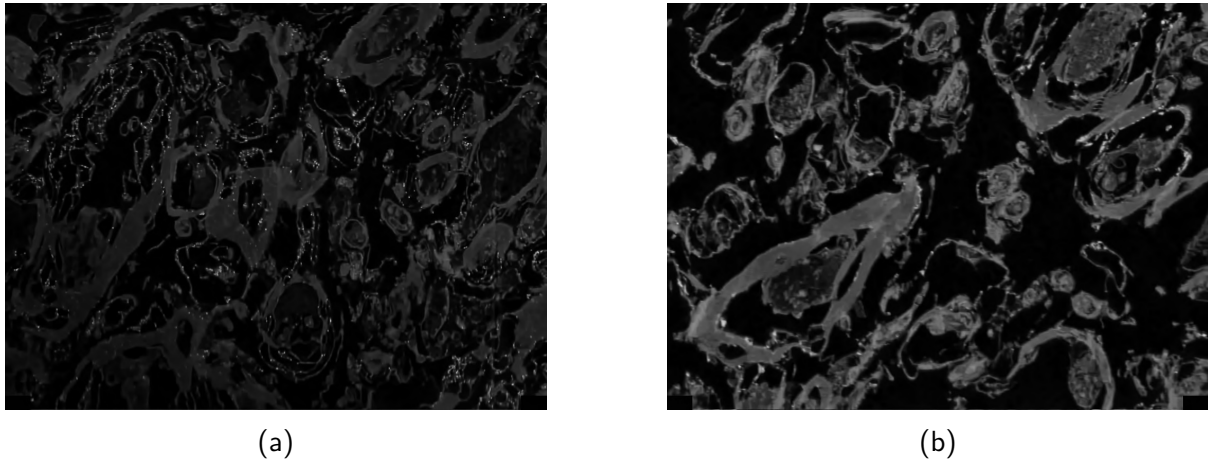


Figure 4.11: A slice of the sheep WM samples with homogenised background intensities: (a) Fornix01 and (b) Fornix02. Both samples, taken from two different sheep, are from the fornix in the brain.

Binarisation normalises the image by setting pixels above an intensity threshold to the maximum intensity and all below to the minimum intensity. An adaptive thresholding technique is used to automatically set an appropriate separate intensity threshold for the differently bright Fornix01 and Fornix02 datasets. As binarised images only have two different pixel intensities, the intensity discrepancy between the datasets will be solved entirely. The adaptive thresholding technique and procedure used to binarise the datasets are described in Section 3.3.3, and the results of the DCNNs trained on binarised datasets are presented and discussed in Section 4.2.2.

The binarised results will be presented after the upcoming results of the denoised Fornix02 dataset.

4.2.1.2 Segmentation of the Fornix02 Dataset

As mentioned previously, the segmentations of Fornix02 from the DCNNs trained on Fornix01 are mainly presented for completeness and only examined to check if there are any additional noteworthy results.

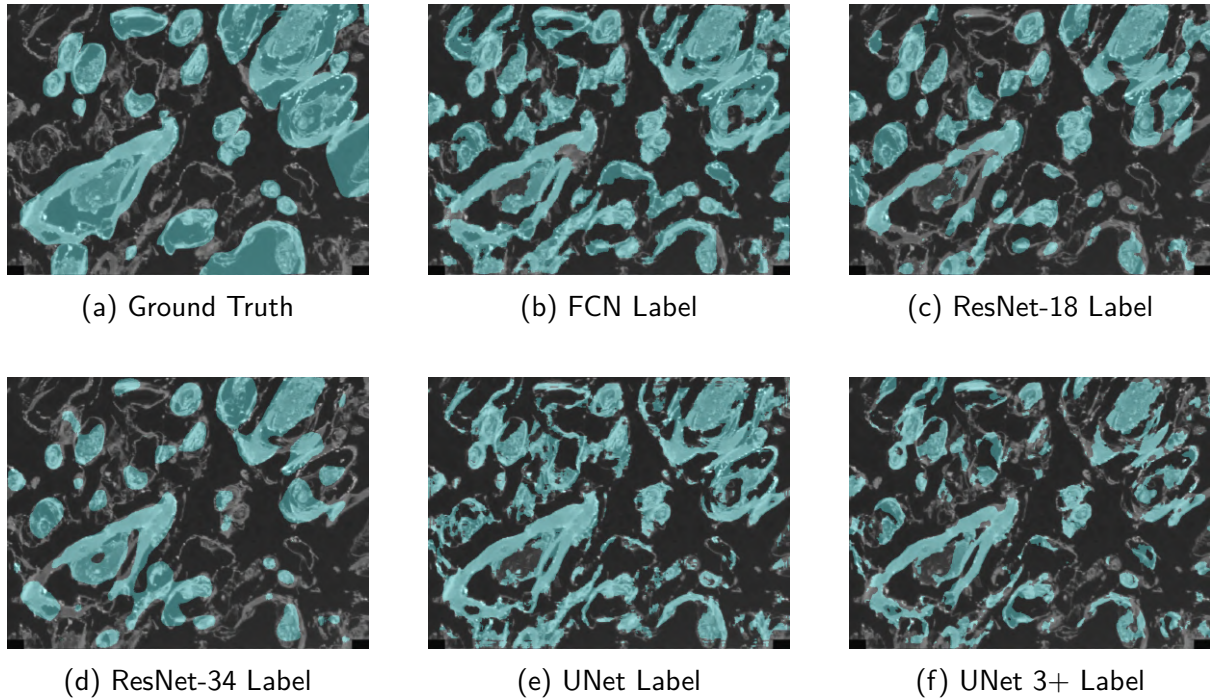


Figure 4.12: A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the denoised sheep white matter (Fornix02).

Figure 4.12 depicts a slice of the denoised Fornix02 sheep WM dataset overlaid by the label from the ground truth and the labels from the five DCNNs. At a glance, it seems that the segmentation accuracy has greatly improved over the segmentation of Fornix01 in Figure 4.10. However, at closer inspection, it becomes obvious that the DCNNs have simply labelled high-intensity pixels, instead of properly segmenting the structure of myelin-axon pairs that contain both high and low-intensity pixels.

This is again a result of a discrepancy in the image's intensities, as described in Section 4.2.1.1, but this time the opposite example. The DCNNs were trained on an overall lower-intensity dataset (Fornix01), so when the networks were presented with higher intensities in Fornix02, they ended up segmenting too many pixels in the dataset. This is especially evident for the FCN's and UNet's segmentations in Figures 4.12b and 4.12e, respectively, where practically every high-intensity pixel in the dataset has been labelled.

Table 4.6: Evaluation Metrics for the Segmentation of Denoised Sheep WM (Fornix02).

Models	Precision	Recall	F1 score
FCN	0.644	0.612	0.628
ResNet-18	0.667	0.560	0.609
ResNet-34	0.710	0.545	0.617
U-Net	0.665	0.628	0.646
U-Net 3+	0.719	0.490	0.583

Although the evaluation metrics in Table 4.6 do not look terrible, with an average F1 score of 0.617, it is inflated due to the tendency of the models to label all high-intensity pixels in the image.

In summary, the results from the Fornix02 segmentation reinforce that the intensity normalisation discussed in Section 4.2.1.1 is necessary to improve segmentation results.

4.2.2 Binarised Dataset

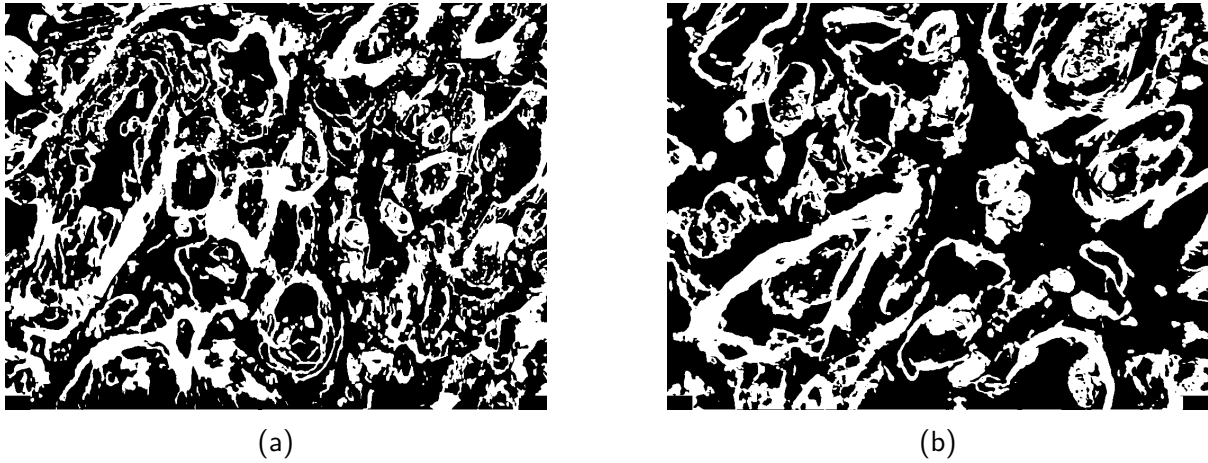


Figure 4.13: A slice of the binarised sheep WM samples: (a) Fornix01 and (b) Fornix02. Both samples, taken from two different sheep, are from the fornix in the brain.

The Fornix01 and Fornix02 sheep WM datasets used to train the networks in this section were binarised using the procedure described in Section 3.3.3, the results of which are shown again in Figure 4.13.

4.2.2.1 Segmentation of the Fornix01 Dataset

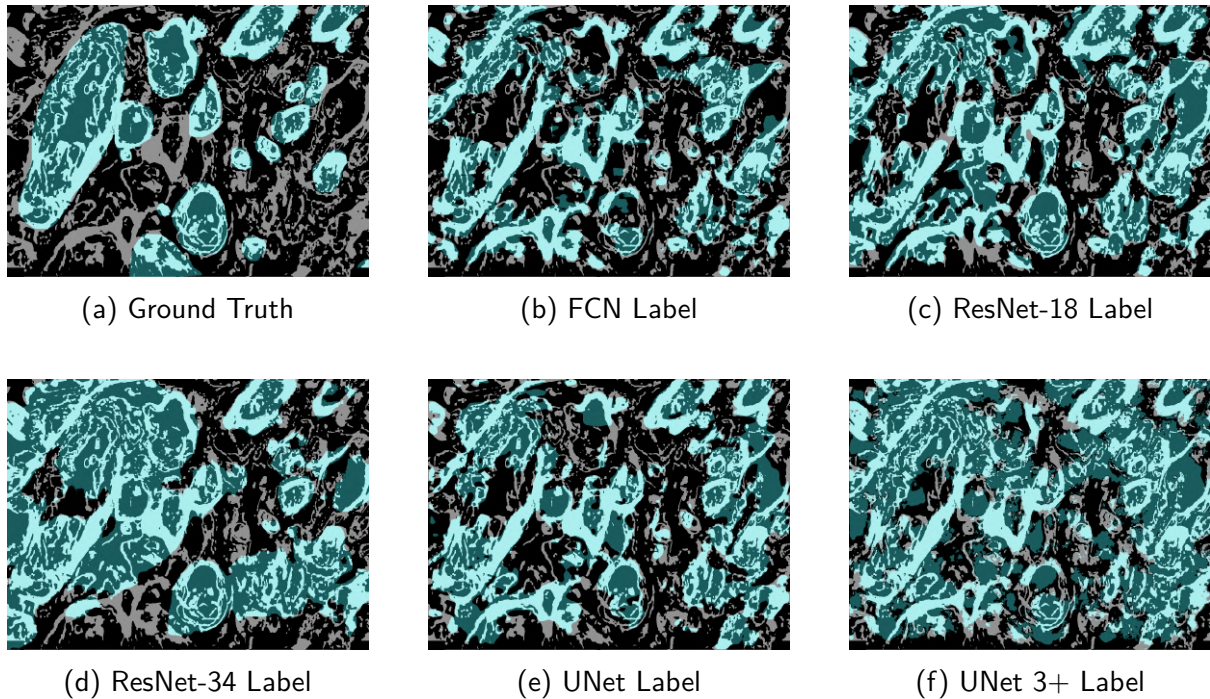


Figure 4.14: A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the binarised sheep WM (Fornix01).

Figure 4.14 shows a slice of the binarised Fornix01 sheep WM dataset overlaid by the ground truth label and the segmentation results from the five DCNNs. None of the five networks have produced any accurate segmentation results, as they have all labelled too much of the image. Nevertheless, it is evident that the issue of intensity discrepancies was resolved. Unlike the denoised Fornix01 dataset in Figure 4.10, the binarised Fornix01 now has segmentations in the bottom left corner of the image, as it is no longer darker than the rest of the datasets.

Table 4.7: Evaluation Metrics for the Segmentation of Binarised Sheep WM (Fornix01).

Models	Precision	Recall	F1 score
FCN	0.556	0.602	0.578
ResNet-18	0.519	0.694	0.584
ResNet-34	0.551	0.700	0.616
U-Net	0.550	0.628	0.587
U-Net 3+	0.463	0.765	0.577

The evaluation metrics in Table 4.7 reflect what was observed in Figure 4.14, as the visibly poor segmentation results have an average precision of 0.528. As per the definition of precision from Section 3.6, this means that almost 50% of the segmented area is wrongly labelled (i.e., false positive). The F1 score is not terrible with an average of 0.588, which is significantly higher than the denoised Fornix01 dataset's F1 score of 0.377. However, this F1 score is inflated, as it is a result of the DCNNs labelling a significant area of the image rather than accurately detecting myelin-axon pairs.

Although the intensity discrepancy described in Section 4.2.1.1 has been resolved, the accuracy of the segmentation results has not improved significantly. This is due to another issue in the Fornix01 dataset, which has become more apparent after the binarisation, namely the high amounts of clutter. As seen in Figure 4.13a, the binarised Fornix01 dataset is covered with a high density of small segments (i.e., clutter). This is in contrast to the binarised Fornix02 dataset in Figure 4.13b, which has plenty of empty regions between the myelin-axon pairs. It is highly likely that a DCNN trained on the low-clutter Fornix02 dataset is not able to distinguish between the clutter and the myelin-axon pairs in Fornix01, causing the segmentations to span across large areas of the image.

A possible hypothesis as to why the DCNNs are including the clutter in the segmentation is that the binarisation has removed all intensity gradients in the image. In a binary image, any clutter will have the exact same intensity as the myelin. However, the intensity gradients present in the image before binarisation may allow the DCNNs to more effectively distinguish between the clutter and the myelin-axon pairs.

To test this hypothesis, a more sophisticated intensity normalisation technique will be tested,

namely intensity remapping. In intensity remapping, the range of intensities in the image is stretched to span the maximum possible range. This way, Fornix01 and Fornix02 will both have the background pixels at the lowest possible intensity and the myelin pixels at the highest possible intensity. This will also solve the issues related to intensity discrepancies discussed in Section 4.2.1.1, however, unlike binarisation, it will still preserve the intensity gradients. The procedure used to remap the intensities is described in Section 3.3.4, and the results of the DCNNs trained on intensity-remapped datasets are presented and discussed in Section 4.2.3. The intensity-remapped results will be presented after the upcoming results of the binarised Fornix02 dataset.

4.2.2.2 Segmentation of the Fornix02 Dataset

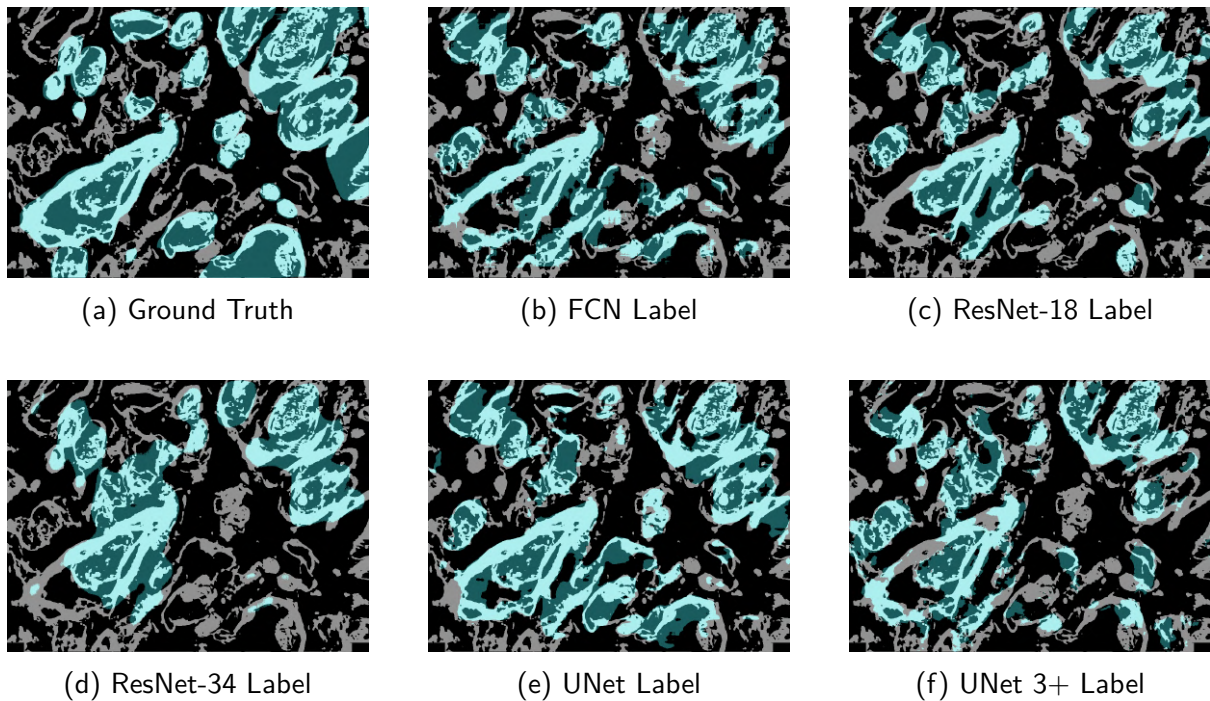


Figure 4.15: A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the binarised sheep WM (Fornix02).

Figure 4.15 shows a slice of the binarised Fornix02 sheep WM dataset overlaid by the ground truth label and the segmentation results from the five DCNNs. Unlike the segmentations for the binarised Fornix01 in Figure 4.14, where large areas of background pixels were included in the segmentation, the results for Fornix02 are more promising. The networks are only segmenting structures that could feasibly be myelin-axon pairs, and segmentation patches that span over larger areas of the image are limited.

In contrast to the denoised Fornix02 segmentation from Section 4.2.1.2, where the networks mostly just labelled the high-intensity pixels, here the networks are properly labelling both the high and low-intensity pixels present in myelin-axon pairs. Therefore, the intensity normalisa-

tion via binarisation has greatly improved the results for the Fornix02 dataset.

Table 4.8: Evaluation Metrics for the Segmentation of Binarised Sheep WM (Fornix02).

Models	Precision	Recall	F1 score
FCN	0.682	0.532	0.598
ResNet-18	0.692	0.472	0.561
ResNet-34	0.707	0.508	0.591
U-Net	0.697	0.570	0.627
U-Net 3+	0.636	0.452	0.529

As the evaluation metrics for the denoised Fornix02 in Table 4.8 have an average F1 score of 0.617, the evaluation metrics for the binarised Fornix02 in Table 4.8, with an average F1 score of 0.581, are not an improvement. However, as discussed in Section 4.2.1.2, those evaluation metrics are inflated and a comparison between the qualitative results shows that the binarised Fornix02 has been segmented more successfully.

Similarly, as for Fornix01 in Section 4.2.2.1, the binarisation has resolved the intensity discrepancy for the Fornix02 dataset. Unlike the Fornix01 dataset, however, the segmentation for Fornix02 in Figure 4.15 is not labelling large quantities of background pixels. This observation is also seen in the evaluation metrics in Table 4.8, as the average precision for Fornix02 is 0.683, as opposed to an average of 0.564 for Fornix01. As explained in Section 3.6.1, the higher precision indicates that the segmentation of Fornix02 has included fewer background pixels in the segmentation (i.e., false positives).

This indicates that the reverse of the situation from the previous Section, i.e., using a DCNN trained on a high-clutter dataset to segment a low-clutter dataset, does not affect the segmentation results to the same degree. However, it is still likely that the clutter in the Fornix01 dataset is having a negative effect on the quality of the DCNN training. Therefore, as discussed previously in Section 4.2.2.1, intensity remapping will be tested as an alternative to binarisation. This might improve the segmentation results, as the intensity gradients may help the DCNN distinguish between the clutter and the myelin-axon pairs during training.

4.2.3 Intensity-remapped Dataset

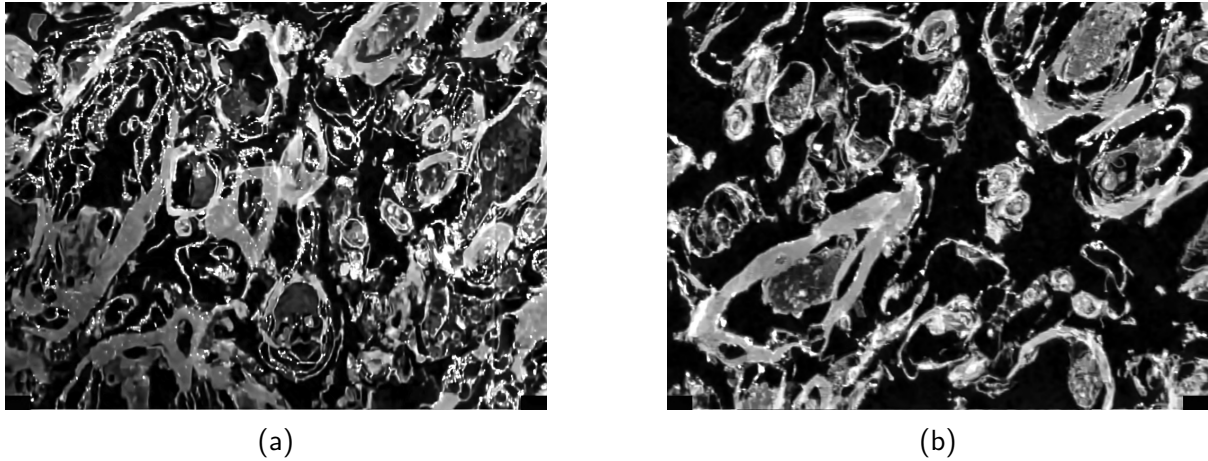


Figure 4.16: A slice of the intensity-remapped sheep WM samples: (a) Fornix01 and (b) Fornix02. Both samples, taken from two different sheep, are from the fornix in the brain.

The Fornix01 and Fornix02 sheep WM datasets used to train the networks in this section had their intensities remapped using the procedure described in Section 3.3.4, the results of which are shown again in Figure 4.16.

As seen in Figure 4.16a, the clutter present in the Fornix01 dataset now has intensity gradients, rather than being at a single intensity like in the binarised dataset.

4.2.3.1 Segmentation of the Fornix01 Dataset

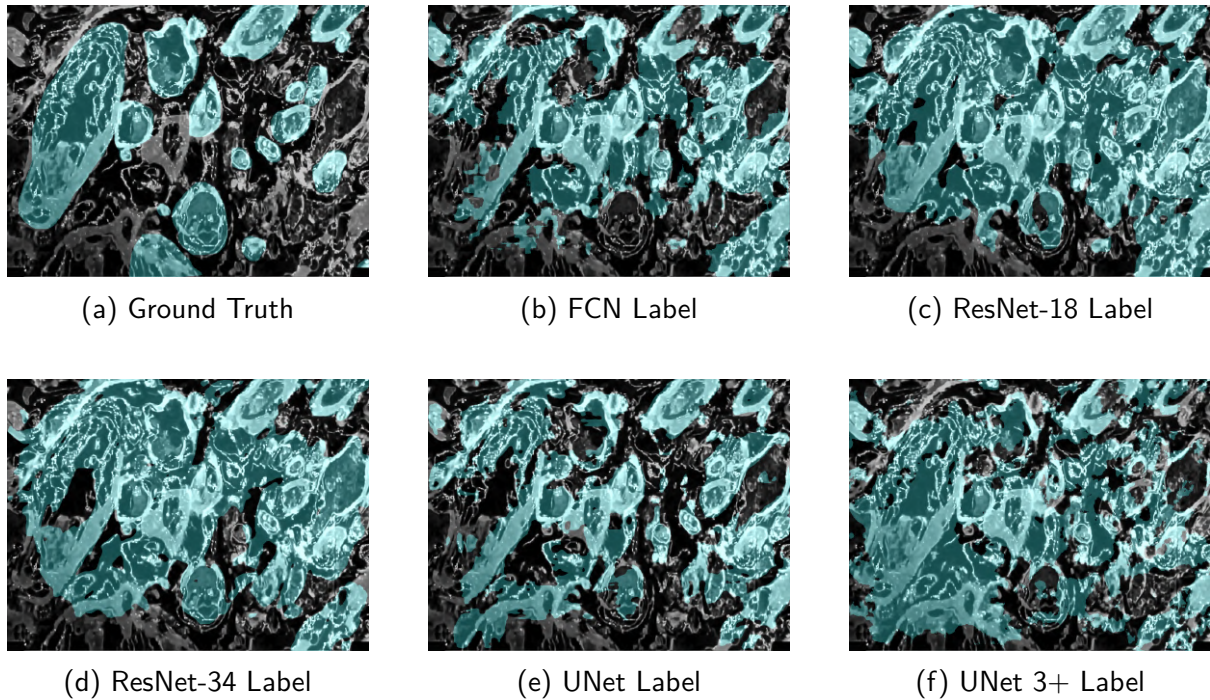


Figure 4.17: A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the intensity-remapped WM (Fornix01).

Figure 4.17 depicts a slice of the intensity-remapped Fornix01 sheep WM dataset overlaid by the ground truth label and the segmentation results from the five DCNNs. All the networks are labelling large patches of the image's background, just like for the binarised Fornix01 datasets in Figure 4.14. This indicates that the lack of intensity gradients was not responsible for the clutter being segmented in the Fornix01 dataset, meaning that the hypothesis presented in Section 4.2.2.1 was incorrect.

Table 4.9: Evaluation Metrics for the Segmentation of the intensity-remapped Sheep WM (Fornix01).

Models	Precision	Recall	F1 score
FCN	0.499	0.692	0.580
ResNet-18	0.492	0.818	0.615
ResNet-34	0.503	0.859	0.635
U-Net	0.577	0.658	0.615
U-Net 3+	0.499	0.718	0.589

The evaluation metrics in Table 4.9 follow the same trends as those for the binarised Fornix01 dataset, further reinforcing that the inclusion of intensity gradients has not changed the segmentation results. The average precision is 0.514, which is similar to the binarised Fornix01 dataset's average of 0.528, meaning that the segmentation of both models includes almost 50% background pixels. The F1 score is also similar for both, with an average of 0.607 versus the average of 0.588 for the binarised Fornix01 dataset. This is once again inflated, as it is a result of the DCNNs labelling a significant area of the image, rather than accurately detecting myelin-axon pairs.

The segmentation errors caused by the high amount of clutter in the Fornix01 dataset were clearly not resolved by the inclusion of intensity gradients. Therefore, it seems almost certain that a high-clutter dataset cannot be segmented by a DCNN that was trained exclusively on a low-clutter dataset. There are two remaining alternative possibilities for this error. Either there is some aspect of the Fornix02 dataset that makes it unsuitable for training, or there is an aspect other than the clutter in the Fornix01 dataset which prevents more accurate segmentations. To be able to explore these possibilities, a final preprocessing technique was devised to reduce the amount of clutter in the Fornix01 dataset.

The goal of this preprocessing technique is to fully confirm whether a high amount of clutter is responsible for the poor segmentation accuracy when the DCNN has been trained on a low-clutter dataset. In the preprocessing technique, large and unlabelled features in the Fornix01 dataset which closely resemble WM (i.e., clutter) are removed. To emulate the structure of the Fornix02 dataset, the smaller features are not removed from the Fornix01 dataset. The procedure used to reduce the clutter is described in Section 3.3.5, and the results of the

DCNNs trained on intensity-remapped datasets are presented and discussed in Section 4.2.4.1. The decluttered results will be presented after the upcoming results of the intensity-remapped Fornix02 dataset.

4.2.3.2 Segmentation of the Fornix02 Dataset

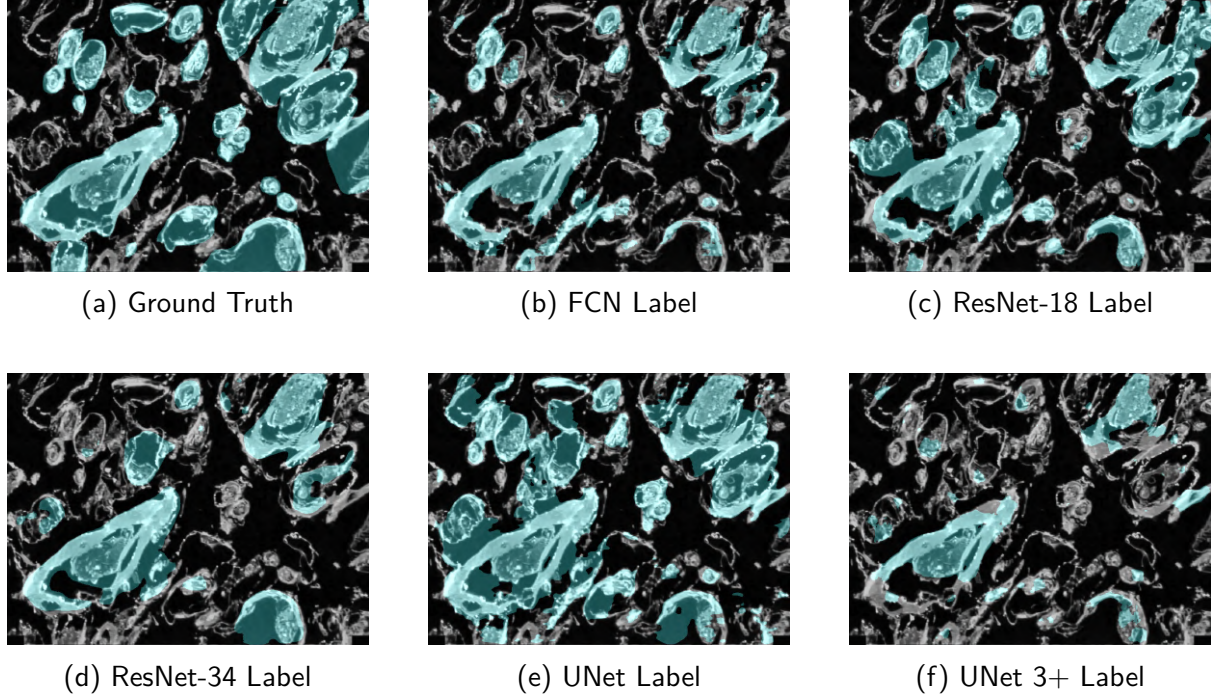


Figure 4.18: A slice of the ground truth segmentation (a) and the segmentation results from the five models (b-f) of the intensity-remapped sheep WM (Fornix02).

Figure 4.18 depicts a slice of the intensity-remapped Fornix02 sheep WM dataset overlaid by the ground truth label and the segmentation results from the five DCNNs. Visually, the results are very similar to the segmentations from the binarised Fornix02 datasets in Figure 4.15. Only structures that could feasibly be myelin-axon pairs are being segmented, and segmentation patches that incorrectly span a large area of the image are limited.

Table 4.10: Evaluation Metrics for the Segmentation of the intensity-remapped Sheep WM (Fornix02).

Models	Precision	Recall	F1 score
FCN	0.802	0.481	0.601
ResNet-18	0.725	0.623	0.670
ResNet-34	0.781	0.542	0.640
U-Net	0.681	0.701	0.691
U-Net 3+	0.814	0.325	0.465

The evaluation metrics for the intensity-remapped Fornix02 dataset in Table 4.10 have an average F1 score of 0.613, which is a slight improvement over the binarised Fornix02's average F1 score of 0.581.

The qualitative and quantitative results for both the binarised and intensity-remapped Fornix02 datasets are similar. Therefore, it is safe to say that both preprocessing techniques are valid choices to solve the problem of intensity discrepancies among datasets.

The slightly higher average F1 score for the intensity-remapped Fornix02 dataset indicates that the previously proposed hypothesis, that binarisation will help the network learn structural features, is invalid. According to the results from this Section, the various DCNNs have an equal, if not slightly worse, performance on the binarised dataset compared to the intensity-remapped datasets.

As mentioned in the previous Section, the final preprocessing technique that will be attempted is to remove all major unlabelled features in the dataset. As this preprocessing step's aim is to investigate whether a high amount of clutter is the cause for the Fornix01 dataset's poor segmentation results, it will not be applied to Fornix02.

4.2.4 Decluttered Dataset

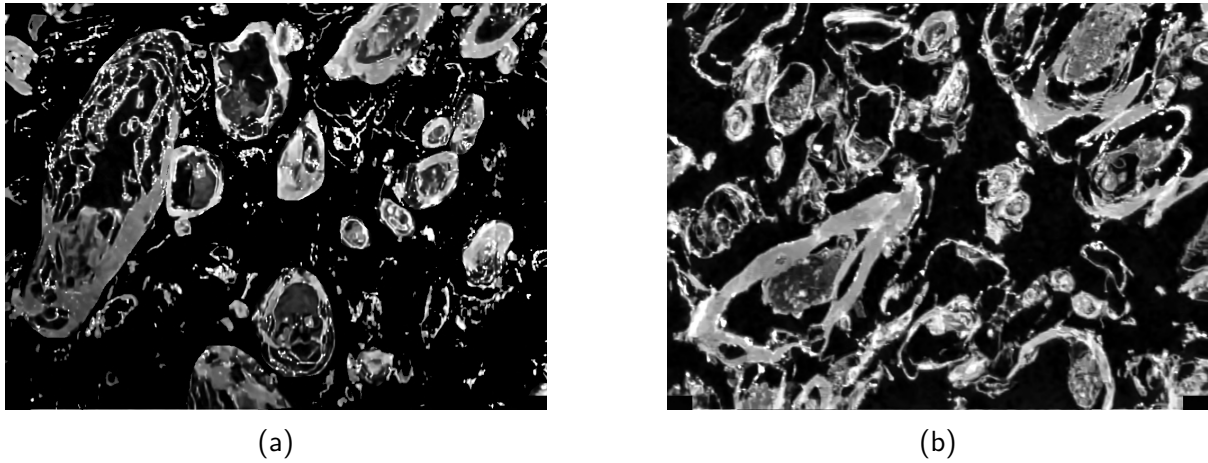


Figure 4.19: A slice of the (a) feature decluttered Fornix01 dataset, used to test the DCNNs, and (b) intensity-remapped Fornix02 dataset, used to train the DCNNs. Both samples, taken from two different sheep, are from the fornix in the brain.

The Fornix01 sheep WM dataset depicted in [4.19a](#) has had its features decluttered, following the procedure outline in [Section 3.3.5](#). After the preprocessing, the amount of clutter in the previously high-clutter dataset is now almost the same as the amount of clutter in the low-clutter Fornix02 dataset, seen in [Figure 4.19b](#).

For the segmentation of the decluttered Fornix01 dataset in [Section 4.2.4.1](#), the DCNNs were trained on the intensity-remapped Fornix02 dataset.

4.2.4.1 Segmentation of the Fornix01 Dataset

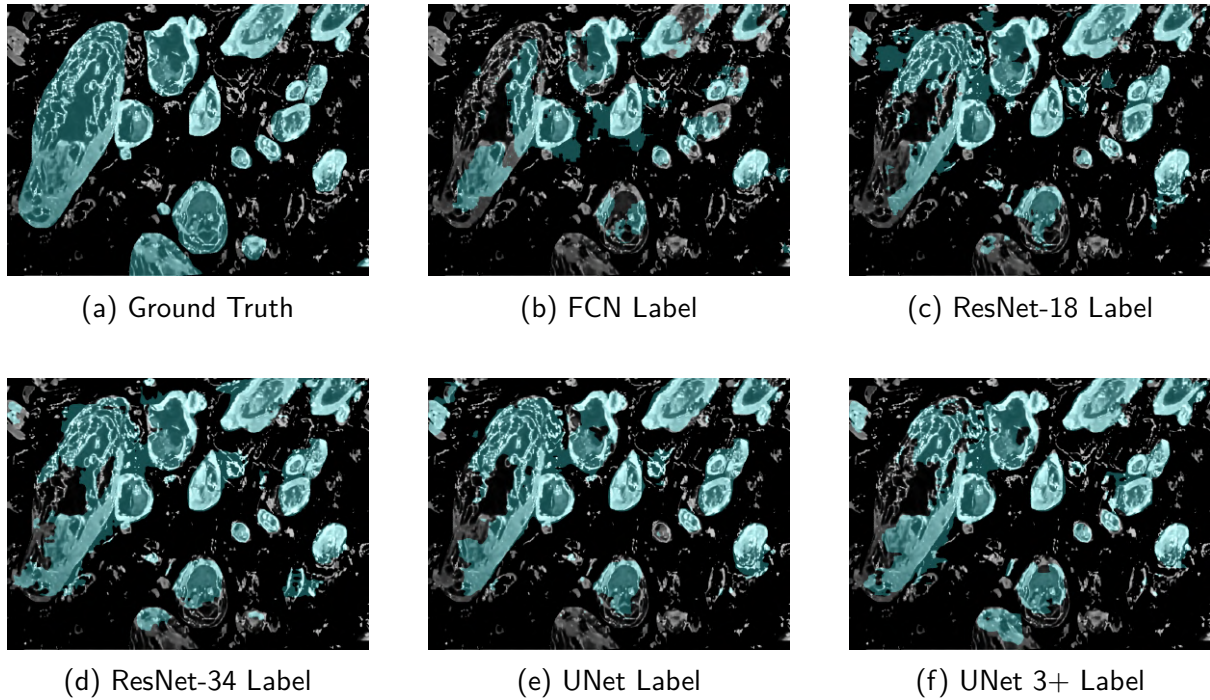


Figure 4.20: A slice of (a) the ground truth segmentation and (b-f) the segmentation results from the five models of the decluttered sheep WM (Fornix01).

Figure 4.20 depicts a slice of the decluttered Fornix01 sheep WM dataset overlaid by the ground truth label and the segmentation results from the five DCNNs. Visually, these results are a significant improvement over the results from both the binarised Fornix01 dataset and the intensity-remapped Fornix01 dataset. Many myelin-axon pairs in the images are individually and fully segmented, without any background being included in the segmentation patch. This was never the case for the binarised and intensity-remapped Fornix01 datasets, as their segmentation patches always included a significant number of background pixels.

The segmentation masks in Figure 4.20 are the first time that the results for the Fornix01 dataset resemble the results of the Fornix02 dataset. This clearly shows that the high amount of clutter in the Fornix01 dataset made it unsuitable to be segmented by a DCNN that was trained on a low-clutter dataset like Fornix02.

Table 4.11: Evaluation Metrics for the Segmentation of Decluttered Sheep WM (Fornix01).

Models	Precision	Recall	F1 Score
FCN	0.805	0.596	0.685
ResNet-18	0.784	0.667	0.721
ResNet-34	0.772	0.781	0.777
U-Net	0.795	0.708	0.749
U-Net 3+	0.797	0.749	0.772

The evaluation metrics in Table 4.11 are greatly improved compared to the results from the binarised and intensity-remapped Fornix01 datasets. Both those datasets had an average precision of approximately 0.52 and an average F1 score of approximately 0.59. In contrast, the average precision for the decluttered Fornix01 dataset is 0.791 and the average F1 score is 0.741.

This further reinforces that the high amount of clutter in the Fornix01 dataset was making it impossible for a DCNN trained on a low-clutter dataset to be used for accurate segmentation.

Chapter 5

Conclusions and Perspectives

5.1 Conclusion

To model Convection Enhanced Delivery of drug molecules in the brain requires greater insight into the morphology of myelinated axons within WM. Myelinated axon morphology is best studied using three-dimensional reconstructions of the WM, however, these require accurate WM segmentations. Although manual segmentation is an option to obtain high-accuracy results, this method is extremely time-consuming and cannot be implemented at a large scale. Therefore, developing accurate automatic segmentation techniques is advantageous.

For this thesis, five DCNNs with different levels of architectural complexity were created, with the goal of training the networks for automatic WM segmentation. Two sheep fornix WM datasets, which have been manually segmented, were available for training. Additionally, several preprocessing pipelines were devised iteratively to help the training process and improve the segmentation results.

To validate the constructed DCNNs ability to segment WM datasets, the five networks were tested on an idealised WM dataset and a rat WM dataset. All DCNNs performed excellently on both the idealised and real WM datasets, with hardly any major segmentation errors.

The initial sheep WM DCNNs, trained on just denoised datasets, were not successful. The large intensity discrepancies between the datasets, due to inevitable inconsistencies in the imaging process, made accuracy impossible. This phenomenon is well documented in the literature, and intensity normalisation is the generally preferred solution. Two variants of intensity normalisation, binarisation and intensity remapping, were attempted, both of which solved the intensity discrepancies. However, they revealed another flaw in the datasets, namely the different levels of clutter (i.e., unlabelled structures which resemble myelin-axon pairs). Using a DCNN trained on a high-clutter dataset to segment a low-clutter dataset worked reasonably well. However, the reverse situation does not work. This is because the network is overfitting on the low-clutter dataset and learns to simply segment everything that is similar to

myelin-axon pairs. A final preprocessing technique was devised to declutter datasets, in order to confirm that the poor segmentation results were due to significant clutter. This massively improved the segmentation results, indicating that the clutter was responsible.

Although decluttering datasets is not a feasible preprocessing technique for general use, as it requires the ground truth segmentation, the findings reinforce that DCNNs need the training and testing datasets to be of similar structure. The implications of this are that it may be more difficult than anticipated to train a DCNN for automatic sheep WM segmentation, as two samples from the same region of the brain were already dissimilar enough to inhibit training.

5.2 Future Research

There are a couple of ways in which the work from this thesis can be built upon. Firstly, two datasets high-clutter datasets can be obtained, it can be attempted to see what would happen if a DCNN is trained on a low- and a high-clutter dataset, and then used to segment a high-clutter dataset. It is possible that the combination of the low- and high-clutter datasets will prevent the DCNN from overfitting on either dataset type.

Additionally, simply having more training samples might make a big difference in the results too. Low sample count is a common reason for overfitting, and with only 54 unique samples in the training dataset, it is possible that the results in this report were already being overfitted heavily.

Finally, trying out some simpler DCNNs might prevent the networks from overfitting. The DCNN with the lowest parameter count in this thesis still has well over 10 million parameters. A network like the Half-UNet [99], which has impressive segmentation results despite having less than one million parameters, is tested, it could be found that there is less overfitting to the low-clutter sheep WM datasets.

Bibliography

- [1] Valery L. Feigin et al. "The global burden of neurological disorders: translating evidence into policy". English. In: *The Lancet Neurology* 19.3 (Mar. 2020). Pp. 255–265. ISSN: 1474-4422, 1474-4465. DOI: [10 . 1016 / S1474 - 4422 \(19 \) 30411 - 9](https://doi.org/10.1016/S1474-4422(19)30411-9). URL: [https : // www . thelancet . com / journals / laneur / article / PIIS1474 - 4422 \(19\) 30411 - 9 / fulltext](https://www.thelancet.com/journals/lanneur/article/PIIS1474-4422(19)30411-9/fulltext).
- [2] *Enhanced Delivery Ecosystem for Neurosurgery in 2020 | EDEN2020 Project | Fact Sheet | H2020*. en. URL: <https://cordis.europa.eu/project/id/688279>.
- [3] A. M. Mehta, A. M. Sonabend, and J. N. Bruce. "Convection-Enhanced Delivery". In: *Neurotherapeutics* 14.2 (Apr. 2017), pp. 358–371. ISSN: 1878-7479. DOI: [10 . 1007 / s13311 - 017 - 0520 - 4](https://doi.org/10.1007/s13311-017-0520-4). URL: <https://www.sciencedirect.com/science/article/pii/S187874792301471X>.
- [4] John D. Heiss et al. "Local distribution and toxicity of prolonged hippocampal infusion of muscimol". en. In: (Dec. 2005). DOI: [10 . 3171 / jns . 2005 . 103 . 6 . 1035](https://doi.org/10.3171/jns.2005.103.6.1035). URL: [https : // thejns . org / view / journals / j - neurosurg / 103 / 6 / article - p1035 . xml](https://thejns.org/view/journals/j-neurosurg/103/6/article-p1035.xml).
- [5] Tian Yuan et al. "A comprehensive review on modeling aspects of infusion-based drug delivery in the brain". In: *Acta Biomaterialia* (July 2024). ISSN: 1742-7061. DOI: [10 . 1016 / j . actbio . 2024 . 07 . 015](https://doi.org/10.1016/j.actbio.2024.07.015). URL: <https://www.sciencedirect.com/science/article/pii/S1742706124003878>.
- [6] Caleb A. Stine and Jennifer M. Munson. "Convection-Enhanced Delivery: Connection to and Impact of Interstitial Fluid Flow". In: *Frontiers in Oncology* 9 (Oct. 2019), p. 966. ISSN: 2234-943X. DOI: [10 . 3389 / fonc . 2019 . 00966](https://doi.org/10.3389/fonc.2019.00966). URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC6783516/>.
- [7] Andrea Bernardini et al. "Reconstruction of ovine axonal cytoarchitecture enables more accurate models of brain biomechanics". en. In: *Communications Biology* 5.1 (Oct. 2022). Pp. 1–14. ISSN: 2399-3642. DOI: [10 . 1038 / s42003 - 022 - 04052 - x](https://doi.org/10.1038/s42003-022-04052-x). URL: <https://www.nature.com/articles/s42003-022-04052-x>.

-
- [8] Christopher M Filley. "White Matter Structure and Function". In: *The Behavioral Neurology Of White Matter*. Ed. by Christopher M Filley. Oxford University Press, Sept. 2001, p. 0. ISBN: 978-0-19-513561-9. DOI: [10.1093/oso/9780195135619.003.0002](https://doi.org/10.1093/oso/9780195135619.003.0002). URL: <https://doi.org/10.1093/oso/9780195135619.003.0002>.
- [9] Michiel Kleinnijenhuis et al. *A semi-automated approach to dense segmentation of 3D white matter electron microscopy*. en. Mar. 2020. DOI: [10.1101/2020.03.19.979393](https://doi.org/10.1101/2020.03.19.979393). URL: <https://www.biorxiv.org/content/10.1101/2020.03.19.979393v1>.
- [10] Manuel Berning, Kevin M. Boergens, and Moritz Helmstaedter. "SegEM: Efficient Image Analysis for High-Resolution Connectomics". English. In: *Neuron* 87.6 (Sept. 2015). Pp. 1193–1206. ISSN: 0896-6273. DOI: [10.1016/j.neuron.2015.09.003](https://doi.org/10.1016/j.neuron.2015.09.003). URL: [https://www.cell.com/neuron/abstract/S0896-6273\(15\)00760-6](https://www.cell.com/neuron/abstract/S0896-6273(15)00760-6).
- [11] Yuxuan Huang, Zhongpan Qiu, and Zhijun Song. "3D reconstruction and visualization from 2D CT images". In: *2011 IEEE International Symposium on IT in Medicine and Education*. Vol. 2. Dec. 2011, pp. 153–157. DOI: [10.1109/ITiME.2011.6132078](https://doi.org/10.1109/ITiME.2011.6132078). URL: <https://ieeexplore.ieee.org/document/6132078>.
- [12] Jun Ma et al. "Segment anything in medical images". en. In: *Nature Communications* 15.1 (Jan. 2024), p. 654. ISSN: 2041-1723. DOI: [10.1038/s41467-024-44824-z](https://doi.org/10.1038/s41467-024-44824-z). URL: <https://www.nature.com/articles/s41467-024-44824-z>.
- [13] Mohammad Hesam Hesamian et al. "Deep Learning Techniques for Medical Image Segmentation: Achievements and Challenges". en. In: *Journal of Digital Imaging* 32.4 (Aug. 2019), pp. 582–596. ISSN: 1618-727X. DOI: [10.1007/s10278-019-00227-x](https://doi.org/10.1007/s10278-019-00227-x). URL: <https://doi.org/10.1007/s10278-019-00227-x>.
- [14] Philipp Kickingreder et al. "Automated quantitative tumour response assessment of MRI in neuro-oncology with artificial neural networks: a multicentre, retrospective study". In: *The Lancet Oncology* 20.5 (May 2019), pp. 728–740. ISSN: 1470-2045. DOI: [10.1016/S1470-2045\(19\)30098-1](https://doi.org/10.1016/S1470-2045(19)30098-1). URL: <https://www.sciencedirect.com/science/article/pii/S1470204519300981>.
- [15] Stanislav Nikolov et al. *Deep learning to achieve clinically applicable segmentation of head and neck anatomy for radiotherapy*. en. Jan. 2021. URL: <http://arxiv.org/abs/1809.04430>.
- [16] Yuxing Wang et al. "A systematic evaluation of computational methods for cell segmentation". In: *Briefings in Bioinformatics* 25.5 (Sept. 2024), bbae407. ISSN: 1477-4054. DOI: [10.1093/bib/bbae407](https://doi.org/10.1093/bib/bbae407). URL: <https://doi.org/10.1093/bib/bbae407>.
-

-
- [17] Alin-Florin Totorean et al. "Computational Fluid Dynamics Applications in Cardiovascular Medicine—from Medical Image-Based Modeling to Simulation: Numerical Analysis of Blood Flow in Abdominal Aorta". en. In: *Advances in Fluid Mechanics: Modelling and Simulations*. Ed. by Dia Zeidan et al. Singapore: Springer Nature, 2022, pp. 1–42. ISBN: 978-981-19143-8-6. DOI: [10.1007/978-981-19-1438-6_1](https://doi.org/10.1007/978-981-19-1438-6_1). URL: https://doi.org/10.1007/978-981-19-1438-6_1.
- [18] Jana Vaskovic. *Radiological anatomy and medical imaging*. en. URL: <https://www.kenhub.com/en/library/anatomy/medical-imaging-and-radiological-anatomy>.
- [19] Doc J. *How to Improve the Quality of Your MRI Images*. en-US. June 2021. URL: <https://www.3dbiology.com/improve-quality-of-mri/>.
- [20] Johanna Pitkänen et al. "Evaluating severity of white matter lesions from computed tomography images with convolutional neural network". en. In: *Neuroradiology* 62.10 (Oct. 2020), pp. 1257–1263. ISSN: 1432-1920. DOI: [10.1007/s00234-020-02410-2](https://doi.org/10.1007/s00234-020-02410-2). URL: <https://doi.org/10.1007/s00234-020-02410-2>.
- [21] Hyojune Kim et al. "Can deep learning reduce the time and effort required for manual segmentation in 3D reconstruction of MRI in rotator cuff tears?" en. In: *PLOS ONE* 17.10 (Oct. 2022). e0274075. ISSN: 1932-6203. DOI: [10.1371/journal.pone.0274075](https://doi.org/10.1371/journal.pone.0274075). URL: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0274075>.
- [22] Patrick C. Nahirney and Marie-Eve Tremblay. "Brain Ultrastructure: Putting the Pieces Together". In: *Frontiers in Cell and Developmental Biology* 9 (2021). ISSN: 2296-634X. URL: <https://www.frontiersin.org/articles/10.3389/fcell.2021.629503>.
- [23] Fabian Bongratz et al. "Vox2Cortex: Fast Explicit Reconstruction of Cortical Surfaces from 3D MRI Scans with Geometric Deep Neural Networks". en. In: *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. New Orleans, LA, USA: IEEE, June 2022, pp. 20741–20751. ISBN: 978-1-66546-946-3. DOI: [10.1109/CVPR52688.2022.02011](https://doi.org/10.1109/CVPR52688.2022.02011). URL: <https://ieeexplore.ieee.org/document/9878407/>.
- [24] Ali Abdollahzadeh et al. "DeepACSON automated segmentation of white matter in 3D electron microscopy". en. In: *Communications Biology* 4.1 (Feb. 2021). Pp. 1–14. ISSN: 2399-3642. DOI: [10.1038/s42003-021-01699-w](https://doi.org/10.1038/s42003-021-01699-w). URL: <https://www.nature.com/articles/s42003-021-01699-w>.
- [25] Beomsue Kim et al. "A brief review of non-invasive brain imaging technologies and the near-infrared optical bioimaging". In: *Applied Microscopy* 51.1 (June 2021), p. 9. ISSN: 2287-4445. DOI: [10.1186/s42649-021-00058-7](https://doi.org/10.1186/s42649-021-00058-7). URL: <https://doi.org/10.1186/s42649-021-00058-7>.
-

-
- [26] *Preparing samples for the electron microscope* — Science Learning Hub. URL: <https://www.sciencelearn.org.nz/resources/500-preparing-samples-for-the-electron-microscope>.
- [27] Mariella Sele et al. "Optimization of ultrastructural preservation of human brain for transmission electron microscopy after long post-mortem intervals". In: *Acta Neuropathologica Communications* 7.1 (Sept. 2019), p. 144. ISSN: 2051-5960. DOI: [10.1186/s40478-019-0794-3](https://doi.org/10.1186/s40478-019-0794-3). URL: <https://doi.org/10.1186/s40478-019-0794-3>.
- [28] Xiao-Bo Liu and Cynthia M. Schumann. "Optimization of electron microscopy for human brains with long-term fixation and fixed-frozen sections". In: *Acta Neuropathologica Communications* 2.1 (Apr. 2014), p. 42. ISSN: 2051-5960. DOI: [10.1186/2051-5960-2-42](https://doi.org/10.1186/2051-5960-2-42). URL: <https://doi.org/10.1186/2051-5960-2-42>.
- [29] Samantha J. Murray and Nadia L. Mitchell. "The Translational Benefits of Sheep as Large Animal Models of Human Neurological Disorders". In: *Frontiers in Veterinary Science* 9 (Feb. 2022), p. 831838. ISSN: 2297-1769. DOI: [10.3389/fvets.2022.831838](https://doi.org/10.3389/fvets.2022.831838). URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8886239/>.
- [30] Mriganka Sarmah, Arambam Neelima, and Heisnam Rohen Singh. "Survey of methods and principles in three-dimensional reconstruction from two-dimensional medical images". In: *Visual Computing for Industry, Biomedicine, and Art* 6.1 (July 2023), p. 15. ISSN: 2524-4442. DOI: [10.1186/s42492-023-00142-7](https://doi.org/10.1186/s42492-023-00142-7). URL: <https://doi.org/10.1186/s42492-023-00142-7>.
- [31] Santiago González Izard et al. "Nextmed: Automatic Imaging Segmentation, 3D Reconstruction, and 3D Model Visualization Platform Using Augmented and Virtual Reality". en. In: *Sensors* 20.10 (Jan. 2020). P. 2962. ISSN: 1424-8220. DOI: [10.3390/s20102962](https://doi.org/10.3390/s20102962). URL: <https://www.mdpi.com/1424-8220/20/10/2962>.
- [32] Yoann Barnouin et al. "Manual segmentation of individual muscles of the quadriceps femoris using MRI: A reappraisal". en. In: *Journal of Magnetic Resonance Imaging* 40.1 (2014). Pp. 239–247. ISSN: 1522-2586. DOI: [10.1002/jmri.24370](https://doi.org/10.1002/jmri.24370). URL: <https://onlinelibrary.wiley.com/doi/abs/10.1002/jmri.24370>.
- [33] Dongdong Meng et al. "3D reconstruction-oriented fully automatic multi-modal tumor segmentation by dual attention-guided VNet". en. In: *The Visual Computer* 39.8 (Aug. 2023), pp. 3183–3196. ISSN: 1432-2315. DOI: [10.1007/s00371-023-02965-0](https://doi.org/10.1007/s00371-023-02965-0). URL: <https://doi.org/10.1007/s00371-023-02965-0>.
- [34] Yahya Alzahrani and Boubakeur Boufama. "Biomedical Image Segmentation: A Survey". en. In: *SN Computer Science* 2.4 (May 2021), p. 310. ISSN: 2661-8907. DOI: [10.1007/s42979-021-00704-7](https://doi.org/10.1007/s42979-021-00704-7). URL: <https://doi.org/10.1007/s42979-021-00704-7>.
-

-
- [35] Jordan Fletcher. "Methods and Applications of 3D Patient-Specific Virtual Reconstructions in Surgery". en. In: *Biomedical Visualisation: Volume 11*. Ed. by Paul M. Rea. Advances in Experimental Medicine and Biology. Cham: Springer International Publishing, 2022, pp. 53–71. ISBN: 978-3-030-87779-8. DOI: [10.1007/978-3-030-87779-8_3](https://doi.org/10.1007/978-3-030-87779-8_3). URL: https://doi.org/10.1007/978-3-030-87779-8_3.
- [36] Ali Abdollahzadeh et al. "Automated 3D Axonal Morphometry of White Matter". en. In: *Scientific Reports* 9.1 (Apr. 2019). P. 6084. ISSN: 2045-2322. DOI: [10.1038/s41598-019-42648-2](https://doi.org/10.1038/s41598-019-42648-2). URL: <https://www.nature.com/articles/s41598-019-42648-2>.
- [37] Yann LeCun et al. "Gradient-Based Learning Applied to Document Recognition". en. In: (1998).
- [38] Ksenia Se. *Computer Vision's Great Leap Forward: From the 1990s to AlexNet*. en. May 2024. URL: <https://www.turingpost.com/p/cvhistory4>.
- [39] Shervin Minaee et al. *Image Segmentation Using Deep Learning: A Survey*. en. Nov. 2020. URL: <http://arxiv.org/abs/2001.05566>.
- [40] Farooq Alvi. *Deep Learning For Computer Vision: Essential Models and Practical Real-World Applications*. en-US. Nov. 2023. URL: <https://opencv.org/blog/deep-learning-with-computer-vision/>.
- [41] Nik Hasnida Awang Mustapa et al. "A Review on Deep Convolutional Neural Network Architectures for Medical Image Segmentation". en. In: *Proceedings of the 11th International Conference on Robotics, Vision, Signal Processing and Power Applications*. Ed. by Nor Muzlifah Mahyuddin, Nor Rizuan Mat Noor, and Harsa Amylia Mat Sakim. Singapore: Springer, 2022, pp. 973–978. ISBN: 9789811681295. DOI: [10.1007/978-981-16-8129-5_148](https://doi.org/10.1007/978-981-16-8129-5_148).
- [42] Xia Zhao et al. "A review of convolutional neural networks in computer vision". en. In: *Artificial Intelligence Review* 57.4 (Mar. 2024), p. 99. ISSN: 1573-7462. DOI: [10.1007/s10462-024-10721-6](https://doi.org/10.1007/s10462-024-10721-6). URL: <https://doi.org/10.1007/s10462-024-10721-6>.
- [43] *Convolutional Neural Network: A Complete Guide*. en-US. Jan. 2023. URL: <https://learnopencv.com/understanding-convolutional-neural-networks-cnn/>.
- [44] Meet Patel. *Understanding the Rectified Linear Unit (ReLU): A Key Activation Function in Neural Networks*. en. Apr. 2024. URL: <https://medium.com/@meetkp/understanding-the-rectified-linear-unit-relu-a-key-activation-function-in-neural-networks-28108fba8f07>.
- [45] Diego Unzueta. *Fully Connected Layer vs Convolutional Layer: Explained*. en. Oct. 2022. URL: <https://builtin.com/machine-learning/fully-connected-layer>.
-

-
- [46] Rikiya Yamashita et al. "Convolutional neural networks: an overview and application in radiology". en. In: *Insights into Imaging* 9.4 (Aug. 2018), pp. 611–629. ISSN: 1869-4101. DOI: [10.1007/s13244-018-0639-9](https://doi.org/10.1007/s13244-018-0639-9). URL: <https://doi.org/10.1007/s13244-018-0639-9>.
- [47] Sagar Joshi. *Understanding Pooling Layers: A Key to Better Neural Networks*. en-us. Oct. 2024. URL: <https://www.g2.com/articles/pooling-layers>.
- [48] Zoumana Kelta. *An Introduction to Convolutional Neural Networks: A Comprehensive Guide to CNNs in Deep Learning*. en. Nov. 2023. URL: <https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns>.
- [49] Lei Zhao and Zhonglin Zhang. "A improved pooling method for convolutional neural networks". en. In: *Scientific Reports* 14.1 (Jan. 2024). P. 1589. ISSN: 2045-2322. DOI: [10.1038/s41598-024-51258-6](https://doi.org/10.1038/s41598-024-51258-6). URL: <https://www.nature.com/articles/s41598-024-51258-6>.
- [50] Adrian Rosebrock. *Convolutional Neural Networks (CNNs) and Layer Types*. en-US. May 2021. URL: <https://pyimagesearch.com/2021/05/14/convolutional-neural-networks-cnns-and-layer-types/>.
- [51] *ML Practicum: Image Classification | Machine Learning*. en. URL: <https://developers.google.com/machine-learning/practica/image-classification/convolutional-neural-networks>.
- [52] Divam Gupta. *A Beginner's guide to Deep Learning based Semantic Segmentation using Keras*. en. June 2019. URL: <https://divamgupta.com/image-segmentation/2019/06/06/deep-learning-semantic-segmentation-keras.html>.
- [53] Premanand S. *A Comprehensive Guide to UNET Architecture | Mastering Image Segmentation*. en. Aug. 2024. URL: <https://www.analyticsvidhya.com/blog/2023/08/unet-architecture-mastering-image-segmentation/>.
- [54] Jonathan Long, Evan Shelhamer, and Trevor Darrell. *Fully Convolutional Networks for Semantic Segmentation*. en. Mar. 2015. URL: <http://arxiv.org/abs/1411.4038>.
- [55] Naoki Shibuya. *KiKaBeN - FCN: Fully Convolutional Networks (2014)*. en. Sept. 2022. URL: <https://kikaben.com/fcn-2014/>.
- [56] *UpsamplingBilinear2d — PyTorch 2.4 documentation*. URL: <https://pytorch.org/docs/stable/generated/torch.nn.UpsamplingBilinear2d.html>.
- [57] Jason Brownlee. *How to use the UpSampling2D and Conv2DTranspose Layers in Keras*. en-US. June 2019. URL: <https://machinelearningmastery.com/upsampling-and-transpose-convolution-layers-for-generative-adversarial-networks/>.
- [58] Kaiming He et al. *Deep Residual Learning for Image Recognition*. en. Dec. 2015. URL: <http://arxiv.org/abs/1512.03385>.
-

-
- [59] Enes Zvornicanin. *What Does Backbone Mean in Neural Networks? | Baeldung on Computer Science*. en-US. Oct. 2022. URL: <https://www.baeldung.com/cs/neural-network-backbone>.
- [60] Yashowardhan Shinde. *How to code your ResNet from scratch in Tensorflow?* en. Aug. 2021. URL: <https://www.analyticsvidhya.com/blog/2021/08/how-to-code-your-resnet-from-scratch-in-tensorflow/>.
- [61] Huili Yu. *A Detailed Introduction to ResNet and Its Implementation in PyTorch*. en. Oct. 2022. URL: <https://medium.com/@freshtechyy/a-detailed-introduction-to-resnet-and-its-implementation-in-pytorch-744b13c8074a>.
- [62] Y. Bengio, P. Simard, and P. Frasconi. "Learning long-term dependencies with gradient descent is difficult". In: *IEEE Transactions on Neural Networks* 5.2 (Mar. 1994). Pp. 157–166. ISSN: 1941-0093. DOI: [10.1109/72.279181](https://doi.org/10.1109/72.279181). URL: <https://ieeexplore.ieee.org/document/279181/?arnumber=279181>.
- [63] Sk Mahmudul Hassan and Arnab Kumar Maji. "Plant Disease Identification Using a Novel Convolutional Neural Network". In: *IEEE Access* 10 (2022). Pp. 5390–5401. ISSN: 2169-3536. DOI: [10.1109/ACCESS.2022.3141371](https://doi.org/10.1109/ACCESS.2022.3141371). URL: <https://ieeexplore.ieee.org/document/9674894>.
- [64] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. *U-Net: Convolutional Networks for Biomedical Image Segmentation*. en. May 2015. URL: <http://arxiv.org/abs/1505.04597>.
- [65] Reza Azad et al. *Medical Image Segmentation Review: The success of U-Net*. en. Nov. 2022. URL: <http://arxiv.org/abs/2211.14830>.
- [66] Harshall Lamba. *Understanding Semantic Segmentation with UNET*. en. Feb. 2019. URL: <https://towardsdatascience.com/understanding-semantic-segmentation-with-unet-6be4f42d4b47>.
- [67] Carlos García Rodríguez, Jordi Vitrià, and Oscar Mora. "Uncertainty-Based Human-in-the-Loop Deep Learning for Land Cover Segmentation". en. In: *Remote Sensing* 12.22 (Jan. 2020). P. 3836. ISSN: 2072-4292. DOI: [10.3390/rs12223836](https://doi.org/10.3390/rs12223836). URL: <https://www.mdpi.com/2072-4292/12/22/3836>.
- [68] Huimin Huang et al. "UNet 3+: A Full-Scale Connected UNet for Medical Image Segmentation". en. In: *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. Barcelona, Spain: IEEE, May 2020, pp. 1055–1059. ISBN: 978-1-5090-6631-5. DOI: [10.1109/ICASSP40776.2020.9053405](https://doi.org/10.1109/ICASSP40776.2020.9053405). URL: <https://ieeexplore.ieee.org/document/9053405/>.
-

-
- [69] Taukir Alam et al. "Improving Breast Cancer Detection and Diagnosis through Semantic Segmentation Using the Unet3+ Deep Learning Framework". en. In: *Biomedicines* 11.6 (June 2023). P. 1536. ISSN: 2227-9059. DOI: [10.3390/biomedicines11061536](https://doi.org/10.3390/biomedicines11061536). URL: <https://www.mdpi.com/2227-9059/11/6/1536>.
- [70] Olivia Guy-Evans. *What Is White Matter In The Brain*. en-US. Sept. 2023. URL: <https://www.simplypsychology.org/what-is-white-matter-in-the-brain.html>.
- [71] Olivia Guy-Evans. *Myelin Sheath*. en-US. Nov. 2023. URL: <https://www.simplypsychology.org/myelin-sheath.html>.
- [72] Alyssa Soles et al. "Extracellular Matrix Regulation in Physiology and in Brain Disease". en. In: *International Journal of Molecular Sciences* 24.8 (Jan. 2023). P. 7049. ISSN: 1422-0067. DOI: [10.3390/ijms24087049](https://doi.org/10.3390/ijms24087049). URL: <https://www.mdpi.com/1422-0067/24/8/7049>.
- [73] Tian Yuan, Li Shen, and Daniele Dini. "Porosity-permeability tensor relationship of closely and randomly packed fibrous biomaterials and biological tissues: Application to the brain white matter". In: *Acta Biomaterialia* 173 (Jan. 2024), pp. 123–134. ISSN: 1742-7061. DOI: [10.1016/j.actbio.2023.11.007](https://doi.org/10.1016/j.actbio.2023.11.007). URL: <https://www.sciencedirect.com/science/article/pii/S1742706123006542>.
- [74] *Albumentations: Efficient Image Augmentation Library for Machine Learning*. en. URL: <https://albumentations.ai>.
- [75] Connor Shorten and Taghi M. Khoshgoftaar. "A survey on Image Data Augmentation for Deep Learning". In: *Journal of Big Data* 6.1 (July 2019), p. 60. ISSN: 2196-1115. DOI: [10.1186/s40537-019-0197-0](https://doi.org/10.1186/s40537-019-0197-0). URL: <https://doi.org/10.1186/s40537-019-0197-0>.
- [76] Behnaz Abdollahi, Naofumi Tomita, and Saeed Hassanpour. "Data Augmentation in Training Deep Learning Models for Medical Image Analysis". en. In: *Deep Learners and Deep Learner Descriptors for Medical Applications*. Ed. by Loris Nanni et al. Cham: Springer International Publishing, 2020, pp. 167–180. ISBN: 978-3-030-42750-4. DOI: [10.1007/978-3-030-42750-4_6](https://doi.org/10.1007/978-3-030-42750-4_6). URL: https://doi.org/10.1007/978-3-030-42750-4_6.
- [77] *Find circles using circular Hough transform - MATLAB imfindcircles*. URL: <https://www.mathworks.com/help/images/ref/imfindcircles.html>.
- [78] David Viertel. *David-Viertel/DCNN-Segmentation-of-Sheep-WM*. Sept. 2024. URL: <https://github.com/David-Viertel/DCNN-Segmentation-of-Sheep-WM>.
- [79] R. Adams and L. Bischof. "Seeded region growing". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 16.6 (June 1994). Pp. 641–647. ISSN: 1939-3539. DOI: [10.1109/34.295913](https://doi.org/10.1109/34.295913). URL: <https://ieeexplore.ieee.org/document/295913>.
-

-
- [80] Ali Abdollahzadeh. *aAbdz/ACSON*. June 2019. URL: <https://github.com/aAbdz/ACSON>.
- [81] *Materialise Mimics | 3D Medical Image Processing Software*. en. URL: <https://www.materialise.com/en/healthcare/mimics-innovation-suite/mimics>.
- [82] *Morphologically dilate image - MATLAB imdilate*. URL: https://www.mathworks.com/help/images/ref/imdilate.html#responsive_offcanvas.
- [83] *Fill image regions and holes - MATLAB imfill*. URL: <https://www.mathworks.com/help/images/ref/imfill.html>.
- [84] Kostadin Dabov et al. "Image Denoising by Sparse 3-D Transform-Domain Collaborative Filtering". In: *IEEE Transactions on Image Processing* 16.8 (Aug. 2007). Pp. 2080–2095. ISSN: 1941-0042. DOI: [10.1109/TIP.2007.901238](https://doi.org/10.1109/TIP.2007.901238). URL: <https://ieeexplore.ieee.org/document/4271520/?arnumber=4271520>.
- [85] *bm3d: BM3D for correlated noise*. URL: <https://webpages.tuni.fi/foi/GCF-BM3D/>.
- [86] José A. Rodríguez-Rodríguez et al. "The Effect of Noise and Brightness on Convolutional Deep Neural Networks". en. In: *Pattern Recognition. ICPR International Workshops and Challenges*. Ed. by Alberto Del Bimbo et al. Cham: Springer International Publishing, 2021, pp. 639–654. ISBN: 978-3-030-68780-9. DOI: [10.1007/978-3-030-68780-9_49](https://doi.org/10.1007/978-3-030-68780-9_49).
- [87] *Morphologically open image - MATLAB imopen*. URL: <https://www.mathworks.com/help/images/ref/imopen.html>.
- [88] *Adaptive image threshold using local first-order statistics - MATLAB adaptthresh*. URL: https://www.mathworks.com/help/images/ref/adaptthresh.html#bu2a9fp-1_seealso.
- [89] Derek Bradley and Gerhard Roth. "Adaptive Thresholding using the Integral Image". In: *Journal of Graphics Tools* 12.2 (Jan. 2007). Pp. 13–21. ISSN: 1086-7651. DOI: [10.1080/2151237X.2007.10129236](https://doi.org/10.1080/2151237X.2007.10129236). URL: <https://doi.org/10.1080/2151237X.2007.10129236>.
- [90] *Adjust image intensity values or colormap - MATLAB imadjust*. URL: <https://www.mathworks.com/help/images/ref/imadjust.html>.
- [91] Shaeke Salman and Xiuwen Liu. *Overfitting Mechanism and Avoidance in Deep Neural Networks*. en. Jan. 2019. URL: <http://arxiv.org/abs/1901.06566>.
- [92] Jacob Murel and Eda Kavlakoglu. *What is a confusion matrix? | IBM*. en. Jan. 2024. URL: <https://www.ibm.com/topics/confusion-matrix>.
-

- [93] Nghi Huynh. *Understanding Evaluation Metrics in Medical Image Segmentation*. en. Mar. 2023. URL: https://medium.com/@nghihuynh_37300/understanding-evaluation-metrics-in-medical-image-segmentation-d289a373a3f.
- [94] *Accuracy vs. precision vs. recall in machine learning: what's the difference?* en. URL: <https://www.evidentlyai.com/classification-metrics/accuracy-precision-recall>.
- [95] Vaibhav Jayaswal. *Performance Metrics: Confusion matrix, Precision, Recall, and F1 Score*. en. Sept. 2020. URL: <https://towardsdatascience.com/performance-metrics-confusion-matrix-precision-recall-and-f1-score-a8fe076a2262>.
- [96] Svetlana Pavlitskaya, Joël Oswald, and J. Marius Zöllner. *Measuring Overfitting in Convolutional Neural Networks using Adversarial Perturbations and Label Noise*. en. Sept. 2022. URL: <http://arxiv.org/abs/2209.13382>.
- [97] Sebastian Otálora et al. "Staining Invariant Features for Improving Generalization of Deep Convolutional Neural Networks in Computational Pathology". English. In: *Frontiers in Bioengineering and Biotechnology* 7 (Aug. 2019). ISSN: 2296-4185. DOI: [10.3389/fbioe.2019.00198](https://doi.org/10.3389/fbioe.2019.00198). URL: <https://www.frontiersin.org/journals/bioengineering-and-biotechnology/articles/10.3389/fbioe.2019.00198/full>.
- [98] Nina Jacobsen et al. "Analysis of intensity normalization for optimal segmentation performance of a fully convolutional neural network". eng. In: *Zeitschrift Fur Medizinische Physik* 29.2 (May 2019), pp. 128–138. ISSN: 1876-4436. DOI: [10.1016/j.zemedi.2018.11.004](https://doi.org/10.1016/j.zemedi.2018.11.004).
- [99] Haoran Lu et al. "Half-UNet: A Simplified U-Net Architecture for Medical Image Segmentation". In: *Frontiers in Neuroinformatics* 16 (June 2022), p. 911679. ISSN: 1662-5196. DOI: [10.3389/fninf.2022.911679](https://doi.org/10.3389/fninf.2022.911679). URL: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC9218935/>.

APPENDIX A: AI-ASSISTED WRITING DECLARATION

I used BingAI to to reword some sentences. I have the habit of writing overlong long sentences, and sometimes I give it to bingAI to ask if it think the sentence is too long. Usually it shortens it for me and then I will adapt its answer to my text.

I occasionally used BingAI as a tool to find and summarise papers, to make the researching go faster. I would usually ask prompts such as "Can you find me papers that explicitly discuss ..." or "Can you tell me if this [link] paper discusses this"

I also often use BingAI to help me find synonyms for a word or a phrase.

Lastly, I will frequently ask BingAI if a sentence I wrote or a word I used is appropriate for a thesis